IA-Generativa-e-LLMs-Para-Processamento-de-Linguagem-Natural

-> IA generativa é um tipo de inteligência artificial capaz de criar conteúdo novo a partir de um comando ou de exemplos que recebeu durante seu treinamento.

-> Processamento de Linguagem Natural (PLN) é um campo de estudo e aplicação que se situa na interseção da ciência da computação, inteligência artificial e linguística. O objetivo do PLN é capacitar os computadores para entender, interpretar, manipular e responder a linguagem humana de uma forma útil e significativa.

- Ou seja, IA generativa é uma categoria ampla de inteligência artificial que cria conteúdo novo.
- LLM (Large Language Model) é um tipo específico de modelo de IA, especializado em trabalhar com linguagem.
- PLN → área da computação/IA voltada para a linguagem humana.

Qual a diferença entre Transformers e LLMs?

-> Transformers é uma arquitetura de rede neural. É uma rede neural artificial é um modelo de IA inspirado, de forma simplificada, no funcionamento dos neurônios do cérebro humano. Ela aprende analisando muitos exemplos e encontrando padrões. Ele é o "motor" que permite que a IA compreenda e gere linguagem de forma eficiente.

Modelos como GPT3, GPT4, BERT e outros são baseados na arquitetura Transformer

->Modelos como RNNs e LSTMs tinham dificuldade em manter e utilizar informações de partes anteriores de uma sequência, especialmente se a sequência fosse longa. Os Transformers, com seu mecanismo de atenção, podem associar palavras distantes de forma eficiente, permitindo que eles capturem contexto e dependências de longo alcance.

-> Ao contrário das RNNs, que processam sequências de forma serial, os Transformers podem processar todos os tokens de uma sequência simultaneamente. Isso não apenas acelera o treinamento, mas também melhora a capacidade do modelo de aprender padrões complexos.
-> Enquanto modelos como Word2Vec e GloVe geram um único embedding (vetor) para cada palavra, independentemente do contexto, os Transformers produzem embeddings contextualizados. Isso significa que a representação vetorial de uma palavra pode variar dependendo do seu contexto na frase, permitindo nuances semânticas mais ricas.

-> **Hugging Face** é uma **plataforma onde desenvolvedores podem encontrar, compartilhar e usar modelos de Inteligência Artificial**, principalmente **LLMs (Large Language Models)**. Ou seja, a **"loja" mundial onde todo mundo baixa modelos de IA pré-treinados**.

Mod 4 – 05:

- **GPT2Tokenizer (O Tradutor):** O **Tokenizer** é quem pega a frase que você digita e fatia em pedacinhos chamados **tokens**, convertendo cada token em um número específico (ID).
- **GPT2LMHeadModel (O Cérebro Gerador):** É a camada final da rede neural responsável por prever qual é a próxima palavra mais provável de uma frase.

-> A tokenizer ajuda a transformar (converter) as palavras em representações numéricas dentro daquele modelo que foi treinado.
Os Ids só faz sentido com o tokenizador. Ex: prompt = 'What is Artificial Intelligence'
E saiu: tensor([[2061, 318, 35941, 9345]])
-> No **Transformers** como uma **caixa de ferramentas** para usar modelos de IA. Dentro dela, existem três ferramentas principais:

- **Pipeline** → **usar** um modelo de IA facilmente.
- **Trainer** → **treinar** um modelo de IA.
- **generate** → **gerar texto** com um modelo de IA.

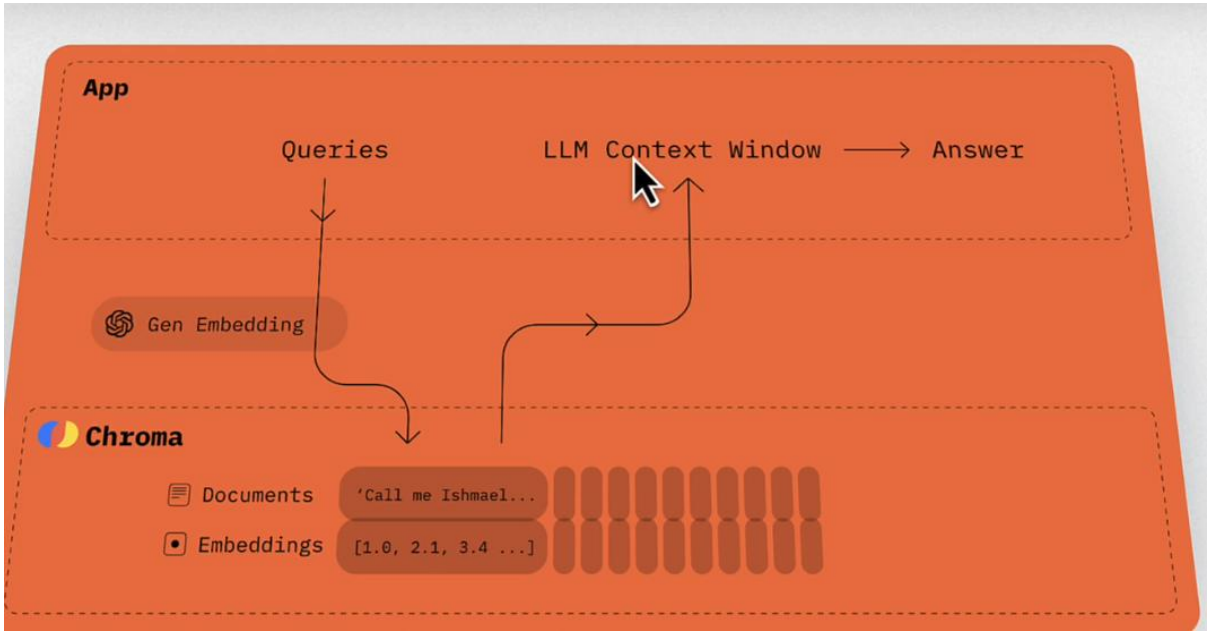
Mod 5 – 03

-> Bancos de dados vetoriais: são sistemas projetados para armazenar, gerenciar e manipular dados em formato vetorial .
Eles são especialmente úteis em contextos onde é necessário realizar buscas baseadas em similaridade ou padrões em grandes conjuntos de dados.

07:

I. **python-dotenv** carrega sua chave da OpenAI.
II. **pypdf** extrai o texto dos PDFs da pasta.
III. O texto é dividido em pedaços e convertido em vetores pelo modelo de embedding via **openai** (com auxílio do **tiktoken**).
IV. Esses vetores são armazenados no **chromadb**.
V. O **langchain** e **langchain-community** orquestram tudo: recebem sua pergunta, buscam o trecho certo no ChromaDB e enviam para o GPT responder.
VI. O **watermark** documenta o ambiente de execução no notebook.

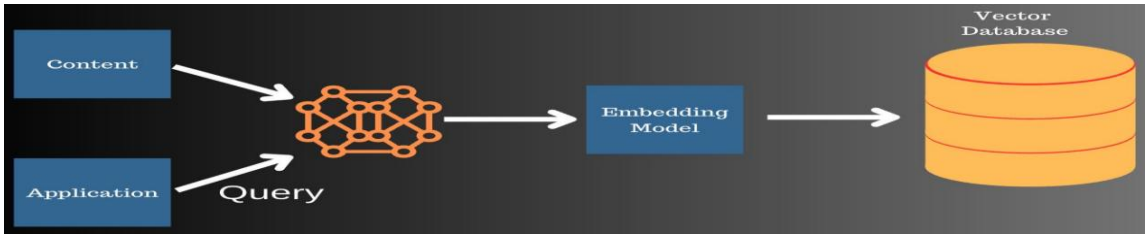
O OpenAIEmbeddings é uma funcionalidade que permite obter representações vetoriais (embeddings) de texto, que são úteis para várias tarefas de processamento de linguagem natural, como comparação de semelhança de texto, agrupamento e classificação.



-> O LLM não entende se é resposta certa ou errada, ele entende qual a probabilidade de a próxima palavra ser essa ou aquela.

13: Por Que Usamos Bancos de Dados Vetoriais com LLMs?

- **Estrutura e organização eficiente:**
  Organizam grandes quantidades de dados para que o LLM consiga encontrar informações facilmente.
- **Recuperação rápida:**
  Permitem que o LLM acesse informações muito rapidamente para responder em tempo real.
- **Busca semântica e contextual:**
  Buscam pelo **significado** da pergunta, e não apenas pelas palavras exatas, gerando respostas mais inteligentes.
- **Escalabilidade:**
  Continuam funcionando bem mesmo quando a quantidade de dados aumenta.
- **Compatibilidade com aprendizado de máquina:**
  Facilitam o treinamento e a melhoria contínua dos modelos de IA.
- **Análise avançada:**
  Permitem analisar grandes volumes de dados para extrair informações e padrões importantes.



<u>Mod 6 – 02: Transfer Learning ou Transferência de Aprendizado:</u>

-> É uma técnica já estabelecida em IA que permite que um modelo desenvolvido para uma tarefa seja reutilizado como ponto de partida para outra tarefa relacionada.

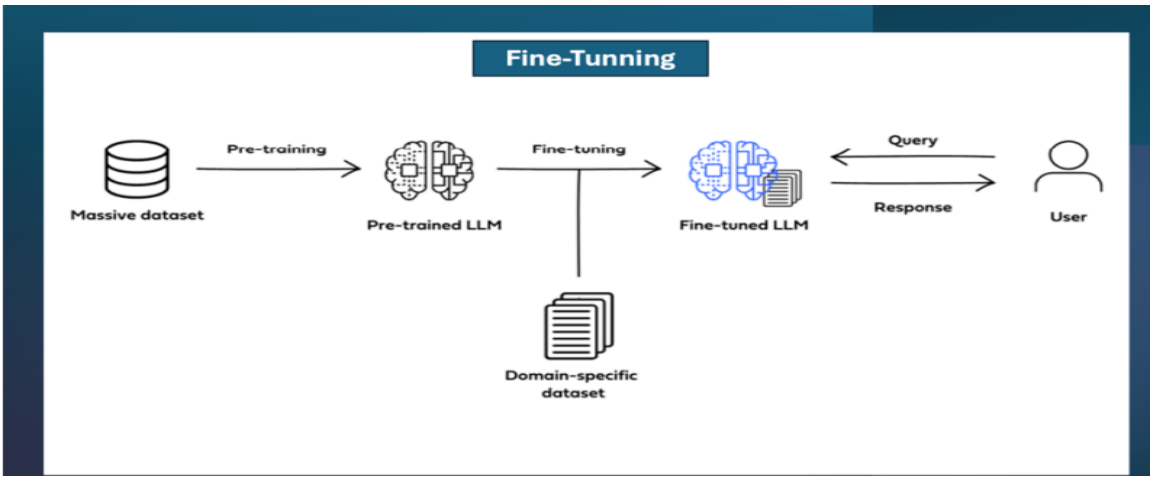
<u>05: FiNE - TUNING:</u>

Fine-Tuning (*Ajuste Fino*): Você pega esse modelo pronto e "inteligente" e treina ele um pouquinho mais, usando apenas um conjunto pequeno de dados da **sua área específica**

É um processo de ajuste fino ou refinamento de um modelo pré-treinado. Esse processo envolve a reconfiguração do modelo para que ele se adapte melhor a uma tarefa específica ou conjunto de dados.

-Economia de tempo e dinheiro; ensina um estilo, padrão ou tarefa específica; domínio de termos técnicos.

- O desafio de evitar o sobreajuste (*Overfitting*)



Etapa 1 (Esquerda): O Pré-treinamento (*Pre-training*)

- **Massive dataset (o cilindro de banco de dados):** Representa um volume gigantesco de textos da internet aberta (artigos, livros, notícias, Wikipedia).
- **Pre-trained LLM (o primeiro cérebro):** É o modelo que aprendeu o idioma, gramática e raciocínio básico a partir desses dados massivos. Ele é inteligente, mas ainda é generalista (sabe de tudo um pouco).

🔹 Etapa 2 (Centro): O Ajuste Fino (*Fine-tuning*)

- **Domain-specific dataset (as folhas de papel embaixo):** São os dados focados na sua área de negócio (por exemplo: contratos jurídicos da sua empresa, prontuários médicos ou transcrições de suporte ao cliente).
- **Fine-tuning (a seta do meio):** É o processo de alimentar o modelo pré-treinado com esses dados específicos.
- **Fine-tuned LLM (o cérebro azul com documentos):** É o produto final. Ele mantém toda a inteligência do cérebro inicial, mas agora sabe responder exatamente com os termos, estilo e regras daquele domínio específico.

- Etapa 3 (Direita): A Interação com o Usuário
  - **User (Usuário) ⇄ Fine-tuned LLM:**
    - O usuário envia uma pergunta ou instrução (**Query**).
    - O modelo especialista responde (**Response**) com precisão muito maior do que um modelo comum conseguiria.

06 - 10:

-> Não apagar o que o modelo já aprendeu no pré-treino (o chamado esquecimento catastrófico) e não deixar o modelo apenas decorar os novos dados (overfitting).
-> O segredo do *Fine-Tuning* não é treinar o máximo possível, mas sim treinar de forma suave (baixa taxa de aprendizado), focada (camadas específicas) e vigiada (regularização e monitoramento contínuo).

11: Parameter Efficient Fine- Tning:

->Essa metodologia é particularmente relevante para modelos grandes e complexos, como os baseados em Transformer em tarefas de Processamento de Linguagem Natural (PLN). O objetivo do PEFT é obter um bom desempenho em tarefas específicas, sem a necessidade de reajustar extensivamente todos os parâmetros do modelo, o que pode ser computacionalmente caro e demorado.

12 - Low-Rank Adaptation (LoRA):

- ->**Vercel AI SDK:** Adaptador em nível de **código/software** (uma biblioteca que conecta seu app a diferentes APIs).
- **LoRA:** Adaptador em nível de **rede neural** (modifica o comportamento matemático interno do modelo).
  -> Ele altera os números (pesos) internos que definem o comportamento da inteligência artificial.

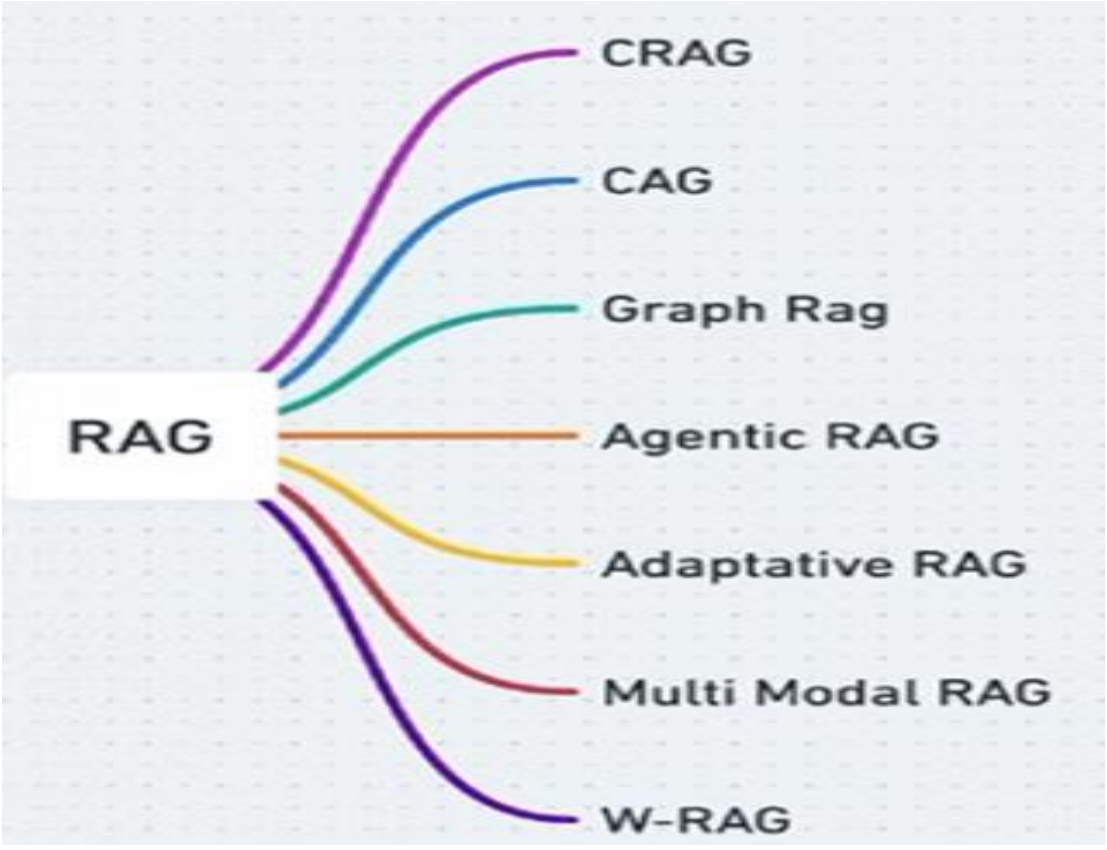
13: Quantized Low-Rank Adaptation (QLoRA)

-> Em QLoRA, a decomposição de baixo posto é utilizada para limitar o número de parâmetros que precisam ser treinados, enquanto a

quantização (QLoRA) é aplicada para reduzir ainda mais o consumo de memória e o custo computacional.
-> Quantização é o processo de "arredondar" os números de um modelo para que ele ocupe muito menos memória e rode mais rápido.

15: RAG - Retrieval-Augmented Generation (ou Geração Aumentada de Recuperação)

->É o processo de otimizar a saída de um grande modelo de linguagem (LLM), de forma que ele faça referência a uma base de conhecimento confiável fora das suas fontes de dados de treinamento antes de gerar uma resposta.
-> É uma abordagem econômica para melhorar a produção do LLM, de forma que ele permaneça relevante, preciso e útil em vários contextos.
->O objetivo é criar bots que possam responder às perguntas dos usuários em vários contextos, utilizando-se de fontes de conhecimento confiáveis.



Mod 7 – 06: Descrição dos Principais Pacotes Python Usados no Fine-Tuning de LLMs

1. transformers — O Cérebro (O Modelo)

- **O que faz:** É a biblioteca principal da Hugging Face. É ela quem faz o download do modelo (como o Llama) e gerencia a inteligência dele (arquitetura, atenção e pesos).
- **Na prática:** Sem ela, você não tem o modelo pré-treinado.

2. bitsandbytes — O Compactador (Economizador de Memória)

- **O que faz:** Ele "esmaga" o tamanho dos números do modelo de 16 bits para 4 bits (técnica chamada de **Quantização**). É graças a ele que conseguimos carregar um LLM gigante em uma GPU comum.

3. peft — O "Post-it" Inteligente (Técnica LoRA)

- **O que faz:** *Parameter-Efficient Fine-Tuning*. Em vez de retreinar todos os 7 bilhões de parâmetros (o que demoraria dias e exigiria supercomputadores), o PEFT **congela o modelo original** e treina apenas uma folha de anotações fininha por cima (os adaptadores LoRA, menos de 1% do modelo).

4. accelerate — O Maestro do Hardware

- **O que faz:** Gerencia a comunicação entre o código e a GPU. Ele decide como alocar a memória de vídeo, fazer cálculos em precisão mista (FP16/BF16) e otimizar o fluxo de dados.

07:

1.trl (Transformer Reinforcement Learning) — O Treinador Pronto

- **O que faz:** É a biblioteca especializada em conduzir o treino de LLMs.
  Ele fornece a função SFTTrainer. Em vez de você ter que programar 50 linhas de código matemático para rodar o loop de treino na mão, o trl faz todo o processo de ajuste fino com **apenas 1 comando**.

2. Gradio — A Interface Visual

- **O que faz:** Cria telas e aplicativos web para modelos em poucas linhas de Python, sem você precisar mexer com HTML ou CSS.

3. protobuf (Protocol Buffers) — O Empacotador dos Bastidores

- **O que faz:** É uma tecnologia criada pelo Google para salvar e transmitir dados estruturados de forma ultraleve e compacta. Quando o modelo e o tokenizador são salvos em disco, o protobuf garante que eles fiquem em um formato binário leve para carregar rápido.

4. SciPy — A Calculadora Científica

- **O que faz:** Biblioteca clássica do Python para matemática avançada, estatística e álgebra linear.

08: O foco é como os textos chegam até o modelo (dados e tradução para números):

1. datasets

- **O que faz:** Biblioteca da Hugging Face feita para carregar e organizar bases de dados com máxima velocidade, sem consumir toda a memória RAM do computador.

2. tokenizers — O Tradutor de Palavras para Números

- **O que faz:** Transforma texto humano em números inteiros (IDs de tokens).

3. SentencePiece — O Fatiador Inteligente de Palavras

- **O que faz:** Tecnologia desenvolvida pela Google para quebrar textos em "subpalavras" (pedacinhos de palavras), sem depender apenas de espaços em branco.

Conexão prática entre os três:

O **datasets** busca o texto na planilha; o **SentencePiece** define como fatiar as palavras de forma inteligente; e o **tokenizers** converte essas fatias em números para a GPU calcular.

Diferença entre SetencePiece x Chunks:

--> SentencePiece (Escala MICRO — Fatias de Letras)

- **O que fatia:** Uma única palavra em pedacinhos de letras (**subpalavras**).

--> Chunks (Escala MACRO — Fatias de Parágrafos)

- **O que fatia:** Um documento ou livro inteiro em **blocos de texto**.

Ex: um pdf se eu cortar em várias parte é chunks (blocos). Para o pc conseguir guardar esses chunks guardar esses chunks no banco de dados vetorial ou fazer o LLM ler a resposta, o **SentencePiece** entra em ação para traduzir cada palavra desses chunks em números (**tokens**).

- SIMILARIDADE x SEMÂNTICO:
  Léxico: *"Quero exatamente essas palavras/termos iguais."*
- **Semântico:** *"Quero textos que falem sobre este mesmo assunto/significado, mesmo que usem palavras diferentes."*
- **Similaridade:** É a pontuação (o cálculo) que diz o quão perto ou parecido um texto está do outro.

-> Na computação/IA, eles **não** chamam palavra exata de "similaridade". Eles chamam de **LÉXICA** (ou busca por palavra-chave).
-> HÍBRIDO: a busca por palavras exatas (letras iguais) + semântico e similaridade.

MOD 9 – 08 LLM T-5 Open Souce: Nesse mód 9 está se referindo sobre: Realizar o ajuste fino (*fine-tuning*) supervisionado de um modelo de linguagem open-source (google/flan-t5-base) para criar um assistente capaz de responder dúvidas e perguntas sobre Direito.

-> FLAN-T5 Base é um **modelo de linguagem (LLM)** criado pela Google para entender e gerar textos.
->Depois para trabalhar com LLM, precisa do tokenizador (conjunto de regras para processar os dados de texto e entregar o LLM T-5).
Cap 13: Métrica de avaliação do ajuste fino
-> Quer saber se o modelo acertou os termos essenciais? Use **ROUGE-1** / **ROUGE-2**.

Quer saber se manteve o sentido e a estrutura natural das frases? Olhe o **ROUGE-L** (ou **ROUGE-W**).

- ROUGE-1: Palavras soltas (unigramas). Vê se as palavras-chave principais apareceram.
  • ROUGE-2: Pares de palavras juntos (bigramas, ex: *"inteligência artificial"*).
- ROUGE-L: As palavras não precisam estar coladas uma na outra, mas precisam estar na mesma ordem temporal.
- ROUGE-W: É = ao Rouge-L, mas dá pontuação maior se as palavras estiverem juntas.
- ROUGE-S: Pega pares de palavras na ordem certa, permitindo outras palavras no meio. (ex: na frase *"o gato preto dormiu"*, avalia o par *"gato ... dormiu"*). Mede a coesão.

CAP 10 – LLM FALCON-7b (bilhões de parâmetros) – chatboot atendimento ao cliente - PREPARAÇÃO DO MODELO

10:

-> Quantização serve para colocar o modelo na memória.
-> Para não sobrecarregar a memória da GPU (VRAM) e preservar o conhecimento já adquirido pelo modelo, por isso congela os pesos.

1. O modelo base fica congelado (imutável). Nenhuma matriz original é alterado.
2. O LoRA insere pequenas matrizes adaptadoras ao lado das camadas originais.
3. No treinamento, apenas esses adaptadores (menos de 1% do modelo) recebem gradientes e são atualizados com os dados em português do seu chatbot.

-> Quando o modelo está aprendendo, entrega uma matriz numérica, ele começa a etapa de multiplicação de matrizes, como é um modelo de arquitetura transformers. Até que chega na previsão = resultado (forward_pass). Pega a previsão e compara com a valor real para calcular o erro.
Quando tiver o erro, aplica blackard pass, que é de fato o cálculo da derivada.

Quando o **checkpoint de gradientes** está habilitado, o modelo armazena apenas alguns pontos de verificação, em vez de todos os valores intermediários da passagem para frente (*forward pass*). Na passagem para trás (*backward pass*), os valores não armazenados são recomputados a partir desses pontos. Isso reduz o uso de memória da GPU, mas aumenta o tempo de computação devido à recomputação dos valores. Essa técnica é útil para treinar modelos muito grandes que não caberiam na memória da GPU.
14-Adaptadores LoRa:

- r (Quantidade de Parâmetros Adaptativos) = TAMANHO/CAPACIDADE DO ADAPTADOR.

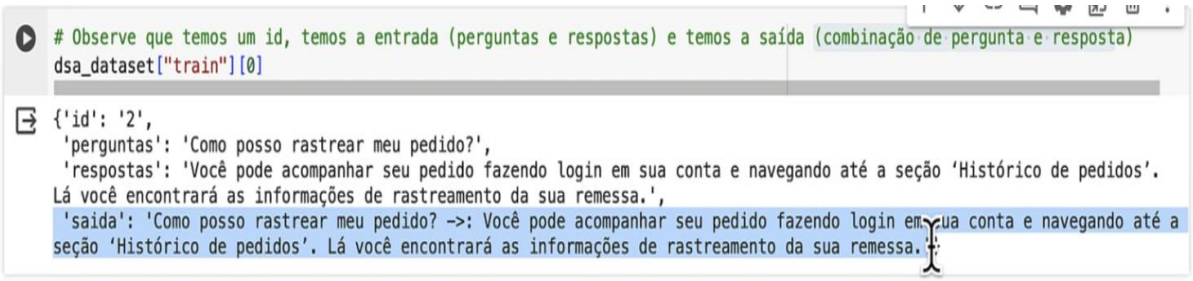
- lora_alpha (Intensidade do Aprendizado)

- lora_dropout (Prevenção de Decoreba): Proteção contra *overfitting* (memorização).

-bias (Ajuste de Viés): Define se os termos constantes de viés das camadas serão treinados.

- task_type (Objetivo do Modelo)

17:



*id, entrada (perguntas e respostas), saída (combinação de pergunta e resposta. Está ensinando o modelo, para ter respostas coerentes de acordo com a pergunta. (a sequência).
-> Depois vai tokenizar os dados para converter as palavras em formatos numéricos.
->Definindo os argumentos do treino, usa o pad para poder complementar o tamanho da matriz.

22 métrica bleu:

-> O **BLEU** é uma métrica usada em Inteligência Artificial para avaliar a qualidade de textos gerados automaticamente (ex: tradução ou resumos). Ela calcula a nota **comparando o texto da máquina com um texto de referência feito por humanos**.
Bleu faz a tradução, ex: tradução da língua exata, mas nosso o modelo gerou texto? Então isso também é uma tradução.

Mód12 - Criando aplicações com LangChain:
-> LangChain é uma biblioteca (framework), escrita principalmente em **Python** (também existe para JavaScript/TypeScript), que facilita criar aplicações com modelos de IA, como o ChatGPT. (construir aplicações com LLMs). Componentes centrais:

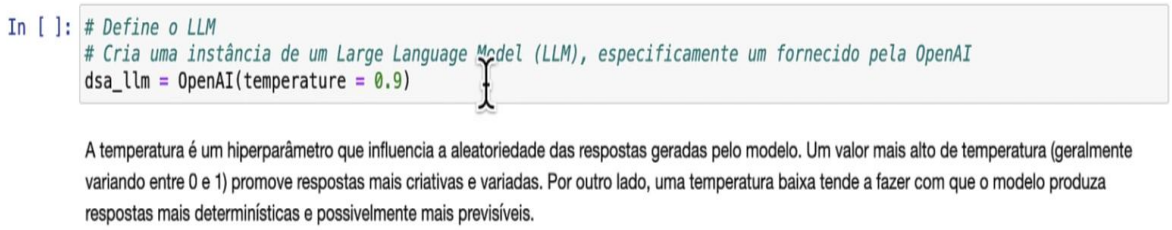
- **Abstrações para LLMs:** Interface consistente para conectar diferentes provedores (OpenAI, Anthropic, Cohere).
- **Chains:** Sequências de operações que combinam LLMs e outras utilidades para processar entradas e saídas.
- **Agents:** Abstrações autônomas que usam o LLM para decidir dinamicamente quais ferramentas ou cadeias executar.
- **Memory:** Guarda o histórico de interações e estados intermediários entre cadeias e agentes.
- **Tools:** Integrações com APIs e ferramentas externas (ex.: Wikipedia, Python REPLs) para execução de tarefas.
- **Prompts:** Utilitários para gestão e otimização de instruções para o modelo.
- **Vectorstores e Retrievers:** Suporte a bancos vetoriais e busca semântica para recuperação eficiente de dados.

Ou seja, O **LangChain** oferece uma estrutura modular flexível para o desenvolvimento e prototipagem de aplicativos com LLMs. Ele padroniza **Abstrações para LLMs** de diversos provedores e conecta processos via **Chains**. Para comportamentos autônomos, utiliza **Agents** integrados a **Tools** (APIs externas) e **Memory** (histórico de estado). O fluxo é otimizado por utilitários de **Prompts** e mecanismos de **Vectorstores e Retrievers** para busca semântica, permitindo iterar e substituir componentes

facilmente em sistemas como agentes de conversação e fluxos automatizados.

08:

-> Se eu colocar uma temperatura alta é pq quero uma resposta criativa, igual colocou abaixo, mas se quisesse uma resposta exata, uma precisão técnica, então = ex: algo jurídico, aí poderia diminuir a temperatura.



11: Simple sequential chain

-> O que o **Simple Sequential Chain** faz é transformar a **RESPOSTA** do Robô 1 na **ENTRADA** (o assunto) do Robô 2.

Porque, para um modelo de inteligência artificial (LLM), qualquer texto que ele recebe para processar é chamado de **Entrada** (ou *Prompt*).
Então:

- A **saída (resposta)** do Robô 1 ➔ vira o **assunto (entrada)** sobre o qual o Robô 2 vai trabalhar.

12 sequential-chain:

-> Nesse pode ter **múltiplas entradas** e **múltiplas saídas** ao longo do processo.

A diferença real **NÃO é a quantidade de etapas (Chain 1, Chain 2, Chain 3...)**, mas sim **quantas informações viajam ao mesmo tempo**:

***Simple** = Passa apenas **1 informação por vez** de robô para robô (estilo telefone sem fio).
***Sequential (Completo)** = Passa **múltiplas informações simultâneas** e guarda o histórico de todas as respostas que você quiser.

13 criando memória para o LLM:

-> O **Buffer** no LangChain é uma das formas mais comuns de dar **memória** para o LLM (como o ChatGPT), permitindo que ele lembre das mensagens anteriores durante uma conversa.

ConversationBufferMemory x ConversationBufferWindowMemory:

• ConversationBufferMemory: Armazena a íntegra de todo o histórico da sessão, garantindo retenção total de contexto ao custo de um aumento progressivo no consumo de tokens e na latência.

• ConversationBufferWindowMemory: Retém apenas as K interações mais recentes através de uma janela deslizante, otimizando custos e tempo de resposta em conversas extensas ao descartar contextos antigos.

- Ou seja, **ConversationBufferMemory** é como um caderno onde você escreve todas as páginas da conversa. A cada nova pergunta, o robô re-lê o caderno inteiro desde a página 1.
- **ConversationBufferWindowMemory** é como uma lousa pequena onde só cabem as últimas 3 frases. Para escrever uma nova frase, você precisa apagar a frase mais antiga da lousa. Então, podemos limitar o contexto de memória.

16 LangChain e VectorDB:

->ChromaDB é um **banco de dados vetorial (*vector database*) e infraestrutura de busca open-source projetada para Inteligência Artificial**.

17 - WebBaseLoard:

-> Acessa o site, extrai o conteúdo e converte em documento.

19 Vector-Store x Vector-DB:

-> *Vector Store: é um conceito mais genérico relacionado ao armazenamento de vetores*. *P*ara armazenar e consultar vetores de forma ágil.

Vector Database: é um sistema de banco de dados especializado projetado para lidar com vetores de alta dimensão e otimizado para

consultas de similaridade (significado, sentido) e outras operações relacionadas a vetores.
Mód 13 – LangChain usando a biblioteca Python para construir aplicações LLMs:

05: Instruction-Fine-Tuning LLms:

-> É uma técnica de ajuste fino (*fine-tuning*) aplicada a modelos pré-treinados (LLMs) com foco em capacitá-los a seguir comandos e instruções dos usuários com mais precisão, relevância e alinhamento humano.

->Por que usar IFT?

- Melhor compreensão de comandos:
- Especificidade e formatação:
- Mitigação de viés e redução de erros:

-> Como funciona o processo? (3 Etapas)

1. Seleção de Dados: Curadoria de um dataset com exemplos de instruções reais e suas respectivas respostas ideais.
2. Ajuste Fino (*Fine-Tuning*): O LLM base é treinado nesse novo dataset para aprender o mapeamento direto entre instrução e resposta.
3. Avaliação e Iteração: Análise do desempenho do modelo em comparação com o que os humanos esperam dele. Realizando rodadas adicionais de ajuste e validação se necessário.

4. Principais Aplicações

- Assistentes Virtuais
- Geração de Conteúdo
- Educação e Tutoria

08 LLm Llma 2:

Formato dos dados =

instruction	**A ordem ou tarefa:** O que você está mandando o modelo fazer.	

input	**O contexto / os dados de apoio:** O material sobre o qual ele vai trabalhar (pode ser uma pergunta, um trecho de texto, sintomas, etc.).	
output	**A resposta ideal / gabarito:** Exatamente como você gostaria que o modelo respondesse.	

10 - Processo de quantização:

->Reduzir a precisão dos parâmetros base.

Reduzir o tamanho do modelo na memória.

12: O Adaptador LoRA (PEFT)

Em vez de mexer nos bilhões de pesos originais do modelo =

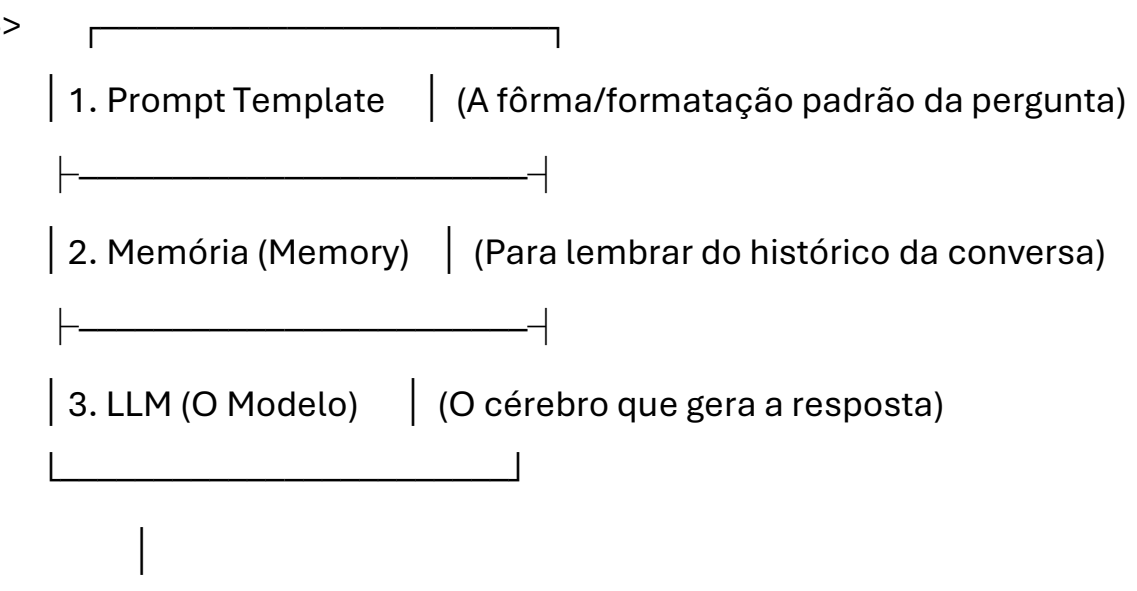
1. **Congelamos o modelo original:** Ele fica intacto, 100% congelado e levinho na memória em 4 bits. Isso evita perder o conhecimento que ele já tinha (evita a chamada *"amnésia catastrófica"*).

15: A função pipeline da Hugging Face:

-> Ela funciona exatamente como a esteira de suco: ela empacota o Tokenizador + Modelo + Tradução em uma linha só. Você só passa o texto de entrada, e ela devolve a resposta final pronta!

16 Construindo o Pipeline de geração de texto com LangChain:

->



▼

[LLM Chain]

-> Ele criou um template do prompt, montou a máquina de gerar resposta (esteira pipeline) para produir o texto da resposta, {coloca dentro modelo + tokenizador + regras de geração}, pega a esteira pronta Hugging Face e coloca um adaptador que é o HuggingFacePipeline, para que a bibliboteca LanChain consiga conversar e comandar essa esteira.

Mód 14 – 02 Busca semântica:

-> É uma técnica de pesquisa que entende a **intenção e o contexto** da consulta do usuário, em vez de procurar apenas palavras-chave iguais. Para isso, utiliza **Processamento de Linguagem Natural (PLN)** e **Inteligência Artificial (IA)** para compreender o significado e as relações entre as palavras, tornando os resultados mais precisos e relevantes.

Características =

- **Compreensão do Contexto:**
  - Analisa o cenário e as circunstâncias em que as palavras foram empregadas na consulta para identificar a real intenção do usuário.
- **Uso de Sinônimos e Variações:**
  - Expande os termos pesquisados incluindo termos equivalentes, variações linguísticas e gramaticais.
  - Garante que documentos abordando o mesmo assunto sejam recuperados, mesmo que usem termos diferentes.
- **Análise de Relacionamentos e Conceitos:**
  - Não trata de forma isolada, mas analisa como palavras e frases se conectam dentro de um campo semântico.
  - Identifica entidades, categorias temáticas e conceitos correlacionados.

Tecnologias Envolvidas =

- **Ontologias e Taxonomias:** Estruturas que organizam informações e vocabulário de um domínio específico, ajudando a relacionar termos de maneira significativa.

- **Análise de Sentimento:** Avaliação do tom emocional para compreender melhor o contexto e a intenção subjacente.
- **Reconhecimento de Entidades Nomeadas (NER):** Identificação e classificação de nomes próprios, locais, organizações e itens específicos no texto.

Aplicações e Benefícios =

- **Aplicações Práticas:**
    - **Motores de Busca Web:** Ex.: Google, para entregar resultados mais relevantes e diretos.
    - **Assistentes Virtuais:** Interpretação precisa de comandos de voz e respostas a perguntas conversacionais.
    - **Recuperação em Bases de Dados Corporativas:** Eficiência na localização de informações em grandes volumes de dados não estruturados.

Benefícios =

- **Maior Precisão nos Resultados:** Respostas mais alinhadas ao que o usuário realmente procura.
- **Experiência do Usuário Aprimorada:** Respostas mais rápidas e redução de retrabalho na formulação de buscas.

13:

-> StreamLit: montar a aplicação web de forma rápida e que não precise de tantos detalhes na interface, se o objetivo for funcionalidade. Ele facilita, pq chama funções para a interação.
-> LanChain: montar a conversação com llm.

Mód 16 Vector Database:

05 Como funcionam os Vector DataBase:
-HNSW: Organiza vetores em um grafo para encontrar rapidamente os mais semelhantes.
-LSH: Agrupa vetores parecidos em grupos chamados buckets.
-Projeção Aleatória: Reduz o tamanho dos vetores para facilitar e acelerar a busca.
-PQ: Divide os vetores em partes menores e compacta seus dados para economizar espaço.

**Todos ajudam a encontrar informações semelhantes mais rapidamente, mas usam técnicas diferentes.

06 Consulta e recuperação em Vector:

-Similaridade de Cosseno: Mede o quanto dois vetores são semelhantes pela direção. Quanto mais perto de 1, mais semelhantes.
-Distância Euclidiana: Mede a distância entre dois vetores. Quanto mais perto de 0, mais semelhantes.
-Produto Escalar: Considera a direção e o tamanho dos vetores. Quanto maior o resultado positivo, maior tende a ser a similaridade.
**São três formas diferentes de medir o quanto duas informações são parecidas.

08 A Long-Term Memory:

-> (memória de longo prazo) em LLMs é a capacidade de armazenar informações para serem utilizadas em conversas futuras, sem precisar começar do zero a cada interação.

09 Como escolher o banco vetorial?

->Os bancos de dados vetoriais são divididos em duas categorias:

- Dedicados: criados e otimizados para armazenar embeddings e buscar vetores semelhantes. Chroma/ Pinecone/ Weaviate/Milvus/Qdrant/Faiss

-> Com capacidade vetorial: bancos de dados tradicionais adaptados para armazenar e pesquisar vetores.

MongoDB/ Elasticsearch/ PostgreSQL (pgvector)/ Redis/ Neo4j

15 Divisão dos dados:

-> Muitos LLms tem um limite de tokens que podem processar em uma única interação. Dividir o texto em pedaços menores garante que o texto seja enviado dentro do limite permitido.

20 MMR:

-> MMR (Maximal Marginal Relevance): é uma técnica usada em sistemas de recuperação de informações, busca semântica e geração de resumos. Seu objetivo é selecionar documentos ou trechos de texto que sejam relevantes para a consulta e, ao mesmo tempo, diversos entre si, evitando repetições.

Seu funcionamento equilibra dois fatores:

1. Relevância: mede a similaridade entre o documento e a consulta, geralmente por meio de vetores (embeddings) e similaridade do cosseno.
2. Diversidade: evita selecionar documentos muito semelhantes aos que já foram escolhidos.

O MMR avalia e seleciona, de forma iterativa, os documentos que melhor equilibram esses dois fatores, tornando os resultados mais informativos e menos repetitivos.

- O RAG busca documentos relacionados à sua pergunta.
- Já o MMR pode selecionar, entre os trechos encontrados, os mais relevantes e menos repetitivos.

28: Arquitetura --> https://bbycroft.net/llm

RESUMO DA AULA:

Arquivo PDF ➔ Extração de Texto (Lê o arquivo inteiro via PyPDFLoader) ➔ **Divisão em Chunks** (Corta o texto em pedaços pequenos de 1000 caracteres para a IA conseguir ler via RecursiveCharacterTextSplitter) ➔ **Geração de Embeddings** (Transforma os pedaços de texto em representações matemáticas/vetores usando all-MiniLM-L6-v2) ➔ **Vector Database** (Armazena esses vetores no banco de dados ChromaDB para acesso rápido) ➔ **Pergunta do Usuário** (A pergunta também é transformada em vetor) ➔ **Busca Semântica / MMR** (O banco cruza o vetor da pergunta com os vetores do banco para encontrar os trechos mais relevantes e sem repetição) ➔ **Construção do Prompt** (Junta a Pergunta + Trechos Relevantes encontrados no banco) ➔ **LLM** (Envia o prompt para o

modelo Qwen2.5-1.5B-Instruct) ➔ **Resposta Final** (O LLM gera a resposta baseada exclusivamente no contexto do PDF).

--> **Modelo de Embedding (all-MiniLM-L6-v2):** Transforma textos em vetores numéricos, permitindo encontrar informações semelhantes no banco de dados. É pequeno e rápido, mas não gera textos nem responde perguntas.

--> Modelo Gerador / LLM (Qwen2.5-1.5B-Instruct): Lê, interpreta e gera respostas em linguagem natural. Utiliza os trechos encontrados pelo Embedding para elaborar respostas coerentes e contextualizadas.

Mód 17 – *NESSE MÓD O LMM É CAPAZ DE GERAÇÃO DE SEQUÊNCIA DE SAÍDA. *

07 Reasoning:

->Reasoning (Raciocínio) em LLMs:
É a capacidade dos (LLMs) de **fazer deduções lógicas e resolver problemas** usando as informações que aprenderam durante o treinamento.

-1-Como funciona o Raciocínio (Componentes-chave) =

- **Inferência Lógica:** O modelo reconhece padrões no texto para tirar conclusões óbvias (ex: se todo cão é mamífero e Rex é um cão, Rex é mamífero).
- **Resolução de Problemas:** Consegue seguir múltiplas etapas para resolver questões de matemática, gerar códigos ou responder perguntas difíceis.
- **Compreensão de Contexto:** Avalia a situação em que a pergunta foi feita para dar respostas que façam sentido.
- **Generalização:** Consegue aplicar o que aprendeu em uma área para outra (ex: usar regras gramaticais do inglês para ajudar a entender outro idioma).

2. Tipos de Raciocínio (Exemplos) =

- **Dedutivo:** Parte de uma regra geral para chegar a uma conclusão específica.
- **Indutivo:** Observa vários casos específicos para criar uma regra geral (ex: João e Ana estudaram e passaram, logo, estudar ajuda a passar).
- **Estratégico:** Alguns modelos conseguem resolver quebra-cabeças complexos ou jogar xadrez.

3. Limitações e Desafios =

- **Depende 100% dos Dados:** O raciocínio do LLM só será bom se os dados que ele usou para treinar forem bons. Dados incompletos ou preconceituosos geram respostas ruins.
- **Falta de Contexto:** Eles podem se perder em situações muito sutis ou específicas que não viram no treinamento.
- **"Alucinações" / Interpretação Errada:** Às vezes, eles simplesmente interpretam as coisas de forma errada e chegam a conclusões incorretas baseadas em premissas falsas.

--> Então, o modelo não "pensa" como um humano, ele calcula a probabilidade matemática de qual palavra (token) deve vir a seguir com base em padrões gigantescos, o que muitas vezes simula com perfeição um raciocínio lógico!

<u>10- Criando tokenizador:</u>

-> **LLMs não sabem fazer matemática nem raciocinar.** Os modelos de linguagem (como Transformers) apenas **prevêem o próximo token.**

Ex: 9 + 5 = 14 da erro / 9 + 5 = 1 4 agora da certo

- **Padding (Preenchimento):** Redes neurais exigem entradas de tamanho fixo. O *padding* (valor -1) serve para preencher os espaços vazios de sequências mais curtas, garantindo que todas tenham o mesmo tamanho.
- **Encoder e Decoder:** O modelo só entende números. O **Encoder** (codificador) converte o texto para números (IDs) para a IA

processar. O **Decoder** (decodificador) pega a resposta em números da IA e converte de volta para texto para nós lermos.

- **Métodos (__call__ e decode):** Na prática, o __call__ aplica o Encoder (codifica) e o método decode aplica o Decoder (decodifica).

20:

Encoder: converte a string de entrada para o token correspondente, enquanto o Decoder faz ao contrário.

-> O Modelo não trabalha com strings, mas sim com valores números que é o resultado do valor do token.

Mód 18: Construção do backend é API de um sistema de IA com RAG

02: LLMOps

-> LLMOps (Large Language Model Operations) refere-se ao conjunto de práticas e ferramentas utilizadas para gerenciar e operar modelos de linguagem de grande porte (LLMs).

->Estas operações incluem o desenvolvimento, treinamento, implantação e monitoramento contínuo desses modelos.

->O objetivo do LLMOps é garantir que os modelos de linguagem sejam eficientes, escaláveis, seguros e bem integrados em aplicações práticas

Diferença entre LLM x LLMOps =

-> **LLM** é a inteligência em si; **LLMOps** é a engenharia necessária para transformar essa inteligência em um produto real, estável e sustentável.

03 RAG:

----Componentes do RAG:

- Retrieval: busca informações relevantes em documentos, bancos de dados ou na web.
- Augmentation: adiciona essas informações ao contexto da pergunta.
- Generation: o LLM usa seu conhecimento + as informações recuperadas para gerar uma resposta mais precisa.

----Funcionamento do RAG:

1- Recepção da consulta: recebe a pergunta.

2. Busca informações relevantes.

3. Adiciona essas informações ao contexto.

4. O LLM gera uma resposta mais precisa usando seu conhecimento + os dados recuperados.

----Vantagens do RAG:

- Atualização continua.
- Melhoria na precisão.
- Flexibilidade: Pode ser aplicado em diversos domínios, desde atendimento ao cliente até pesquisa científica.

09:

- **Importar a base do projeto:** Prepara as ferramentas que vão fatiar o texto (LangChain), gerar os vetores (HuggingFace) e salvar no banco vetorial (Qdrant).
- **Varredura de pastas (dsa_lista_arquivos):** Cria uma função que entra em **todas as pastas e subpastas** para listar todos os arquivos existentes, sem deixar nenhum de fora.

10:

- **Extração de textos especiais:** Cria funções para conseguir ler formatos que não são texto puro:
  - **Word (.docx):** percorre o arquivo e extrai o texto parágrafo por parágrafo.
  - **PowerPoint (.pptx):** percorre slide por slide e puxa o texto de dentro de caixas e formas.
- **Execução via terminal:** Configura o script para rodar direto pelo prompt passando a pasta dos documentos (ex: python dsa_rag.py Documentos).

13:

-> BERT: Bidirectional Encoder Representations from Transformers) é um modelo de linguagem criado pelo Google para entender textos.

-> O diferencial é que ele lê uma palavra considerando o contexto antes e depois dela. Por isso é chamado de bidirecional.

MÓD 19 - continuação do projeto 9, que agora é o 10 – A construção do frontend de um sistema de IA Generativa com RAG:

05:

->API é um app web e a interface também é um app web = por isso usa o JSON para elas se comunicarem entre si.

06:

-> StreamLit = é um framework em Python criado especialmente para cientistas de dados, engenheiros de IA e desenvolvedores que querem **criar páginas e aplicativos web completos sem precisar programar em HTML, CSS ou JavaScript.**

É para criar aplicações simples e rápido.

CURSO 02 – CONSTRUÇÃO E DEPLOY DE AGENTES DE IA.

Mód 3 – 02:

LLMs (Large Language Models) x SLMs (Small Language Models):

-> São categorias de modelos de Inteligência Artificial voltados para o processamento de linguagem natural.

-> A diferença essencial entre os dois está no equilíbrio entre desempenho e eficiência. Enquanto os LLMs oferecem resultados mais sofisticados e versáteis, os SLMs são mais acessíveis, rápidos e adequados para cenários em que a economia de recursos é prioridade.

04: Agentes de IA

-> Agentes de Inteligência Artificial (IA) são programas de software baseados em LLMs projetados para interagir com seu ambiente, coletar dados e utilizá-los para realizar tarefas de forma autônoma, visando alcançar objetivos predeterminados.

05 Agentes de IA são RPA 2.0?

-> Não exatamente, Agentes de IA não são apenas uma atualização do RPA, mas uma mudança de paradigma: substituem **regras fixas** por **raciocínio e adaptação autônoma**.

->RPA Tradicional: Focado na automação de tarefas repetitivas e baseadas em regras fixas. Ele replica cliques e digitações humanas no sistema, funcionando de maneira determinística (se a interface mudar ou houver exceções não mapeadas, ele falha).

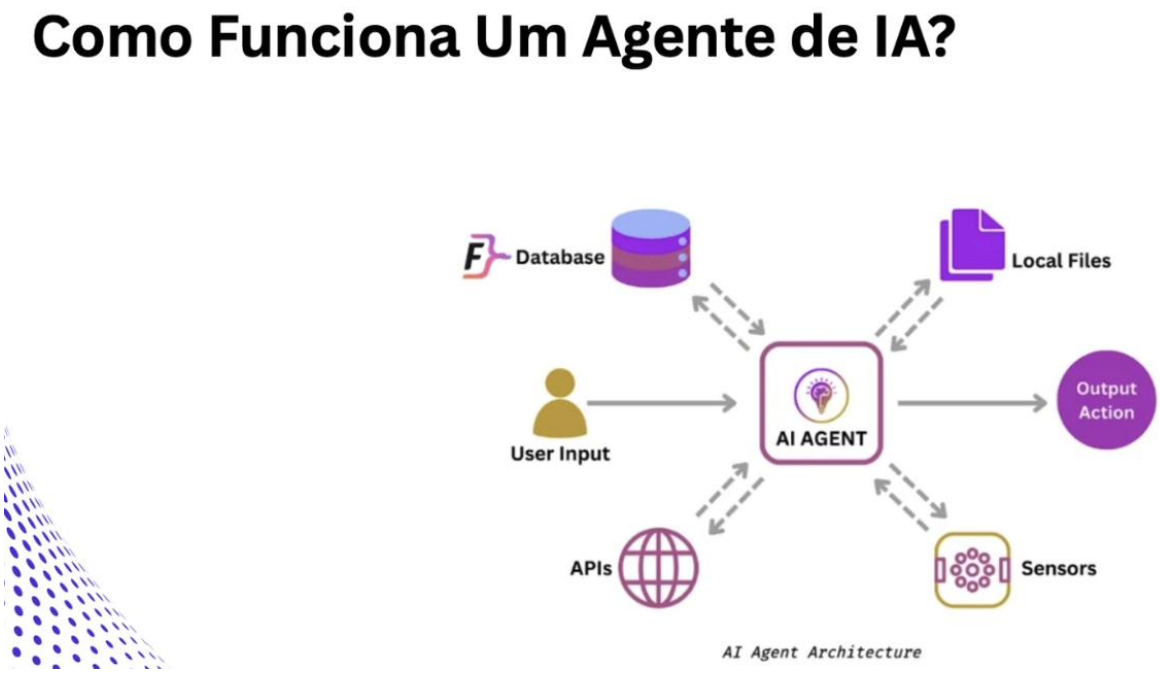
-> Agentes de IA: Sistemas dotados de autonomia, capazes de raciocinar, tomar decisões, planejar e se adaptar a contextos dinâmicos e dados não estruturados (textos, imagens, linguagem natural).

10 Componentes principais de um agente de IA:

- **Os sensores (entrada e percepção) permitem que o agente colete dados de seu ambiente.**
- **Um processador (cérebro e tomada de decisão) analisa esses dados e toma decisões com base neles.**
- **Os atuadores (ação e saída) são os meios pelos quais o agente afeta o ambiente, executando as ações decididas pelo processador.**

LLMs são os processadores mais comuns para agentes de IA atualmente.

12:



13: Large Action Models = LAMs:

-> Os LAMs são orientados para a execução de tarefas, permitindo que interajam diretamente com sistemas digitais ou físicos para realizar ações específicas.

Pode ser utilizado para automatizar processos complexos abre o navegador, busca as opções, preenche os formulários da página e avança até a etapa de confirmação/pagamento.

14 Orquestração de modelos e uso de SMLs:

-> O RAG permite que **SLMs menores tenham desempenho próximo ao de LLMs maiores**, complementando sua falta de conhecimento e tornando-os mais eficientes e especializados.

15 Chamada de função e saída estruturada:

-> A "chamada de função" (Function Calling) é um recurso em agentes de IA que permite a invocação de funções específicas para executar tarefas ou obter informações adicionais. Isso é especialmente útil quando o agente precisa interagir com APIs externas ou realizar operações que vão além do processamento de linguagem natural.

-> A combinação de chamadas de função e saídas estruturadas aprimora a capacidade dos Agentes de IA de interagir com sistemas externos e fornecer respostas mais precisas e contextualmente relevantes aos usuários.

16 O papel das ferramentas – Pipelines e Human in the loop:

-> Ferramentas são recursos usados pelos Agentes de IA para interagir com o mundo. Elas podem ser digitais (APIs, bancos de dados, códigos) ou humanas (especialistas validando decisões). Elas funcionam como uma ponte entre o raciocínio da IA e a execução de ações, tornando os agentes mais capazes, seguros e adaptáveis.

-> **Por que as ferramentas são importantes?**

Elas fazem os Agentes de IA serem:

- **Mais poderosos** → conseguem executar ações, não apenas conversar.
- **Modulares** → podem adicionar ou trocar ferramentas conforme a necessidade.
- **Conscientes do contexto** → conseguem buscar informações relevantes antes de agir.

--> Uma **pipeline** é uma sequência de etapas onde uma informação passa por vários processos até chegar ao resultado.

--> Human in the loop = É quando uma pessoa participa do processo da IA em algum momento para revisar, corrigir ou aprovar algo. A IA não trabalha 100% sozinha.

Conceito	O que é?	Papel no sistema
Function Calling	Recurso do LLM	O modelo diz *o que* quer chamar em formato estruturado (JSON).
Tool (Ferramenta)	Habilidade / Integração	O código real que conecta o agente ao mundo exterior (APIs, BDs, etc.).

Pipeline	Arquitetura / Fluxo	O caminho passo a passo que os dados percorrem no sistema.
Human-in-the-Loop	Ponto de Controle	Uma pausa no pipeline para um humano revisar/aprovar o uso de uma ferramenta.

19 Frameworks para construção de agentes:

-> 1. LangChain

- **Foco:** A fundação e integração universal.
- **Como funciona:** É a biblioteca pioneira do ecossistema. Funciona como uma grande caixa de ferramentas que conecta LLMs a fontes de dados externas, conectores de APIs e rotinas pré-configuradas.

-> 2. LangGraph

- **Foco:** Orquestração avançada com fluxos cíclicos (grafos de estado).
- **Como funciona:** Foi criado pela mesma equipe do LangChain para resolver cenários complexos onde fluxos lineares não bastam. Ele modela o comportamento do agente como nós e arestas de um grafo, permitindo *loops* de autoavaliação (ex.: *"o código gerado tem erro? Se sim, volte ao nó de correção"*), controle de estado persistente e intervenção humana (*Human-in-the-Loop*).

-> 3. CrewAI

- **Foco:** Orquestração de múltiplos agentes em equipe (*Multi-Agent Role-Playing*).
- **Como funciona:** É pensado na metáfora de uma empresa ou equipe humana. Você define agentes com **papéis (*roles*)**, **objetivos (*goals*)** e **histórico (*backstory*)**, atribuindo tarefas específicas a cada um para colaborarem de forma sequencial ou hierárquica.

-> 4. LlamaIndex

- **Foco:** Conexão profunda com dados e recuperação inteligente (*Data Agents* / RAG).

- **Como funciona:** Se o seu agente depende intensamente de consultar grandes bases de documentos, bancos vetoriais, planilhas ou dados não estruturados, o LlamaIndex é o especialista em indexar, consultar e injetar esses dados no contexto do agente.

-> 5. Phidata

- **Foco:** Desempenho, agentes multimodais e facilidade de deploy.
- **Como funciona:** Um framework projetado para ser leve, rápido e pronto para produção, com suporte nativo a memória de sessão em bancos SQL, ferramentas integradas e capacidade multimodal (áudio, imagem e texto).

20 Time Travel:

-> Refere-se à capacidade desses agentes de acessar, relembrar ou prever estados passados e futuros dentro de um ambiente simulado ou operacional.

-> Não significa uma "viagem no tempo" literal, mas sim a capacidade de navegar entre diferentes momentos de um processo para melhorar aprendizado, planejamento e tomada de decisão.

21 Prompt caching e Reasoning em agentes:

-> Prompt Caching é a prática de armazenar respostas geradas por modelos de IA para consultas recorrentes, evitando a necessidade de recomputação sempre que a mesma solicitação for feita.

-> Reasoning (raciocínio) em Agentes de IA é a capacidade do agente de analisar informações de forma lógica para **tomar decisões e resolver problemas**.

Principais tipos:

- **Dedutivo:** usa regras gerais para chegar a uma conclusão específica.

- *Ex.: Todo humano é mortal → João é humano → João é mortal.*

- **Indutivo:** observa exemplos e cria padrões gerais, mas pode ter incertezas.

Ex.: Vi vários cisnes brancos → concluo que todos podem ser brancos.

- **Abdutivo:** cria hipóteses para explicar algo observado.

Ex.: Vejo fumaça → pode haver fogo.

- **Chain-of-Thought (CoT):** técnica em que a IA organiza etapas de raciocínio para gerar respostas mais coerentes. (cadeia de pensamentos)
- **Multi-Agente:** vários agentes de IA trabalham juntos, combinando habilidades para resolver problemas mais complexos.

***Prompt Caching melhora a eficiência e a velocidade dos agentes, enquanto Reasoning aprimora sua inteligência e capacidade de tomada de decisão.

Mód 4 – 04: Re Ranking:

1. O mecanismo de busca (**retriever**) consulta a base de dados usando:
    a. **Busca Semântica (Vetorial/Embeddings)**: como FAISS ou Annoy;
    b. **Busca Lexical (Palavras-chave)**: como BM25.
2. **O ponto fraco**: Esses métodos priorizam velocidade e fazem comparações rápidas/aproximadas. Por isso, a busca inicial costuma trazer **ruído**: trechos que parecem relevantes superficialmente, mas que não respondem de fato à pergunta ou estão fora de contexto.
3.

    --> O que é o Re-Ranking e Como Ele Funciona?

    - O **Re-Ranking** funciona como uma **segunda etapa de filtragem** que reavalia e reorganiza a lista de documentos antes de enviá-los ao modelo gerador (LLM):

- **Na busca inicial (Bi-Encoder):** A pergunta e os documentos são codificados separadamente em vetores e comparados por distância (rápido, porém menos detalhado).
- **No Re-Ranking (Cross-Encoder):** Modelos como **BERT, RoBERTa ou T5** analisam a **pergunta e o documento juntos** como um par (pergunta, documento) usando mecanismos de atenção completa. Isso gera uma **nota de relevância (score)** muito mais precisa e fiel ao sentido real da consulta.

Benefício	Impacto no Sistema
Melhora a Precisão	Evita que textos irrelevantes entrem no prompt e induzam o LLM a errar ou alucinar.
Aprimora a Coerência	Reduz o risco de utilizar dados conflitantes, ambíguos ou desatualizados.
Aumenta a Eficiência	Injeta apenas o conteúdo estritamente necessário no contexto do LLM, economizando tokens e tempo de processamento.
Adaptação a Contextos	Permite calibrar o sistema para priorizar certas fontes (ex.: dar mais peso a normas regulatórias, artigos médicos ou fontes oficiais).

12 primeiro agente criado, agente de pesquisa usando a função: DuckDuckgo =

01_dsa_agente_pesquisa.py

```python
from phi.agent import Agent
from phi.model.openai import OpenAIChat
from phi.tools.duckduckgo import DuckDuckGo
from dotenv import load_dotenv

# Bloco de execução
try:

    # Carrega variáveis de ambiente do arquivo .env
    load_dotenv()

    # Criação do Agente com tratamento de erro
    try:
        agente_pesquisa_dsa = Agent(name = "Agente de Pesquisa",
                                    role = "Pesquisar na Web",
                                    model = OpenAIChat(id = "gpt-4o"),
                                    tools = [DuckDuckGo()],
                                    instructions = ["Gere a resposta no idioma Português do Brasil"],
                                    markdown = True,
                                    show_tool_calls = True)
    except Exception as e:
        print(f"Erro ao criar o agente: {e}")
        agente_pesquisa_dsa = None

    # Verifica se o agente foi criado com sucesso antes de executar a consulta
    if agente_pesquisa_dsa:
        try:
            agente_pesquisa_dsa.print_response("Quais os fatos jurídicos recentes ligados à Nvidia?", stream = True)
        except Exception as e:
            print(f"Erro ao obter resposta do agente: {e}")
    else:
        print("O agente não foi criado corretamente. Verifique os erros anteriores.")
```

14 segundo agente criado, consulta de dados financeiros, usando: YFinanceTools =

02_dsa_agente_financeiro.py

```python
import os
from phi.agent import Agent
from phi.model.openai import OpenAIChat
from phi.tools.yfinance import YFinanceTools
from dotenv import load_dotenv

# Carrega variáveis de ambiente do arquivo .env
load_dotenv()

try:

    # Criação do Agente Financeiro com ferramentas para consulta de dados financeiros
    agente_financeiro_dsa = Agent(
        name = "Agente Financeiro",
        role = "Coletar Dados de Ativos Financeiros",
        model = OpenAIChat(id = "gpt-4o"),
        tools = [
            YFinanceTools(
                stock_price = True,
                analyst_recommendations = True,
                company_info = True,
                company_news = True
            )
        ],
        instructions = ["Sempre use tabelas para exibir dados", "Gere a resposta no idioma Português do Brasil"],
        markdown = True,
        show_tool_calls = True
    )

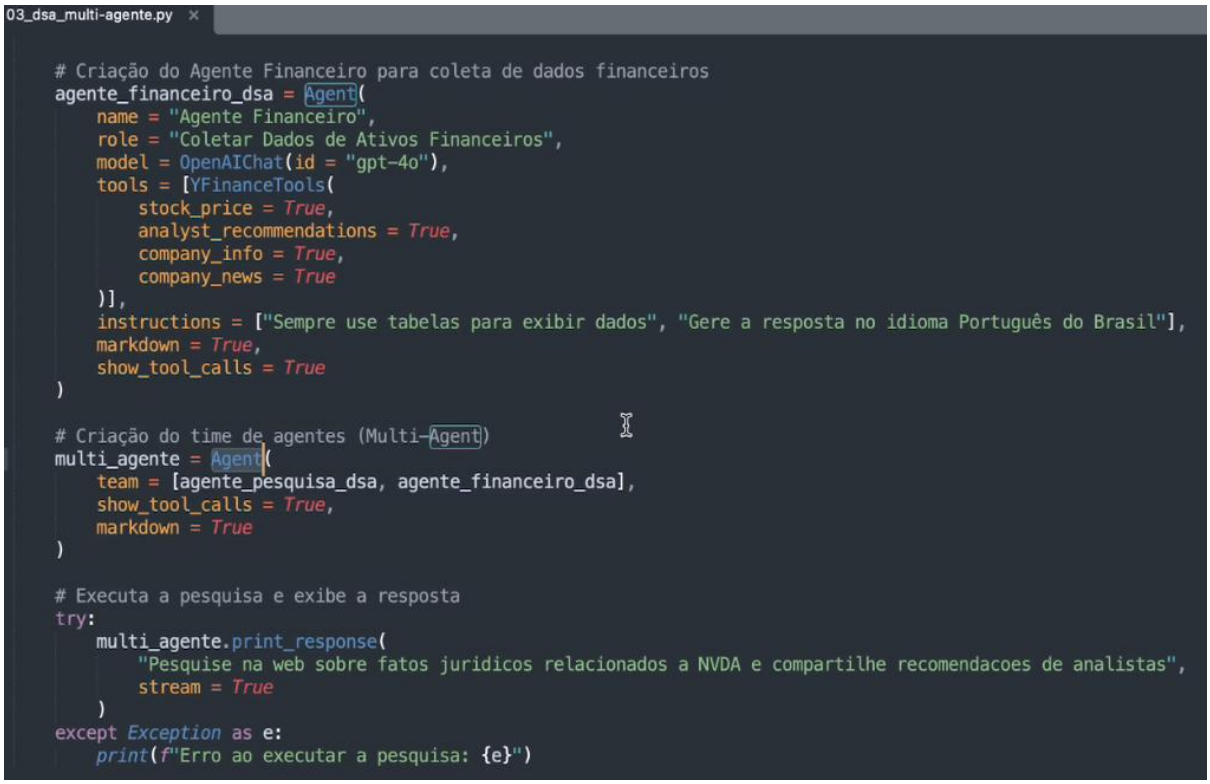
    # Solicita ao agente recomendações de analistas para a ação NVDA (Nvidia)
    agente_financeiro_dsa.print_response("Compartilhe recomendações de analistas para NVDA.", stream = True)

except Exception as e:
```

15:

-> Terceiro agente é multi-agente, vai copiar o primeiro e segundo agente, pq é como se ele fosse o “chefe” dos outros 2 que criei acima:

-> em cima está o primeiro agente que n apareceu no print: E na instrução do multi-agente ele copiou dos 2.

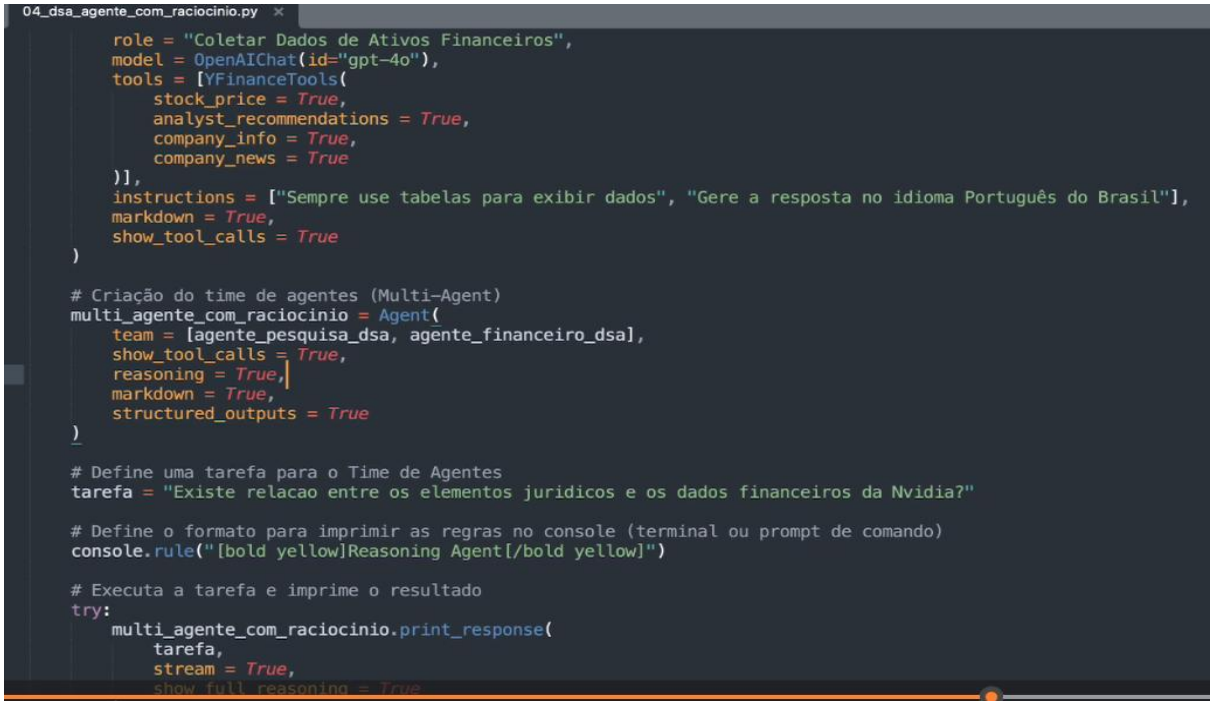


17 Construindo o sistema Multi-agente com raciocínio:

-> Pq ele até agora estava só prevendo, agora ele vai pensar, com o {reasoning = true}

->No caso damos um contexto melhor, porque ele continua prevendo.

- **Sem raciocínio guiado:** Se você fizer uma pergunta difícil de matemática ou lógica e pedir a resposta direto, o modelo tenta "chutar" o próximo token final baseado em correlação estatística direta, errando com frequência.
- **Com raciocínio passo a passo (Chain-of-Thought):** Ao forçar o modelo a decompor a resposta em etapas (Pensamento -> Ação -> Observação), cada palavra gerada vira contexto para a próxima.
- Dessa forma, ele **simula** o processo de raciocínio humano. Estatisticamente, prever o próximo passo intermediário é muito mais fácil e preciso do que pular direto para a conclusão.



19 banco vetorial:

->O professor escolheu o LanceDB , pois é uma opção poderosa para quem precisa armazenar e recuperar embeddings vetoriais de forma rápida e eficiente, sem a complexidade de bancos de dados distribuídos tradicionais.

20:

1. **O Embedder é o "Bibliotecário / Arquivista":** Ele não conversa e nem gera respostas, apenas lê o documento, etiqueta cada trecho com códigos numéricos (vetores) e organiza nas estantes do banco de dados vetorial. Quando você faz uma pergunta, ele localiza rapidamente a página certa.
2. **O GPT-4o é o "Consultor / Especialista":** Ele pega as páginas que o bibliotecário encontrou, lê, entende a sua pergunta e redige a explicação final para você. É o cérebro que **conversa, raciocina, resume e redige texto**.

30: O que é Phidata?

Phidata é uma estrutura para a construção de agentes e fluxos de trabalho multimodais.

- **Construa agentes com memória, conhecimento, ferramentas e raciocínio.**
- **Forme equipes de agentes que possam trabalhar juntos para resolver problemas.**
- **Interaja com seus agentes e fluxos de trabalho usando uma interface de usuário de agente elegante e intuitiva.**

Mód 5: Construção de um sistema de Multi-Agentes de IA com uma das principais ferramentas do mercado, o CrewAI.

02:

-> O CrewAI é uma plataforma projetada para criar e gerenciar equipes de Agentes de Inteligência Artificial (IA) que colaboram para automatizar fluxos de trabalho complexos.

-> Com o CrewAI, é possível construir e implantar automações utilizando qualquer Modelo de Linguagem de Grande Escala (LLM) e plataforma em nuvem.

-> Cada agente é uma unidade autônoma com atributos específicos, como: -

- Papel (Role): Define a função e a especialização do agente dentro da equipe.
- Meta (Goal): Objetivo individual que orienta a tomada de decisões do agente.
- Histórico (Backstory): Fornece contexto e personalidade ao agente, enriquecendo as interações.

04 - Comunicação e Sincronização entre Agentes de IA:

- **Comunicação:** é a troca de informações entre agentes para compartilhar dados, estados e intenções, permitindo que trabalhem juntos e tomem decisões em conjunto. Exemplo: um agente encaminha uma solicitação para outro mais especializado.
- **Sincronização:** é a coordenação das ações dos agentes no tempo, garantindo que executem tarefas na ordem correta, evitando

conflitos, erros e trabalho duplicado. Exemplo: no controle de tráfego aéreo, os agentes sincronizam suas ações para evitar colisões.

- **Desafios:** garantir consistência dos dados, evitar conflitos, manter a eficiência e permitir que o sistema continue funcionando bem mesmo com a adição de novos agentes.
- **Solução:** utilizar protocolos de comunicação e mecanismos de sincronização adaptativos, que permitem aos agentes se ajustarem às mudanças do ambiente e colaborarem de forma eficiente.

16:

-> O allow_delegation (ou Allow Delegation) é um parâmetro usado em alguns frameworks de agentes de IA (como o CrewAI) para definir se um agente pode delegar tarefas para outros agentes.

23 Deploy do Time de Agentes de IA:

-> O deploy de Agentes de IA consiste em disponibilizá-los por meio de uma API. O agente é otimizado para reduzir latência e consumo de recursos, empacotado em um container Docker e implantado localmente ou na nuvem (AWS, GCP ou Azure), podendo usar Kubernetes, AWS Lambda ou servidores Flask/FastAPI.

A API recebe as requisições, processa os dados, executa o agente e retorna as respostas. Para garantir escalabilidade, confiabilidade e segurança, utilizam-se balanceamento de carga, cache, monitoramento (Prometheus e Grafana) e autenticação (OAuth2, JWT ou API Keys).

O processo também pode ser automatizado com CI/CD para facilitar atualizações e manutenção contínua.

CI (Continuous Integration:

- **Integração Contínua):** sempre que um desenvolvedor faz uma alteração no código, ela é integrada ao projeto automaticamente. O sistema executa testes para verificar se nada foi quebrado.

CD (Continuous Delivery/Deployment):

- **Continuous Delivery (Entrega Contínua):** o sistema prepara automaticamente uma nova versão do software, deixando-a pronta para ser publicada, mas a publicação depende de uma aprovação humana.
- **Continuous Deployment (Implantação Contínua):** além de preparar a nova versão, o sistema também a publica automaticamente, sem necessidade de aprovação manual.

25:

-> O Uvicorn é um servidor web ASGI usado para executar aplicações Python, principalmente aquelas feitas com frameworks modernos como FastAPI e Starlette.

- **Uvicorn** → é o servidor que faz a API funcionar.
- **Docker** → é a "caixa" (container) que transporta e executa a API com todas as dependências.

29: Executando o deploy e comsumindo API

Por que não criou um ambiente virtual e quando executei o agente criou um ambiente virtual?

->Quando executa de maneira manual, está sendo executado na máquina local, no deploy não é na máquina virtual e sim no container, então não há necessidade de criar um ambiente virtual.

Mód 06 – LangGraph:

- **LangChain** é um framework projetado para facilitar o desenvolvimento de aplicações que utilizam modelos de linguagem (LLMs).

--> Ele fornece ferramentas para a criação de pipelines de processamento de linguagem natural, integração com diferentes fontes de dados e memória, além de permitir a construção de agentes interativos e cadeias complexas de prompts.

--> Seu objetivo é tornar a engenharia de IA mais modular e reutilizável, acelerando o desenvolvimento de aplicações baseadas em IA.

- **LangSmith** é uma plataforma para depuração, monitoramento e avaliação de pipelines de IA desenvolvidos com LangChain.

--> Ele permite rastrear execuções, visualizar fluxos de dados, testar diferentes configurações de prompts e otimizar a performance de aplicações que utilizam modelos de linguagem.

- **LangServe** é uma ferramenta que **transforma aplicações feitas com LangChain em APIs**. Assim, elas podem ser acessadas pela internet (via HTTP), integradas a outros sistemas e colocadas em produção de forma rápida e escalável.
- **LangGraph** é uma biblioteca que amplia o **LangChain**, permitindo criar **fluxos de trabalho mais complexos** usando **grafos**. Nele, os **nós** representam etapas do processo e as **arestas** definem o caminho entre elas, possibilitando criar **agentes de IA mais flexíveis e avançados**.

LangChain	**Constrói** a aplicação.
LangSmith	**Testa e monitora** a aplicação.
LangServe	**Publica** a aplicação como API.
LangGraph	**Organiza fluxos complexos** e coordena agentes.

04 Fundamentos da teoria dos Grafos:

- Um **grafo** é uma forma de representar um caminho entre etapas. Ou seja, todas as etapas conectadas pelos caminhos.
- **Nó** = uma etapa (ex.: pesquisar, responder, resumir).
- **Aresta** = o caminho entre uma etapa e outra.

--> A Teoria dos Grafos é um ramo da matemática que estuda relações entre objetos, representadas por estruturas chamadas grafos. Um grafo é composto por vértices (ou nós) e arestas (ou ligações) que conectam esses vértices.

05: LangGraph e construção de agentes:

-> É uma biblioteca construída sobre o LangChain, ou seja, aproveita as ferramentas e adiciona a capacidade de orquestrar fluxos com decisões e loops.

--> A biblioteca introduz a ideia de modelar fluxos de trabalho como grafos cíclicos, adicionando memória embutida – recursos fundamentais para agentes autônomos.

Ou seja, grafo é só um jeito de desenhar o fluxo: caixinhas conectadas por setas. Cíclico é porque, se precisar, o fluxo pode voltar para um passo anterior. Já a memória embutida significa que o LangGraph lembra o que já aconteceu durante a execução. Tipo: já pesquisei antes, então posso mudar de estratégia sem perguntar tudo de novo.

Juntando isso, você cria agentes que conversam, repetem etapas quando precisa e mantêm contexto, que é justamente o que dá inteligência e flexibilidade para o sistema. É como se o workflow não fosse só uma linha reta, mas um caminho que aprende enquanto anda.

-> Um agente com LangGraph pode planejar, decidir, pesquisar, consultar, talvez chamar outro modelo, checar se deu certo e, se não deu, repetir. Ele não fica só conversando, ele executa um processo para chegar num objetivo. Essa é a grande virada de chave entre chatbot e agente de IA.

Então, o LangGraph ele gerencia, ele apenas coordena.

06 Arquitetura e conceitos:

-> o LangGraph permite criar **sistemas multiagentes**, onde vários agentes especializados trabalham juntos. Um agente principal pode coordenar outros agentes menores, dividindo problemas complexos em tarefas específicas.

Essa ideia de agentes colaborativos usando grafos foi inspirada em modelos de processamento distribuído, como **Pregel (Google) e Apache Beam**, mas adaptada para trabalhar com **modelos de linguagem (LLMs)**.

- Nós (Nodes) = etapas ou componentes do fluxo. Cada nó pode encapsular uma operação atômica, como responder a uma pergunta, buscar dados externos ou calcular um valor.
- Arestas (Edges) = Conectam os nós, definindo a direção do fluxo de controle e de dados entre etapas.
- Estado (State) = Representa a memória e contexto mantidos durante o fluxo.

-> Ou seja, as informações fornecidas pelo usuário ou resultados de etapas anteriores podem ser armazenadas e reutilizadas em etapas posteriores.

09 Vantagens e desvantagens ao usar o LangGraph:

->Workflow é uma sequência organizada de etapas que precisam acontecer para realizar uma tarefa.

É o caminho que o sistema segue.

- ★ Vantagens: orquestração de fluxos complexos, coordenação multiagentes, memória persistente e estado global, customização e controlabilidade, colaboração Humano IA, ecossistema e ferramentas integradas.
- ★ Desvantagens: curva de aprendizado e complexidade, configuração elaborada, risco de loops e erros lógicos, uso intensivo de recursos, ecossistemas em evolução.

10 Comparação com outras soluções:

Solução	Melhor Caso de Uso	Nível de Controle / Esforço
LangGraph	Workflows complexos, controle minucioso via grafos, loops e memória persistente robusta	**Controle total** (maior esforço de implementação)
CrewAI	Times de agentes com papéis/especialidades bem definidos	**Rápido de estruturar**, baseado em convenções

AutoGen	Simulação de conversas e interações multiagente estilo diálogo	**Orientado a chat**, menos determinístico
LangChain puro	Pipelines diretos, lineares ou chatbots simples	**Simples e sequencial**

14 O que é um Workflow no LangGraph:

-> No **LangGraph**, um **workflow** é a estrutura que organiza como **LLMs (modelos de linguagem)** e **ferramentas** trabalham juntos para executar uma tarefa. Existem dois tipos principais:

- **Workflows com caminhos predefinidos:** seguem uma sequência de etapas já definida pelo desenvolvedor.
- **Agentes com controle dinâmico:** o próprio LLM decide, durante a execução, quais ações e ferramentas utilizar conforme a necessidade.

-> O LangGraph suporta ambos os modelos, permitindo criar desde **fluxos lineares simples** até **arquiteturas complexas com múltiplos agentes** colaborando de forma dinâmica.

Além disso, oferece **persistência de contexto** (para manter informações em interações longas) e **suporte nativo a streaming**, exibindo em tempo real o raciocínio e as ações dos agentes.

--> Em resumo, um **workflow no LangGraph** é a estrutura que define e organiza a interação entre os componentes de IA para executar tarefas com flexibilidade e controle.

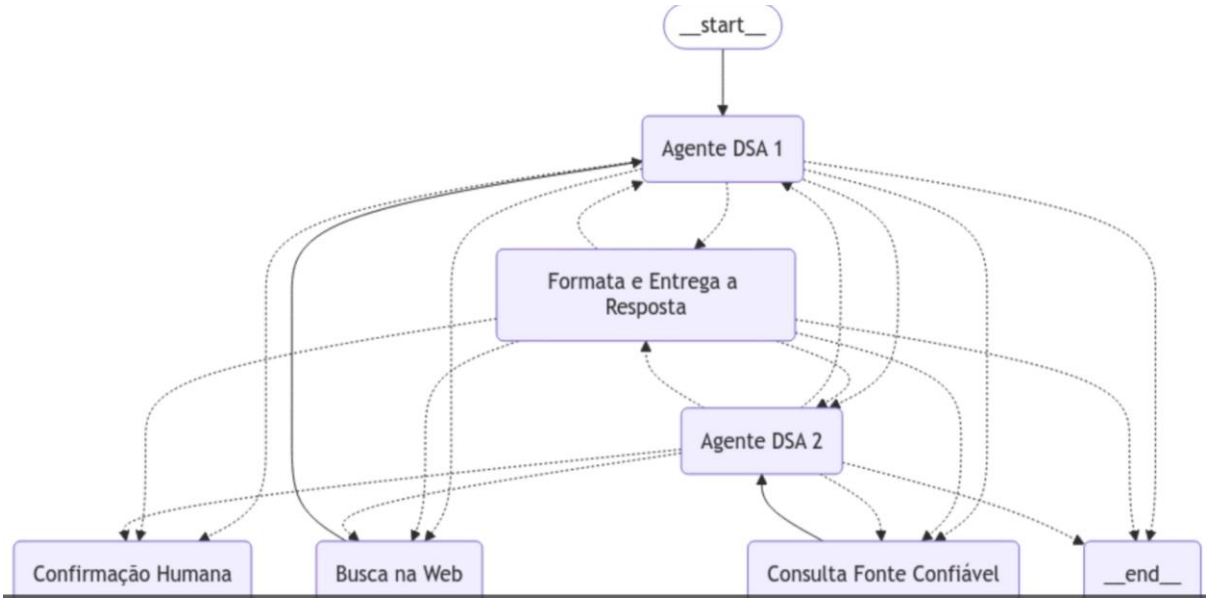
Mód 07: LangGraph Para Construção de Agentes de IA - Ferramentas, Memória, Estado, Workflow e Human-in-the-Loop

08 adicionando as ferramentas:

-> Ele definiu o LLM open-source com ollama, definiu as ferramentas (DuckDuckGo), adicionou as ferramentas para o processo de tomada de decisão do Agente.

Se ele parasse ali, teria feito uma aplicação estática (aplicação IA generativa) que resolveria alguns problemas, mas vamos construir algo mais dinâmico, que o próprio agente tome decisão (autônomo, sem precisar da ação humana).

09: Compreendendo o Workflow:



-> O agente DSA 1 ele pode já formatar e entrega a resposta se ele souber, mas se for uma pergunta que ele não saiba, então fará uma busca na web, se a resposta for suficiente, então finalizou, se não, chamará o agente DSA 2 (especialista em fontes confiável) que irá consultar a fonte confiável, se teve a resposta necessária formata e entrega a resposta e o humano irá confirmar se foi a resposta satisfeita ou não.

10 Criando a estrutura do agente:

->O sistema de agente de IA é um workflow, um processo, um fluxo de trabalho.

11 Configurando a Memória do Sistema:

-> No LangGraph, a memória refere-se à capacidade de armazenar e recuperar informações ao longo da execução do fluxo de conversação. Isso permite que o agente mantenha contexto entre interações, lembre-se de dados do usuário e ajuste suas respostas com base em interações anteriores. A memória pode ser implementada de diferentes formas, como

armazenando variáveis no estado do grafo ou integrando soluções externas de armazenamento.

14 Criando os componentes do Graph – workflow – ESTADO:

-> O **Graph** é a **ideia do fluxo** (os passos e conexões que você desenha).

-> **Todo grafo no LangGraph é um workflow**, mas nem todo workflow tradicional tem a flexibilidade de um grafo (com loops, idas e voltas e decisões inteligentes que os agentes precisam).

- ❖ Estado

No LangGraph, o estado é a estrutura de dados que mantém o contexto ao longo da execução do fluxo. Ele armazena informações relevantes, como entradas do usuário, saídas de funções e variáveis persistentes que influenciam o comportamento do agente.

15 – NODE:

- ❖ Node (Nó)

No LangGraph, um node (nó) é uma unidade de processamento dentro do Graph Workflow. Cada nó executa uma função específica e pode modificar o estado antes de passar a execução para outro nó.

16 – NODE ESPECIALIZADO:

- ❖ Node Especializado

O node tool como dsa_agente_node_tool é um nó especializado no LangGraph responsável por processar chamadas de ferramentas (tools) dentro do fluxo do agente. Ele atua como um intermediário que:

- Identifica qual ferramenta foi chamada com base no estado (state["lst_res"][-1]).
- Extrai e valida os parâmetros necessários para a ferramenta funcionar corretamente.
- Invoca a ferramenta correspondente (dic_tools[res.tool_name].invoke(tool_input)).
- Retorna a saída de acordo com a ferramenta utilizada.

➢ Diferença entre node e node especializado:

Característica	Node do Agente (dsa_agente_node)	Node Especializado (dsa_agente_node_tool)
Responsabilidade	Raciocínio, tomada de decisão e geração de texto via LLM.	Execução de ferramentas e chamada de APIs/funções externas.
Executa código externo?	Não (só pergunta ao LLM o que fazer).	Sim (executa DuckDuckGo, Wikipedia, etc.).
Analogia	O **arquiteto/planejador** que diz qual material buscar.	O **operário/executor** que vai até o depósito buscar e entregar o material.

17 Criando os componentes do Graph workflow – Edges (arestas) condicionais:

-> As edges condicionais determinam dinamicamente qual será o próximo nó (node) a ser executado no fluxo, com base no estado atual. Elas permitem que o grafo siga caminhos diferentes dependendo da lógica definida.

->Obtém a última resposta do agente armazenada no estado, decide qual é o próximo node com base na ferramenta usada, exibe qual será o próximo node e retorna o próximo node que deve ser executado.

```python
# Função para determinar os próximos nodes com base nas condições atuais do estado
def dsa_edges_condicionais(state):

    # Indica início da execução das condições das edges (transições)
    print("--- Edges Condicionais ---")

    # Obtém a última resposta do agente armazenada no estado
    last_res = state["lst_res"][-1]

    # Decide qual é o próximo node com base na ferramenta usada
    next_node = last_res.tool_name if isinstance(state["lst_res"], list) else "dsa_tool_resposta_final"

    # Exibe qual será o próximo node
    print("Próximo Node:", next_node)

    # Retorna o próximo node que deve ser executado
    return next_node
```

21 Configuração Multi-Agente e Humain in the loop:

-> A Configuração Multi-Agente no LangGraph refere-se ao uso de vários agentes dentro de um único grafo para colaborar e resolver tarefas complexas. Cada agente pode ter um papel específico e interagir dinamicamente no fluxo do grafo.

-> O Human-in-the-Loop (HITL) é uma abordagem onde humanos intervêm no fluxo do agente quando necessário. Isso é útil em casos que exigem supervisão, validação ou decisões que um modelo sozinho não pode tomar.

-> Para facilitar a implementação de interações humanas, o LangGraph introduziu a função *interrupt*. Essa função permite pausar a execução do fluxo de trabalho em um ponto específico, apresentar informações ao usuário e, posteriormente, retomar o processo com base na entrada fornecida pelo humano.

Os principais padrões são:

- ★ **Aprovar ou rejeitar:** antes de ações críticas (como uma chamada de API), o fluxo pode ser pausado para revisão humana. Se a ação for rejeitada, o workflow pode seguir um caminho alternativo.
- ★ **Revisar e editar o estado:** o humano pode revisar e modificar o estado atual do workflow para corrigir erros ou adicionar informações.
- ★ **Solicitar entrada:** o sistema pode pedir informações adicionais ao usuário em pontos específicos do fluxo para esclarecer dúvidas ou obter mais contexto.

-> Para suportar essas intervenções, o LangGraph utiliza **checkpoints**, que salvam o estado do workflow em momentos específicos. Assim, o processo pode ser pausado e retomado sem perder o contexto nem comprometer a integridade do fluxo.

22:

Vamos recriar o grafo.

```python
# Inicializando o grafo de estados do workflow
workflow = StateGraph(State)

######################### Agente 1 #########################

# Adiciona o primeiro node do agente ao workflow
workflow.add_node(node = "AgenteDSA1", action = dsa_agente_node)

# Define o ponto de entrada inicial do workflow como "AgenteDSA1"
workflow.set_entry_point(key = "AgenteDSA1")

# Adiciona nodes específicos das ferramentas ao workflow
workflow.add_node(node = "dsa_tool_busca", action = dsa_agente_node_tool)
workflow.add_node(node = "dsa_tool_resposta_final", action = dsa_agente_node_tool)

# Cria uma edge retornando da ferramenta 'busca' para o agente inicial
workflow.add_edge(start_key = "dsa_tool_busca", end_key = "AgenteDSA1")

# Adiciona edges condicionais partindo do node "AgenteDSA1"
workflow.add_conditional_edges(source = "AgenteDSA1", path = dsa_edges_condicionais)

######################### Humano #########################

# Adiciona um node para interação humana no workflow
workflow.add_node(node = "Human", action = dsa_human_node)

# Adiciona uma edge condicional após 'resposta_final', decidida pelo usuário
workflow.add_conditional_edges(source = "dsa_tool_resposta_final", path = dsa_human_edges)

######################### Agente 2 #########################

# Adiciona o segundo node do agente ao workflow
workflow.add_node(node = "AgenteDSA2", action = dsa_agente_node_2)

# Adiciona o node da ferramenta 'wikipedia' ao workflow
workflow.add_node(node = "dsa_tool_wikipedia", action = dsa_agente_node_tool)

# Cria uma edge retornando da ferramenta 'wikipedia' para o Agente 2
workflow.add_edge(start_key = "dsa_tool_wikipedia", end_key = "AgenteDSA2")

# Adiciona edges condicionais partindo do node "AgenteDSA2"
workflow.add_conditional_edges(source = "AgenteDSA2", path = dsa_edges_condicionais)

######################### End #########################

# Compila o workflow final para execução
g2 = workflow.compile()
```

✓ 0.0s

Mód 08 – Consultoria na área de dados e IA:

07: Definição do problema e objetivos do Cliente:

-> O primeiro passo em um projeto de consultoria de dados e IA é entender claramente o problema de negócio e os objetivos do cliente. Nessa fase, a equipe de consultoria realiza reuniões de descoberta para identificar as necessidades, desafios e metas da organização.

08: Avaliação de dados disponiveis Discovery de dados:

-> Esse passo corresponde ao Entendimento dos Dados no CRISP-DM.

Dados que o consultor deve levantar:

- **Fontes de dados:** identificar quais bancos de dados, planilhas e sistemas o cliente possui e que são relevantes para o problema.
- **Qualidade e volume:** verificar se os dados são suficientes e confiáveis, identificando dados ausentes, inconsistentes ou a necessidade de fontes externas.
- **Acesso e governança:** analisar permissões de acesso, segurança e conformidade com leis, como a **LGPD**.
- **Ferramentas e infraestrutura:** entender o ambiente tecnológico do cliente, como uso de nuvem, data warehouses (ex.: **Snowflake** e **BigQuery**) e tecnologias preferidas.

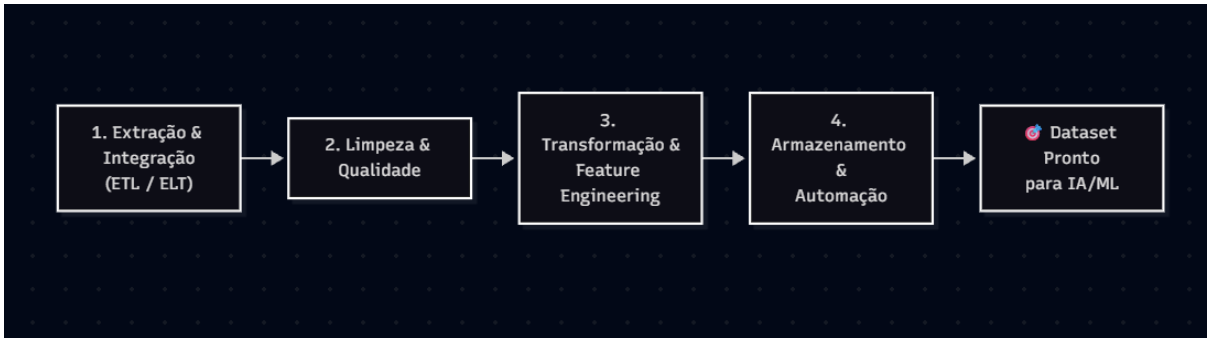
09: Planejamento do projeto e proposta de solução

-> O próximo passo é planejar a solução e o projeto. Aqui, o consultor define qual abordagem técnica será adotada e elabora um plano de projeto.

Elementos dessa fase que incluem:

-Escopo detalhado –Abordagem técnica -Cronograma e Marcos –Equipe e responsabilidades –Riscos e mitigações (ações tomadas para reduzir ou evitar riscos).

12 - Implementação técnica engenharia de dados:



1- Buscar os dados de onde quer que eles estejam e juntar tudo num lugar só.

2- Corrigir os defeitos dos dados antes de entregá-los à IA.

3- Moldar os dados para o formato que a IA/modelo matemático consegue aprender e tomar decisões.

4- Onde guardar o resultado e como garantir que o processo rode sozinho no futuro.

Dataset limpo e dividido em amostras de *treino* e *teste* pronto para treinar os modelos de Machine Learning.

--> **Machine Learning (ML)**, ou **Aprendizado de Máquina**, é uma área da Inteligência Artificial (IA) que permite que um computador **aprenda padrões a partir de dados**, sem que alguém precise programar todas as regras manualmente.

21 Especificação técnica e funcional:

1. Especificação Técnica (O *Como*)

Focada em **tecnologia, arquitetura e infraestrutura** necessárias para construir a solução:

- **Stack e Ferramentas:** Linguagens de programação, bibliotecas e frameworks.
- **Infraestrutura em Nuvem:** Servidores, serviços de computação e banco de dados (ex.: AWS EC2, Lambda, DynamoDB).
- **IA e Modelos:** Tecnologias e plataformas de execução de agentes inteligentes (ex.: Groq, DeepSeek).
- **Integrações:** APIs externas e requisitos de escalabilidade/armazenamento.

2. Especificação Funcional (O *O quê*)

Focada na **visão do usuário, regras de negócio e comportamento esperado**:

- **Comportamento do Sistema:** Quais funcionalidades serão entregues e como os usuários interagem com elas.
- **Automação e Processos:** Fluxos operacionais e processos de trabalho otimizados pela solução.
- **Critérios de Qualidade e Resultados:** Métricas esperadas, análises preditivas em tempo real, relatórios automatizados e dashboards interativos.

- **Regras de Negócio:** Diretrizes que os agentes inteligentes seguem para tomar decisões e fornece recomendações personalizadas.

22 Ferramentas usadas no projeto I:

Tecnologias Apresentadas e Justificativas

1. **Python**
    a. **Motivo:** Linguagem amigável, altamente flexível, roda em qualquer plataforma tecnológica e possui o ecossistema mais rico do mercado para Ciência de Dados, Engenharia de Dados e Inteligência Artificial.
2. **Anaconda**
    a. **Motivo:** Fornece o interpretador Python juntamente com um gerenciador de ambientes e ferramentas complementares essenciais pré-configuradas para ciência de dados.
3. **yfinance (Yahoo Finance API)**
    a. **Motivo:** O projeto envolve análise de *Day Trade* e necessita de dados financeiros de cotações de ações em tempo real. A biblioteca é gratuita, acessa a API do Yahoo Finance via Python e já entrega os dados estruturados e pré-processados.
4. **Plotly**
    a. **Motivo:** Utilizado na camada de *Analytics* (análise de dados) para criar visualizações gráficas de alta qualidade e totalmente interativas de forma integrada ao Python.

23 Ferramentas usadas no projeto II:

Tecnologias Apresentadas e Justificativas

- **Phidata (em transição para Agno / Agno AGI)**

O que é: Framework para construção e orquestração de **Agentes de IA** em Python.

- **Groq com modelo DeepSeek (LLM)**

O que é: Plataforma de inferência rápida em nuvem (Groq) servindo o modelo de linguagem de ponta DeepSeek.

Motivo/Justificativa:

- Todo agente de IA precisa de um LLM que atue como seu motor de raciocínio.
- O Groq oferece acesso gratuito via API para consumo de modelos na nuvem com alta velocidade de inferência.

➢ **Streamlit**

O que é: Framework Python para desenvolvimento de aplicações web voltadas para dados.

➢ **AWS (Amazon Web Services)**

O que é: Provedor de computação em nuvem.

Motivo/Justificativa:

- Necessidade de hospedar o aplicativo na nuvem para que o cliente acesse de qualquer lugar via internet.

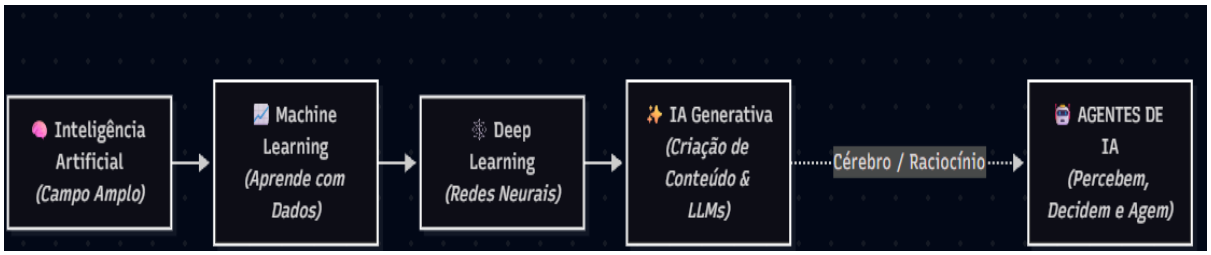
Camada	Ferramenta	Papel no Projeto
Linguagem & Ambiente	Python + Anaconda	Base de desenvolvimento e ecossistema de dados.
Ingestão de Dados	yfinance	Cotações de ações em tempo real.
Analytics & Visualização	Plotly	Gráficos analíticos interativos.
Agentes de IA	Phidata (Agno)	Orquestração dos agentes especialistas.
Cérebro / LLM	DeepSeek via Groq	Modelo de linguagem para análise e pareceres.
Interface de Usuário	Streamlit	Dashboard/Web App interativo em Python.
Infraestrutura & Deploy	AWS	Hospedagem em nuvem (PoC gratuita).

24 Construção do modulo de Analytics:

-> Analytics é o processo de coletar, analisar e interpretar dados para extrair informações úteis e apoiar decisões estratégicas. Utiliza estatística, ferramentas computacionais e inteligência artificial para identificar padrões, tendências e insights, permitindo decisões mais assertivas, reduzindo riscos e aumentando a eficiência operacional.

https://www.cienciaedados.com/agentes-de-ia-conceito-arquitetura-e-aplicacoes-parte-1/

Conceito	O que é? (Resumo Direto)	Foco Principal	Exemplo Prático
Inteligência Artificial (IA)	IA é o amplo campo da ciência da computação focado na criação de máquinas capazes de executar tarefas que normalmente exigem inteligência humana.	Criar máquinas que simulam capacidades e raciocínio humanos.	Sistemas especialistas, automações cognitivas, NPCs em jogos.
Machine Learning (ML)	Subconjunto da IA baseado em dados e estatística.	Aprender padrões a partir de dados históricos sem regras fixas codificadas a mão.	Previsão de preços, detecção de fraudes em cartão, filtros de spam.
Deep Learning (DL)	Subconjunto de ML baseado em redes neurais profundas.	Identificar padrões extremamente complexos em grandes volumes de dados.	Reconhecimento facial, visão computacional para carros autônomos.
IA Generativa (GenAI)	Ramo de IA (geralmente DL) focado em **criação**.	Gerar novos conteúdos originais (textos, imagens, áudios, códigos).	ChatGPT, Claude, Midjourney, transcrição/geração de voz.
Agentes em IA	Entidades autônomas que percebem o ambiente e agem.	**Perceber $\rightarrow$ Decidir $\rightarrow$ Agir**: tomar ações dinâmicas para atingir objetivos.	Agente de day trade que monitora mercado, decide comprar e executa a ordem sozinho.



https://hiflylabs.com/blog/2024/8/1/ai-agents-multi-agent-overview:

Os componentes típicos de sistemas multiagentes :

- **Agente orquestrador/líder:**
  Supervisiona um conjunto de modelos e processos.
- **Agente seguidor/executor:**
  Responsável por ações, decisões ou recursos dentro de áreas específicas de especialização.

- **Ferramentas e scripts:**
  O elo que mantém tudo unido — ações e chamadas de função programadas vinculam os agentes a processos predeterminados.
- **Elementos da interface do usuário:**
  Uma parte vital do design de aplicativos.
- **Infraestrutura e Ecossistema Operacional:**

-> O que é DAG (Grafo Acíclico Dirigido / *Directed Acyclic Graph*)?
Um **DAG** é uma estrutura de dados matemática em forma de grafo composta por nós (tarefas ou ações) e setas direcionadas (dependências entre elas), com uma propriedade essencial: **não existem ciclos** (ou seja, você nunca volta para um nó anterior).
São tarefas reais que serão executadas.

-> MCTs (*Monte Carlo Tree Search***):** Os ramos são **ALTERNATIVAS EXCLUSIVAS** (Dilemas / Imaginação)
No MCTS, as ramificações **NUNCA** acontecem juntas. Cada galho da árvore é uma **linha do tempo alternativa** ("e se eu fizer isso?").
Os ramos são a imaginação/simulação do agente antes de tomar uma decisão real.

https://markovate.com/blog/agentic-ai-architecture/
-> **IA Generativa:** É a **tecnologia de base** (a inteligência / o modelo de linguagem que gera texto, código e imagens).
-> Arquitetura Agêntica: É o **sistema construído em volta dessa inteligência**, dando a ela a capacidade de planejar, tomar decisões e executar tarefas no mundo real.

- **Agentes e Multiagentes:** São os **elementos práticos** dessa arquitetura. Você pode ter:
  - *Single-Agent:* Um agente sozinho cuidando de uma tarefa do início ao fim.

- *Multi-Agentes:* Vários agentes trabalhando em equipe (um pesquisa, outro analisa dados, outro escreve, outro envia e cobra).

Mód 10 – Infraestrutura e biblioteca:

Infraestrutura na Nuvem para Agentes de IA: AWS, GCP e Azure

Provedor	Foco / Principais Vantagens	Principais Serviços e Recursos de IA
AWS *(Amazon)*	• Maior diversidade de soluções • Eficiência de custo e opções de descontos por uso reservado	• **Amazon SageMaker:** Treino, ajuste e deploy de LLMs ponta a ponta • **Amazon Comprehend:** APIs prontas para tarefas de PLN (Processamento de Linguagem Natural)
GCP *(Google Cloud)*	• Inovação em IA e pesquisa • Hardware de alta performance dedicado (TPUs e GPUs)	• **Vertex AI:** Orquestração de pipelines de Machine Learning • Acesso nativo a modelos do Google (Gemini, PaLM, T5)
Azure *(Microsoft)*	• Forte integração com o ecossistema corporativo Microsoft • Padrões rigorosos de conformidade (*compliance*) e segurança	• **Azure OpenAI Service:** Acesso a modelos como GPT-4 e DALL-E com camadas corporativas de segurança e privacidade

Para rodar Agentes e Modelos de IA na Nuvem:

- Escolha **AWS** se o foco for portfólio maduro, flexibilidade e otimização de custos por compromisso.
- Escolha **GCP** se o foco for aproveitar os avanços nativos do Google (Gemini) e aceleração via TPUs.
- Escolha **Azure** se a sua empresa já está imersa no ambiente Microsoft e precisa de integração nativa com modelos OpenAI sob rígido controle de compliance.

04 Frameworks e bibliotecas:

Para construir e implantar Agentes de IA, é necessário escolher as ferramentas certas de acordo com a etapa (pesquisa, treino, inferência ou deploy em produção).

1. As Bases Principais: PyTorch vs. TensorFlow

Framework	Foco Principal	Destaques
PyTorch	**Pesquisa & Experimentação**	• Execução dinâmica/imperativa (fácil de debugar e inspecionar). • **PyTorch Lightning** para facilitar treino distribuído. • **FSDP** (*Fully Sharded Data Parallel*) para gerenciar memória ao treinar modelos gigantes.
TensorFlow	**Produção & Larga Escala**	• Pioneiro em pipelines de produção corporativos. • **TFX** (*TensorFlow Extended*) para pipelines ponta a ponta de ML. • Suporte nativo otimizado para **TPUs do Google**.

2. A Camada Superior: Hugging Face (Transformers)

Atua em um nível de abstração mais alto, sendo essencial para quem trabalha com modelos de linguagem (LLMs):

- **Modelos Prontos:** Acesso rápido ao maior repositório de LLMs open-source sem precisar recriar arquiteturas do zero.
- **APIs Padronizadas:** Facilita carregamento, tokenização, inferência e *fine-tuning*.
- **Pipelines Prontos:** Tarefas prontas para uso (resumos, perguntas e respostas, análise de sentimentos), acelerando a criação do agente funcional.

Mód 11 – Re-Ranking RAG com LangGraph e Lm Studio:

03 Agentic RAG Integração de Recuperação e Geração com Agentes de IA:

-> Os **LLMs (Large Language Models)** geram textos com alta qualidade, mas podem produzir respostas erradas com muita confiança, fenômeno chamado de **alucinação**.

Para reduzir esse problema, surgiu o **RAG (Retrieval-Augmented Generation)**, que faz o modelo buscar informações em **fontes externas** (como documentos e bases de conhecimento), tornando as respostas **mais precisas, atualizadas e contextualizadas**.

Porém, o **RAG tradicional** apenas recupera informações e fornece contexto, sem fazer uma análise aprofundada ou definir a melhor estratégia de busca.

Para resolver essa limitação, surgiu o **Agentic RAG**, que utiliza **Agentes de IA** para **analisar, planejar, decidir e priorizar** quais informações buscar e como utilizá-las. Assim, ele vai além da simples recuperação de dados, realizando uma interação mais inteligente e ativa com o conhecimento disponível.

04 Fundamentos e limitações do RAG tradicional:

-> **Limitações do RAG Tradicional (Vanilla RAG)**

- **Uma única fonte:** normalmente consulta apenas uma base de conhecimento, dificultando combinar informações de várias fontes.
- **Processo one-shot:** faz apenas **uma busca** por consulta, sem validar ou refazer a pesquisa caso os documentos recuperados não sejam os melhores.
- **Dificuldade em priorizar informações:** pode trazer muitos dados pouco relevantes e não destacar as informações mais importantes ou especializadas.
- **Compreensão limitada do contexto:** recupera documentos relacionados, mas nem sempre entende quais realmente respondem à intenção do usuário ou como integrá-los de forma coerente.

➞**Essas limitações levaram ao surgimento do Agentic RAG**, que adiciona **Agentes de IA** capazes de **planejar, raciocinar, escolher fontes, priorizar informações e realizar novas buscas quando necessário**, tornando as respostas mais inteligentes e precisas.

05 Agentes de IA e raciocínio autônomo:

- **Características dos Agentes de IA**

- **LLM com um papel definido e uma tarefa alvo.**
- **Memória**, que pode armazenar informações do diálogo e de tarefas anteriores (curto e longo prazo).
- **Planejamento e raciocínio**, usando técnicas como **Chain of Thought (CoT)** e **ReAct**, permitindo dividir problemas complexos em etapas e ajustar a estratégia.
- **Uso de ferramentas externas (Tool Use)**, como buscadores, calculadoras, bancos de dados e APIs (e-mail, calendário, etc.), ampliando suas capacidades.

Ciclo Percepção-Ação (ReAct)

O agente funciona em um ciclo contínuo:

Pensamento → Ação → Observação → (repete o ciclo, se necessário) → Resposta final

- **Pensamento:** decide a melhor estratégia.
- **Ação:** utiliza uma ferramenta ou executa uma tarefa.
- **Observação:** analisa o resultado obtido.
- **Repetição:** ajusta a estratégia e realiza novas ações até encontrar uma solução satisfatória.
- **Resposta final:** entrega a melhor resposta ao usuário.

Diferença para um LLM comum

- **LLM tradicional:** recebe a pergunta e responde **em uma única passada (one-shot)**.

- **Agente de IA: raciocina, planeja, usa ferramentas, observa resultados e pode repetir o processo** antes de responder, gerando respostas mais completas e precisas.
  - ReAct: é uma técnica/framework que permite que um **Agente de IA pense, execute ações e analise os resultados** antes de dar a resposta final.
  - Chain of Thought (CoT): é uma técnica que faz o **LLM raciocinar passo a passo** antes de chegar à resposta final. (corrente de pensamento)

<u>08 Arquitetura e Funcionamento do Agentic RAG Agente Único vs. Múltiplos Agentes:</u>

1. **Agentic RAG com Agente Único (Router)**

- Possui **um único agente central** responsável por controlar todo o processo.
- Esse agente funciona como um **roteador inteligente**, decidindo:
  - Qual fonte de conhecimento consultar (documentos internos, Web, banco de dados etc.).
  - Se precisa buscar informações ou se já consegue responder com o conhecimento disponível.
- Ele realiza **raciocínio, busca e geração da resposta sozinho**.

Vantagem: arquitetura mais simples e flexível.

Limitação: toda a responsabilidade fica em um único agente, que precisa lidar com diferentes tipos de tarefas.

2. **Agentic RAG Multiagente**

- Utiliza **vários agentes especializados** que trabalham em conjunto.
- Existe um **agente mestre (coordenador)** que:
  - Recebe a pergunta do usuário.
  - Divide a tarefa em subtarefas.
  - Distribui as tarefas para agentes especializados.
  - Junta os resultados e gera a resposta final.

Exemplo: Uma pergunta pode exigir:

- **Agente de dados internos:** busca informações da empresa.
- **Agente de dados pessoais:** consulta histórico do usuário.
- **Agente de busca externa:** pesquisa informações públicas.

Cada agente resolve sua parte e o agente mestre combina tudo.

Vantagens:

- Maior eficiência.
- Respostas mais completas.
- Permite escalar tarefas dividindo responsabilidades.

Outros tipos de agentes dentro do pipeline

Além dos agentes de recuperação de informação, podem existir agentes especializados em:

- **Routing Agent:** direciona perguntas para o caminho correto.
- **Query Planning Agent:** quebra perguntas complexas em partes menores e planeja as buscas.
- **Quality Control Agent:** revisa respostas e verifica possíveis erros.
- **Formatting Agent:** organiza os dados em uma resposta mais clara para o usuário.

09 Re-Ranking:

Re-ranking é uma técnica usada após a busca inicial de informações para **reorganizar os resultados**, colocando em primeiro lugar os documentos ou trechos mais relevantes para a pergunta do usuário.

Como funciona:

1. **Busca inicial (retrieval):**
    a. O sistema recupera vários documentos usando métodos rápidos, como **embeddings e similaridade semântica**.

    b. O objetivo é encontrar possíveis informações relevantes.
2. **Re-ranking:**
    a. Um modelo mais avançado analisa esses resultados.
    b. Ele reorganiza os documentos considerando critérios como: relevância; qualidade; precisão; contexto da pergunta.

3. **Geração da resposta:**
    a. O LLM recebe os conteúdos mais importantes e gera uma resposta mais precisa.

Vantagem do Re-ranking:

- **Velocidade:** da busca inicial por embeddings.
- **Precisão:** da análise mais profunda de modelos avançados.

Assim, reduz informações irrelevantes e garante que o LLM tenha acesso aos **melhores dados disponíveis**.

<u>10 e 11: IA generativa – Ambiente locl x API:</u>

-> API: O acesso via API é prático e dispensa hardware potente, mas exige atenção redobrada quanto à conformidade, segurança e dados sensíveis.

Exemplo: proprietário = OpenAI / OpenSource = Groq

-> Ambiente local:

Exemplo: proprietário = Eu crio meu próprio modelo / OpenSource: Llama, LM Studio, GPT4All

Nenhum dado será enviado para a internet, porém os modelos são pesados.

LM Studio com Servidor de API Local

-> O **LM Studio** será utilizado no curso:

- O LM Studio é gratuito e possui uma interface gráfica muito intuitiva.
- Ele possui a funcionalidade de **ativar um servidor local de API** (um endpoint localhost compatível com o padrão de APIs de mercado).

- **O "Melhor dos dois mundos":** A sua aplicação/código faz chamadas via API normalmente (como se estivesse chamando a OpenAI), mas a requisição é atendida 100% dentro da sua própria máquina pelo modelo open-source rodando no LM Studio, sem enviar nada para a internet.

15 Formatos GGUF e MLX:

Característica	GGUF	MLX
Principal Objetivo	Leveza, compatibilidade universal e quantização flexível	Máxima performance e eficiência em Macs
Hardware Alvo	PCs com Windows, Linux ou Mac (CPUs comuns e GPUs variadas)	Macs com Apple Silicon (M1, M2, M3, M4...)
Motor Principal	`llama.cpp`	Apple Metal / MPS
Vantagem Central	Roda modelos grandes em máquinas com poucos recursos (via INT4/INT8)	Não precisa de placas NVIDIA (CUDA) para ter performance de ponta no Mac

08- Definindo pastas:

O professor está usando:

1. **Hugging Face (Modelo de Embeddings):** É a **máquina de etiquetar**. Ela lê cada parágrafo e gera um código numérico único (o embedding) que representa o significado daquele texto.
2. **ChromaDB (Banco Vetorial):** É o **armário de arquivos**.
3. **JSON** (*JavaScript Object Notation*) é simplesmente um **formato de texto universal para organizar e trocar dados** entre computadores, programas e APIs.

12:

-> Chunks menores ajudam o modelo a identificar rapidamente informações específicas, enquanto chunks maiores preservam melhor o contexto.

15 Normalização de Embeddings:

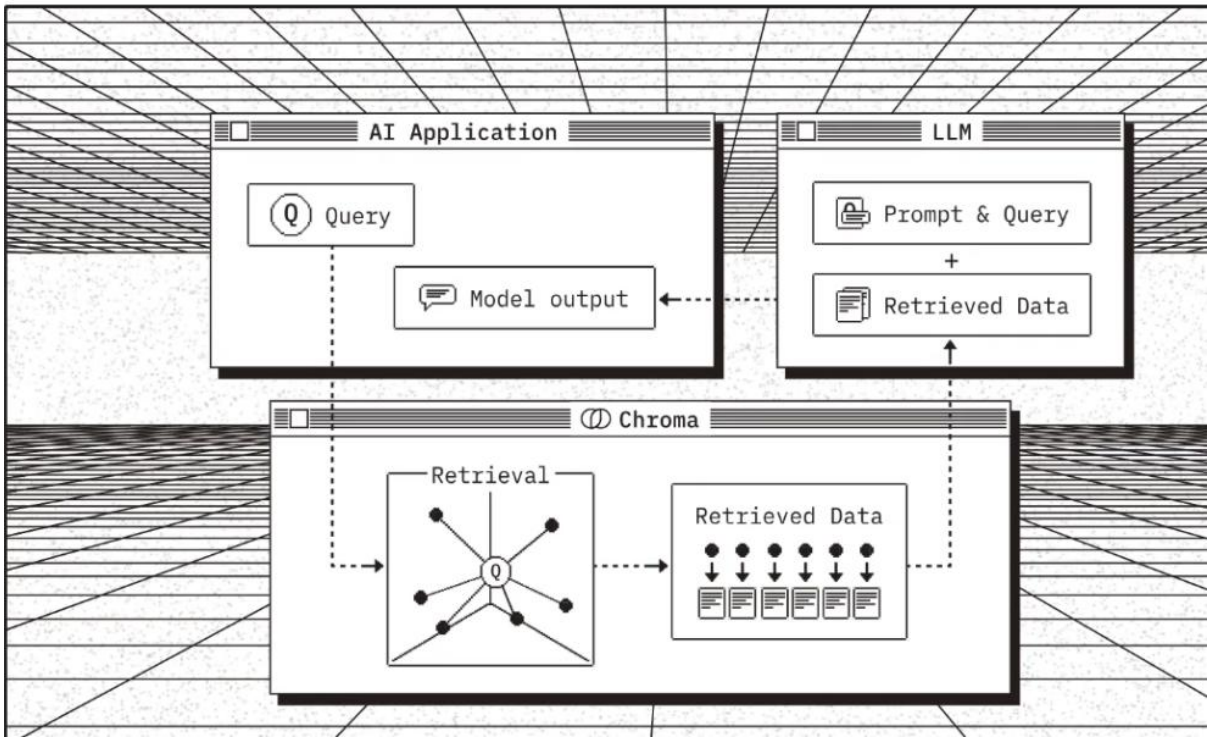
-> Em RAG, os embeddings são representações vetoriais de texto (palavras, frases, documentos). Eles capturam o significado semântico do texto, permitindo que o sistema compare a similaridade entre diferentes textos.

Modelos como OpenAI Embeddings, Sentence Transformers ou outros modelos de linguagem são usados para gerar esses embeddings.

-> É o processo de ajustar todos os vetores de embedding para que tenham o mesmo comprimento padrão (tamanho = 1), preservando apenas a direção para onde apontam.

- **Objetivo:** Garantir que a comparação entre textos avalie apenas o **significado** (direção do vetor), e não a extensão do texto ou repetição de palavras (tamanho/magnitude).
- **Como funciona:** Divide-se o vetor original pela sua própria norma (comprimento), transformando-o em um *vetor unitário*.
- **Vantagem prática:** Torna o cálculo de similaridade (como a similaridade de cosseno) muito mais justo, preciso e rápido para o banco de dados vetorial.

17:

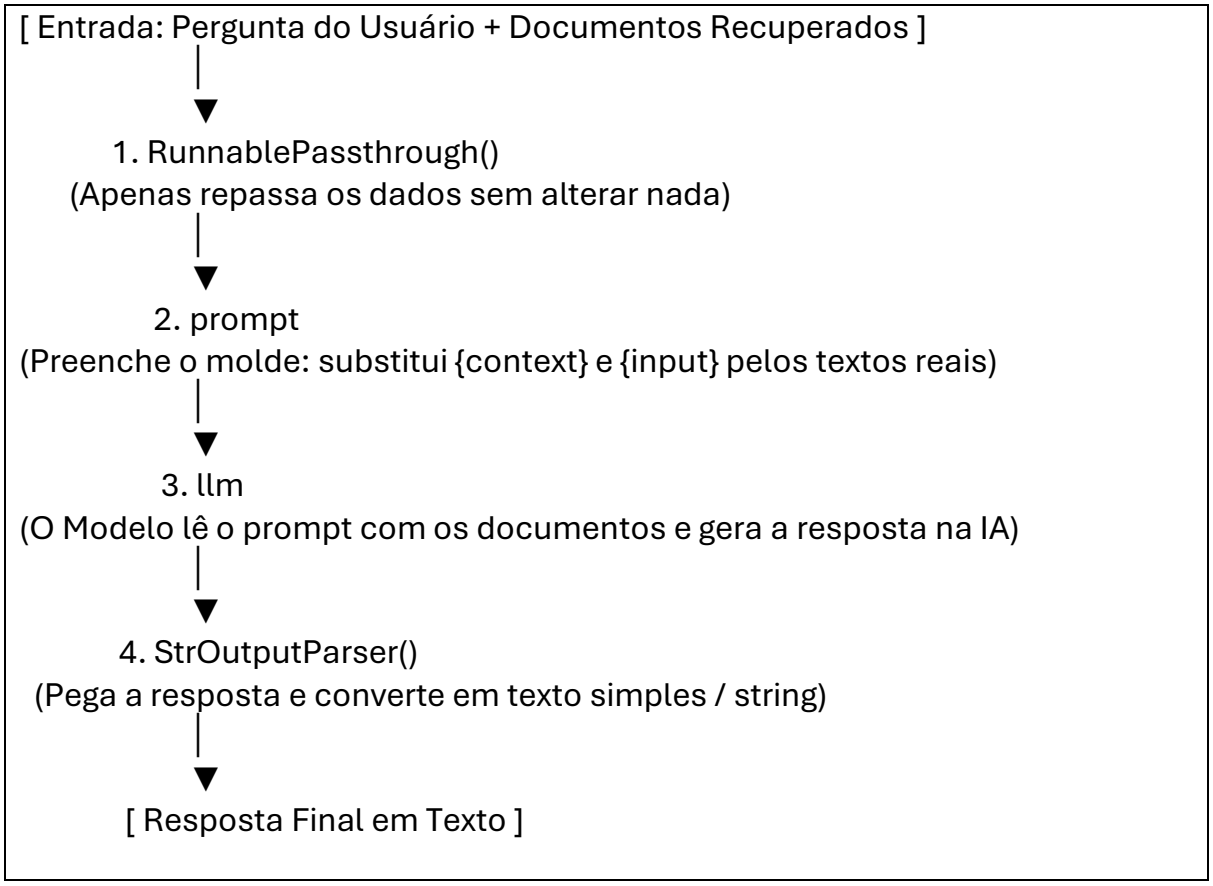


Mód 13 – R-Ranking Agente RAG com LangGraph e LLm Studio

11 Pipeline de processamento e consulta:

- combine_docs_chain = RunnablePassthrough() | prompt | llm | StrOutputParser()

É uma esteira de processamento (*pipeline*) do LangChain responsável por pegar os dados brutos (documentos + pergunta), montar a instrução final e fazer o modelo de linguagem (LLM) gerar uma resposta limpa em texto.



14 Wrapper:

- **Definição:** É uma **camada intermediária (ou adaptador)** que simplifica o acesso a serviços externos ou APIs, oferecendo uma interface padronizada e escondendo a complexidade técnica.
- **O que ele esconde (*encapsula*):** Detalhes operacionais como chamadas HTTP, autenticação, tratamento de erros e formatação dos dados recebidos.
- **Exemplo no projeto:** O DuckDuckGoSearchAPIWrapper permite fazer buscas na Web via DuckDuckGo diretamente no Python, sem precisar programar requisições web ou tratar dados na mão.
- **Principais vantagens (no LangChain):** Torna o código **padronizado, limpo, legível e fácil de manter**.

15 Definição do Estado do Agente de IA e Validação do Modelo com Pydantic:

-> O estado do agente é um conjunto estruturado de informações que descrevem o que o agente está fazendo, pensando ou planejando em um determinado momento.

-> O estado do agente:

- Guarda informações sobre a pergunta original do usuário. - Armazena resultados intermediários, como documentos ou respostas recuperadas.
- Mantém a avaliação sobre a qualidade ou relevância das respostas geradas.
- Permite ao agente decidir com base em contextos anteriores e escolher respostas mais apropriadas.

```
# Definição do estado do agente
class AgentState(BaseModel):
    query: str
    next_step: str = ""
    retrieved_info: list = []
    possible_responses: list = []
    similarity_scores: list = []
    ranked_response: str = ""
    confidence_score: float = 0.0
```

- query: Armazena a consulta original feita pelo usuário.
- next_step: Indica a próxima ação ou etapa planejada pelo agente.
- retrieved_info: Guarda as informações recuperadas de fontes externas, como bancos de dados ou documentos.
- possible_responses: Uma lista com possíveis respostas geradas pelo modelo.
- similarity_scores: Lista de pontuações que avaliam a relevância das respostas ou documentos recuperados.
- ranked_response: Guarda a resposta mais bem classificada após uma etapa de ranqueamento.

- confidence_score: Representa o nível de confiança atribuído à resposta final.

Por que utilizar o Pydantic?

- **Definição:** É uma biblioteca Python para criação de **modelos de dados estruturados**, focada em tipagem e validação rigorosa.

Principais Benefícios:

1. **Validação Automática:** Verifica tipos e regras em tempo real (ex.: se um campo deve ser texto, número ou lista), barrando dados inválidos antes que causem erros na aplicação.
2. **Tipagem e Clareza:** Deixa o código autoexplicativo e documentado, definindo com clareza quais dados cada componente espera receber.
3. **Serialização Simples:** Converte dados entre objetos Python e **JSON** com facilidade, ideal para comunicação entre APIs e ferramentas.
4. **Integração com LangChain / LangGraph:** Essencial para definir e controlar o **Estado do Agente** (*Agent State*), garantindo que a memória e as variáveis do agente permaneçam íntegras e consistentes ao longo de todo o fluxo.

★ ***O Pydantic funciona como um "fiscal de qualidade" dos dados, garantindo que o Agente de IA opere sempre com informações organizadas, válidas e padronizadas*.**

```python
# Definição do estado do agente
class AgentState(BaseModel):
    query: str
    next_step: str = ""
    retrieved_info: list = []
    possible_responses: list = []
    similarity_scores: list = []
    ranked_response: str = ""
    confidence_score: float = 0.0

# Definição dos passos de decisão do agente
def dsa_passo_decisao_agente(state: AgentState) -> AgentState:

    # Converte a consulta do usuário para letras minúsculas para padronizar a análise
    query = state.query.lower()

    # Se detectar palavras-chave que pedem explicação ou conceito geral, define o próximo passo como geração direta de resposta (sem usar RAG)
    if any(palavra in query for palavra in ["explique", "resuma", "defina", "conceito", "geral", "o que é"]):
        state.next_step = "gerar"

    # Caso detecte pedido explícito de busca atualizada na web, define o próximo passo como uso da ferramenta de busca na web (sem usar RAG)
    elif any(palavra in query for palavra in ["busque na web", "notícias", "atualizado", "recente", "últimas informações"]):
        state.next_step = "usar_web"

    # Se nenhuma das condições anteriores for atendida, usa o método padrão RAG com recuperação de documentos
    else:
        state.next_step = "retrieve"

    # Retorna o estado atualizado com a decisão tomada
    return state
```

18 Geração de múltiplas respostas:

Qual é a diferença entre retriever e qa_chain?

Componente	O que ele faz?	O que ele devolve?	Usa LLM?
retriever	É o **pesquisador/bibliotecário**. Ele só entra no banco vetorial e busca os trechos relevantes.	Os **documentos originais brutos** (lista de textos/páginas do edital).	❌ **Não.** Não escreve nada.
qa_chain	É o **pesquisador + redator**. Ele busca os documentos E pede para o LLM formular uma resposta final.	Uma **resposta escrita pronta** em português.	**Sim.**

20 Re-Ranking Avaliação de Similaridade Entre Respostas e Documentos:

-> A similaridade de cosseno é uma métrica matemática usada para medir o quanto dois vetores são semelhantes em termos da direção no espaço.

- Ela compara o ângulo formado entre dois vetores e produz um valor entre -1 e 1. --> Quando o valor é 1, significa que os vetores apontam exatamente na mesma direção, indicando alta similaridade.
  --> Se o valor é -1, eles apontam em direções opostas, ou seja, são muito diferentes.
  --> Quando o valor é 0, significa que os vetores são ortogonais (formam um ângulo de 90 graus), indicando ausência total de semelhança.

21 Processo do que foi feito:

- Defini a classe para o estado do agente.
- Definição dos passos de decisão do agente.
- Função para usar a ferramenta para busca web.
- Função para recuperar documentos.
- Função para gerar múltiplas respostas.
- Função para avaliar similaridade entre respostas e documentos.
- Função para ranqueamento das respostas geradas.

Mód 14 Prompt Chaining Reasoning CoT-Chain-of-Thought
Introdução: Iremos aprender funcionalidade de raciocínio dos Agentes de IA

02 Compreendendo o Conceito de Encadeamento de Prompts (Prompt Chaining):

-> O encadeamento de prompts é uma técnica utilizada ao interagir com modelos de linguagem de grande escala (LLMs). A ideia central é usar a saída (resposta) de um prompt como entrada (ou parte da entrada) para o próximo prompt, criando assim uma sequência ou "cadeia" de interações.

-> Por que usar usar o Encadeamento de Prompts?

- ✓ Decompor Tarefas Complexas
- ✓ Refinar Resultados

- ✓ Manter o Contexto (Controlado): Ajuda a guiar o LLM através de um fluxo de trabalho específico, garantindo que a informação relevante da etapa anterior seja explicitamente passada para a próxima.
- ✓ Combinar Habilidades: Permite combinar diferentes "habilidades" do LLM.
- ✓ Automatizar Fluxos de Trabalho: Sequências de prompts podem ser programadas para automatizar processos que requerem múltiplas etapas de geração ou análise de texto.

- × Propagação de Erros: Se um erro ocorrer em uma etapa inicial, ele pode afetar negativamente as etapas subsequentes.
- × Gerenciamento: Cadeias muito longas podem se tornar complexas de gerenciar e depurar.
- × Custo/Latência: Cada prompt na cadeia representa uma chamada separada para o LLM, o que pode aumentar o tempo total e o custo, se aplicável.

04 - Integração de Prompt Chaining com RAG Para Maior Eficácia:

Recapitulando os Conceitos:

I. Prompt Chaining: Criar uma sequência de prompts onde a saída de um passo alimenta o próximo.
II. RAG (Retrieval-Augmented Generation): Antes de gerar uma resposta, o sistema busca (retrieve) informações relevantes em uma base de conhecimento externa. Essas informações recuperadas são então fornecidas ao LLM junto com o prompt original para que ele gere uma resposta "aumentada" e fundamentada nesses dados externos.

Ideia Central: O RAG não precisa estar em todas as etapas da cadeia. Ele é acionado **apenas nos passos onde o LLM precisa consultar dados externos e específicos**.

Fluxo Típico em 3 Passos:

1. **Passo 1 — Prompt Puro (LLM):**

a. O modelo processa uma entrada inicial ou toma uma decisão.
b. *Exemplo:* Identifica qual cliente teve mais chamados.

2. **Passo 2 — Encadeamento + RAG (Busca Externa):**
    a. A saída do passo anterior define o que buscar na base de conhecimento.
    b. *Retrieve:* Busca os dados externos (ex.: chamados daquele cliente no CRM).
    c. *Augment & Generate:* O LLM lê esses dados e gera uma resposta (ex.: resume os problemas relatados).
3. **Passo 3 — Próxima Ação (Puro ou novo RAG):**
    a. A saída do passo 2 alimenta a próxima etapa:
        i. **Puro:** O LLM apenas raciocina sobre o resumo (ex.: sugere categorias para os problemas).
        ii. **RAG:** O LLM dispara uma nova busca externa (ex.: busca artigos na base de soluções para resolver o problema identificado).

Em resumo: O **Prompt Chaining** cria o fluxo lógico (passo a passo), e o **RAG** entra pontualmente como uma "ferramenta de consulta" quando o LLM precisa de dados reais e atualizados para avançar.

<u>05 Compreendendo o Conceito de Reasoning em Sistemas de Agentes de IA:</u>

-> Dentro do ciclo de um Agente de IA (Perceber -> Decidir -> Agir), o Raciocínio é o componente central do "Decidir". É a capacidade computacional do agente de "pensar" de forma estruturada para determinar a melhor ação ou sequência de ações a serem tomadas, dada a situação atual (percepção) e seus objetivos finais.

<u>06 Compreendendo o Conceito de CoT (Chain-of-Thought):</u>

Cadeia de pensamento =
-> É uma técnica de prompting (ou seja, uma forma de dar instruções) usada com Modelos de Linguagem Grandes (LLMs).
-> O objetivo principal é melhorar a capacidade do modelo de raciocinar sobre problemas complexos, especialmente aqueles que exigem

múltiplos passos lógicos, matemáticos ou de senso comum para serem resolvidos corretamente. A ideia central é mostrar o processo, não apenas o resultado.

Existe duas maneiras:

1. Zero-shot CoT: Adicionar uma simples instrução ao final do prompt.
2. Few-shot CoT: Fornecer alguns exemplos (2 ou 3 geralmente bastam) no próprio prompt, antes da pergunta real. Cada exemplo demonstra o formato desejado: Pergunta -> Raciocínio Passo a Passo -> Resposta Final. Ao ver esses exemplos, o modelo "aprende" a seguir o mesmo padrão para a nova pergunta.

14: Construindo a geração de embbedings:

embeddings.py

```python
# Projeto 4 - Encadeamento de Prompts, Raciocínio, CoT e Agentic RAG em Contexto Multi-Agentes Para App de Análise de Contratos

# Import para interação com o sistema operacional
import os

# Import para geração de embeddings
from langchain_huggingface import HuggingFaceEmbeddings

# Desativar paralelismo na tokenização para garantir compatibilidade entre sistemas operacionais
os.environ["TOKENIZERS_PARALLELISM"] = "false"

# Definir classe para gerenciar embeddings
class Embeddings:

    # Inicializar classe com modelo específico de embeddings
    def __init__(self):

        # Nome do modelo
        model_name = "BAAI/bge-base-en"

        # Cria instância da classe
        self.model = HuggingFaceEmbeddings(model_name = model_name)

    # Método para retornar o modelo de embeddings inicializado
    def get_embedding_model(self):
        return self.model
```

- O script embeddings.py tem como objetivo definir uma classe responsável pela configuração e gerenciamento de um modelo de embeddings.
- TOKENIZERS_PARALLELISM como "false", uma prática comum para evitar conflitos ou problemas de paralelismo em alguns ambientes. (Paralelismo é a capacidade de um programa dividir uma tarefa em vários pedaços e executá-los **ao mesmo tempo**, usando múltiplos núcleos do seu processador (CPU) para ir mais rápido.)

- No construtor __init__, é especificado o modelo "BAAI/bge-base-en", que é conhecido por sua performance eficiente em tarefas de busca semântica e similaridade textual.
- O método get_embedding_model serve como uma interface para outras partes do sistema acessarem esse modelo de embeddings.

15 Construindo o Módulo de RAG e Recuperação Para o Agentic RAG - Parte 1/5:

Classe Retriever (retriever.py)

- **Objetivo Principal:** Implementar o pipeline de Recuperação de Informação (RAG) semântica sobre contratos para que Agentes de IA consultem e recuperem trechos relevantes baseados em perguntas/comandos.

Fluxo de Funcionamento e Métodos:

1. **Inicialização:**
    a. Recebe um modelo de **embeddings** pré-configurado para converter textos em vetores numéricos, permitindo a busca por proximidade de significado (similaridade semântica) e não apenas correspondência exata de palavras.
2. **Carregamento (dsa_carrega_documentos):**
    a. Varre um diretório e extrai o conteúdo de arquivos PDF utilizando a biblioteca **PyMuPDFLoader**, armazenando-os em memória.
3. **Indexação (dsa_cria_vectordb):**
    a. **Divisão de Texto:** Utiliza o RecursiveCharacterTextSplitter para fragmentar os documentos em partes menores (chunks), aumentando a precisão da busca.
    b. **Banco Vetorial:** Cria um índice vetorial usando **FAISS**, otimizado para consultas rápidas de similaridade entre vetores.
4. **Recuperação (dsa_retrieve):**
    a. Recebe uma pergunta (*query*), busca no índice FAISS e retorna os **k trechos mais relevantes**.

- **Papel no Sistema:** Componente essencial de **RAG (Retrieval-Augmented Generation)**, fornecendo aos LLMs contexto dinâmico, atualizado e rastreável a partir de uma base contratual.

retriever.py

```python
# Definir classe Retriever para carregar e recuperar documentos
class Retriever:

    # Inicializar classe com modelo de embedding fornecido
    def __init__(self, embedding):

        # Atribuir modelo de embedding à instância da classe
        self.embedding = embedding.get_embedding_model()

        # Inicializar lista para armazenar documentos carregados
        self.documents = []

        # Inicializar variável para armazenar o índice vetorial
        self.vector_store = None

    # Carregar documentos PDF do caminho especificado
    def dsa_carrega_documentos(self, documents_path):

        # Iterar sobre os arquivos no diretório fornecido
        for filename in os.listdir(documents_path):

            # Verificar se o arquivo tem extensão .pdf
            if filename.endswith(".pdf"):

                # Inicializar carregador de documentos PDF com o caminho do arquivo
                loader = PyMuPDFLoader(os.path.join(documents_path, filename))

                # Carregar o conteúdo do documento PDF
                loaded_docs = loader.load()

                # Adicionar documentos carregados à lista de documentos
                self.documents.extend(loaded_docs)
```

```python
    # Construir índice vetorial utilizando fragmentos dos documentos
    def dsa_cria_vectordb(self):

        # Inicializar o divisor de texto com tamanho e sobreposição de fragmentos
        splitter = RecursiveCharacterTextSplitter(chunk_size = 1000, chunk_overlap = 200)

        # Fragmentar os documentos carregados
        docs_split = splitter.split_documents(self.documents)

        # Criar o índice vetorial a partir dos documentos fragmentados e embedding
        self.vector_store = FAISS.from_documents(docs_split, self.embedding)

    # Recuperar documentos mais relevantes a partir da consulta do usuário
    def dsa_retrieve(self, query, k = 5):

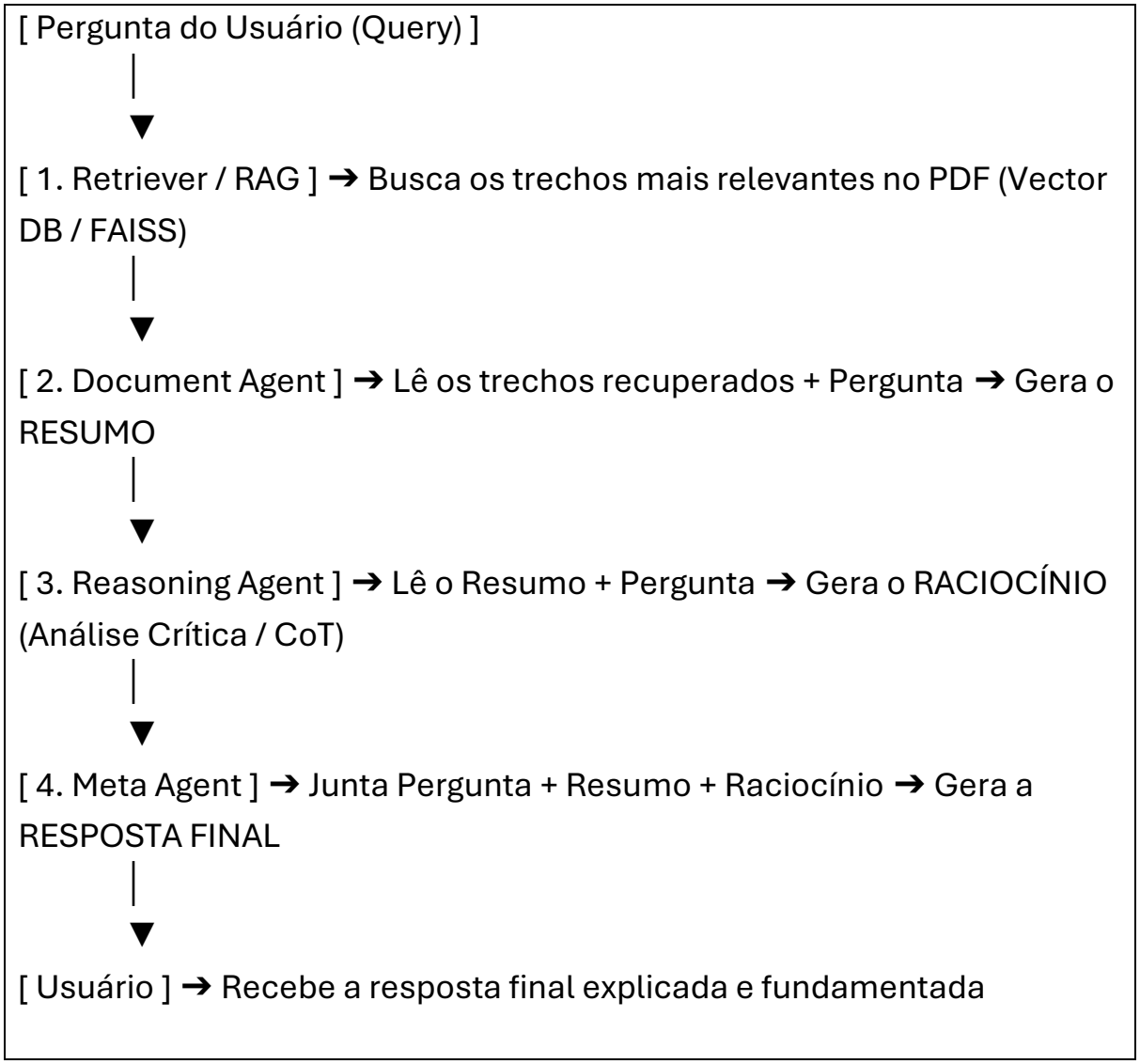
        # Verificar se o índice vetorial foi inicializado
        if not self.vector_store:

            # Lançar erro se o índice vetorial não estiver disponível
            raise ValueError("Vector store não está inicializada.")

        # Realizar busca de similaridade no índice vetorial com a consulta fornecida
        docs = self.vector_store.similarity_search(query, k = k)

        # Retornar os documentos mais relevantes encontrados
        return docs
```

20:



O que é o "Estado Compartilhado" (AgentState)?

Pense no **Estado Compartilhado** do LangGraph como uma **prancheta/ficha** que vai sendo passada de mão em mão entre os agentes. Cada um preenche a sua parte:

1. **Início:** A prancheta tem apenas a query (pergunta).
2. **Retriever (recuperar ou buscar) + Document Agent:** Adicionam documents e o summary (resumo).
3. **Reasoning Agent:** Adiciona o reasoning (raciocínio lógico).
4. **Meta Agent:** Lê tudo o que foi anotado antes e grava a final_answer (resposta final).

3. Papel de Cada Agente (Resumo Rápido)

- **Document Agent** *(LLM: Llama 3.1 8B)*: Extrai o fato bruto dos documentos e sintetiza.
- **Reasoning Agent** *(LLM: Llama 3.3 70B)*: Pensa criticamente sobre o que foi resumido, avaliando cláusulas, impactos e condições.
- **Meta Agent** *(LLM: Qwen 2.5 32B)*: O "gerente" que lapida, organiza e entrega a resposta final clara e amigável.

22:

dsa_reasoning_agente.py

```python
# Projeto 4 - Encadeamento de Prompts, Raciocínio, CoT e Agentic RAG em Contexto Multi-Agentes Para App de Análise de Contratos

# Importar cliente Groq para interação com modelos de IA
from groq import Groq

# Definir classe para o agente especializado em raciocínio lógico
class ReasoningAgent:

    # Inicializar a classe com a chave da API Groq
    def __init__(self, api_key):
        self.client = Groq(api_key = api_key)

    # Método para gerar raciocínio lógico a partir do resumo e da consulta
    def dsa_gera_raciocinio(self, summary, query):

        # Definir mensagens para enviar ao modelo de IA
        messages = [
            {"role": "system", "content": "Você é especialista em raciocínio lógico sobre textos."},
            {"role": "user", "content": f"Com base no resumo: {summary}\n\nFaça uma análise crítica para responder à pergunta: {query}"}
        ]

        # Criar solicitação ao modelo usando parâmetros específicos
        completion = self.client.chat.completions.create(model = "llama-3.3-70b-versatile",
                                                         messages = messages,
                                                         temperature = 0.7,
                                                         max_tokens = 1024)

        # Retornar o raciocínio lógico gerado pelo modelo, removendo espaços adicionais
        return completion.choices[0].message.content.strip()
```

-> Segundo agente é alimentado pelo sumary = resultado do primeiro agente e a pergunta do usuário.

->self no Python: É uma referência interna à **própria classe/objeto**. Ele **não é um dado de entrada** enviado pelo usuário ou pelo fluxo, mas sim a forma como o agente consegue acessar suas próprias ferramentas internas (como a chave e a conexão da API self.client)

Mód 15 Routing (roteamento) e multi agentes de IA:
02 - O Que é e Como Configurar o Roteamento de Tarefas Entre Agentes de IA?

-> É o processo de direcionar uma tarefa específica para o Agente de IA mais adequado para executá-la dentro de um sistema que contém múltiplos agentes. Ou seja, é o **mecanismo de decisão de caminho.**

- Uma nova tarefa (um pedido, uma pergunta, um problema) chega ao sistema.
- O "roteador" (que pode ser ele mesmo um agente simples ou um conjunto de regras) analisa a tarefa.
- Com base nas características da tarefa e nas capacidades conhecidas dos diferentes Agentes de IA disponíveis, o roteador decide qual agente é o melhor para lidar com aquela tarefa específica. A tarefa é então enviada para o agente selecionado ou para o LLM adequado.

-> Uma das vantagens de usar roteador é a resiliência: Se um agente falhar, o roteador pode (dependendo da configuração) redirecionar a tarefa para outro agente capaz.

Característica	**Roteador Puro (Router / Dispatcher)**	**Agente Chefe (Supervisor / Orchestrator)**
Papel principal	**Triagem / Despacho:** Decide apenas para onde a tarefa vai ("*A tarefa X vai para o Agente Y*").	**Gerenciamento / Coordenação:** Decompõe planos, delega, cobra resultados e sintetiza a resposta final.
Complexidade	**Simples e rápido:** Frequentemente **nem é um agente de IA**. Pode ser um `if/else`, palavras-chave, similaridade de embeddings ou um LLM pequeno/barato.	**Agente completo:** Usa um LLM avançado com memória de contexto, raciocínio e ferramentas de planejamento.
Ciclo de vida	**Fim imediato:** Depois que ele encaminha a mensagem para o especialista, o trabalho dele terminou.	**Contínuo:** Ele aguarda a resposta do subordinado, avalia a qualidade e pode chamar outro agente em seguida.
Tratamento da tarefa	Geralmente escolhe **um único** especialista para resolver a tarefa inteira.	Quebra uma tarefa grande em **várias subtarefas** entre múltiplos especialistas e junta as partes.

RESUMO:

- **Roteador: decide para onde vai. /// Agente Chefe: decide o que fazer, quem faz, quando faz e se ficou bom.**

5 Etapas para Configurar Roteamento de Agentes

1. Mapear Agentes e Capacidades

- **Catálogo:** Listar agentes com suas especialidades, limitações e formatos esperados de entrada (*input*) e saída (*output*).

2. Classificar as Tarefas de Entrada

Critérios para identificar a intenção do usuário:

- **Palavras-chave:** Termos explícitos (ex: *"resumir"*, *"traduzir"*).
- **Metadados / Formato:** Tipo de dado recebido (texto, código, imagem).
- **Análise de Intenção:** Um modelo leve que identifica o objetivo da mensagem antes de despachar.

3. Escolher a Lógica do Roteador (O "Cérebro")

- **Baseada em Regras (IF/ELSE):** Simples e rápida, mas rígida para sistemas grandes.
- **Por Habilidades (*Skills*):** Cruzamento de tags necessárias (*ex: requer python + dados*).
- **Por Carga (*Load*):** Envia para o agente que estiver menos ocupado.
- **Por Custo/Performance:** LLM barato para tarefas simples, LLM potente para complexas.
- **Por ML / Semântica:** Classificador ou busca vetorial (embeddings) para decidir a rota. = **significado/sentido.**
- **Híbrida:** Combina métodos (ex: filtra por habilidade e desempata pelo menos ocupado). = palavras-chave exatas + busca semântica por significado.

4. Implementar o Despachante (*Pipeline*)

- **Infraestrutura:** Criar a função ou serviço que recebe a tarefa, roda a lógica e despacha para o agente escolhido via API ou fila de mensagens.

5. Monitorar e Ajustar

- **Métricas:** Acompanhar taxa de acerto do roteador, tempo de resposta (*latência*) e falhas de agentes para calibrar as regras continuamente.

o balanceamento de carga em sistemas multi-agentes foca em distribuir o

04 - Balanceamento de Carga em Sistemas de Multi-Agentes de IA :

-> o balanceamento de carga em sistemas multi-agentes foca em distribuir o trabalho de forma equitativa entre "trabalhadores" (instâncias de agentes) idênticos ou semelhantes para manter o sistema eficiente, rápido e confiável.

Ele opera frequentemente em conjunto com o roteamento de tarefas (que decide o tipo de agente) e a alocação dinâmica de recursos (que ajusta o número de instâncias disponíveis).

06 - Ferramentas Para Implementar Roteamento Eficiente:

Camada	Ferramentas	Ponto Forte / Tipo de Roteamento
Frameworks de Agentes	LangChain, AutoGen, CrewAI	Roteamento semântico, conversacional e por papéis
Orquestradores de Workflow	Airflow, Prefect, Camunda	DAGs, lógica de negócio complexa e controle de estado
Mensageria / Filas	RabbitMQ, Kafka, Redis Streams	Desacoplamento, alta resiliência e processamento assíncrono
Infraestrutura de Rede	Kong, Apigee, Istio	Balanceamento de tráfego e versionamento de agentes
Customizado	Python + FastAPI + scikit-learn	Lógica preditiva sob medida e controle total

1. Frameworks Específicos para LLMs e Agentes de IA

Projetados nativamente para lidar com Modelos de Linguagem e fluxos conversacionais/agênticos:

- **LangChain:**
  - **Mecanismo:** Utiliza abstrações como **Router Chains**.
  - **Como funciona:** Direciona dinamicamente as requisições para diferentes cadeias de processamento ou agentes, avaliando o conteúdo do prompt ou aplicando regras predefinidas.
- **Microsoft AutoGen:**
  - **Mecanismo: Roteamento implícito e colaborativo**.

    - **Como funciona:** O fluxo é guiado pela dinâmica da conversa e pela descrição de capacidades de cada agente, permitindo que a interação entre eles determine o próximo passo.
- **CrewAI:**
    - **Mecanismo: Roteamento baseado em papéis (*Role-based*)**.
    - **Como funciona:** Organiza os agentes em "equipes", onde as tarefas são delegadas e distribuídas de acordo com as funções, objetivos e especialidades de cada agente.

2. Ferramentas de Orquestração de Fluxos de Trabalho Genéricos e Motores BPMN

Ideais para fluxos de longa duração, regras de negócio complexas e integração com ecossistemas corporativos:

- **Apache Airflow & Prefect:**
    - **Mecanismo:** Grafos Acíclicos Direcionados (**DAGs**).
    - **Como funciona:** Cada agente ou tarefa representa um nó no grafo; a transição entre os nós define regras condicionais detalhadas de roteamento e dependências.
- **Motores de BPMN (ex.: Camunda):**
    - **Mecanismo:** Modelagem visual e execução de processos de negócios.
    - **Como funciona:** Mantém o controle de estado e roteia tarefas entre agentes de IA e etapas humanas/sistemas legados em fluxos corporativos complexos.

3. Infraestrutura de Mensageria (Desacoplamento e Resiliência)

Camada essencial para garantir escalabilidade, processamento assíncrono e tolerância a falhas:

- **RabbitMQ, Apache Kafka e Redis Streams:**
    - **Mecanismo:** Filas de mensagens e arquitetura orientada a eventos (tópicos).
    - **Como funciona:** O roteador publica tarefas em filas/tópicos específicos e os agentes consomem sob demanda,

viabilizando **balanceamento de carga**, **processamento assíncrono** e **desacoplamento total** entre roteador e agentes.

4. Nível de Rede: API Gateways e Service Meshes

Focados em roteamento no nível de tráfego de rede:

- **API Gateways (Kong, Apigee) e Service Meshes (Istio):**
  - **Mecanismo:** Roteamento de requisições HTTP/gRPC e políticas de rede.
  - **Como funciona:** Aplicam regras mais diretas de balanceamento de carga e direcionamento de tráfego (ex.: envio de chamadas para diferentes versões de um agente ou deploys canários).

5. Soluções Customizadas (*In-House*)

Para cenários com regras muito específicas ou necessidades de integração sob medida:

- **Stack: Python + FastAPI/Flask + scikit-learn (Machine Learning):**
  - **Mecanismo:** Lógica própria preditiva ou baseada em regras.
  - **Como funciona:** Classificadores de ML ou heurísticas personalizadas analisam a entrada e tomam a decisão de roteamento com controle absoluto sobre a execução.

14 - Implementação - Parte 6/10:

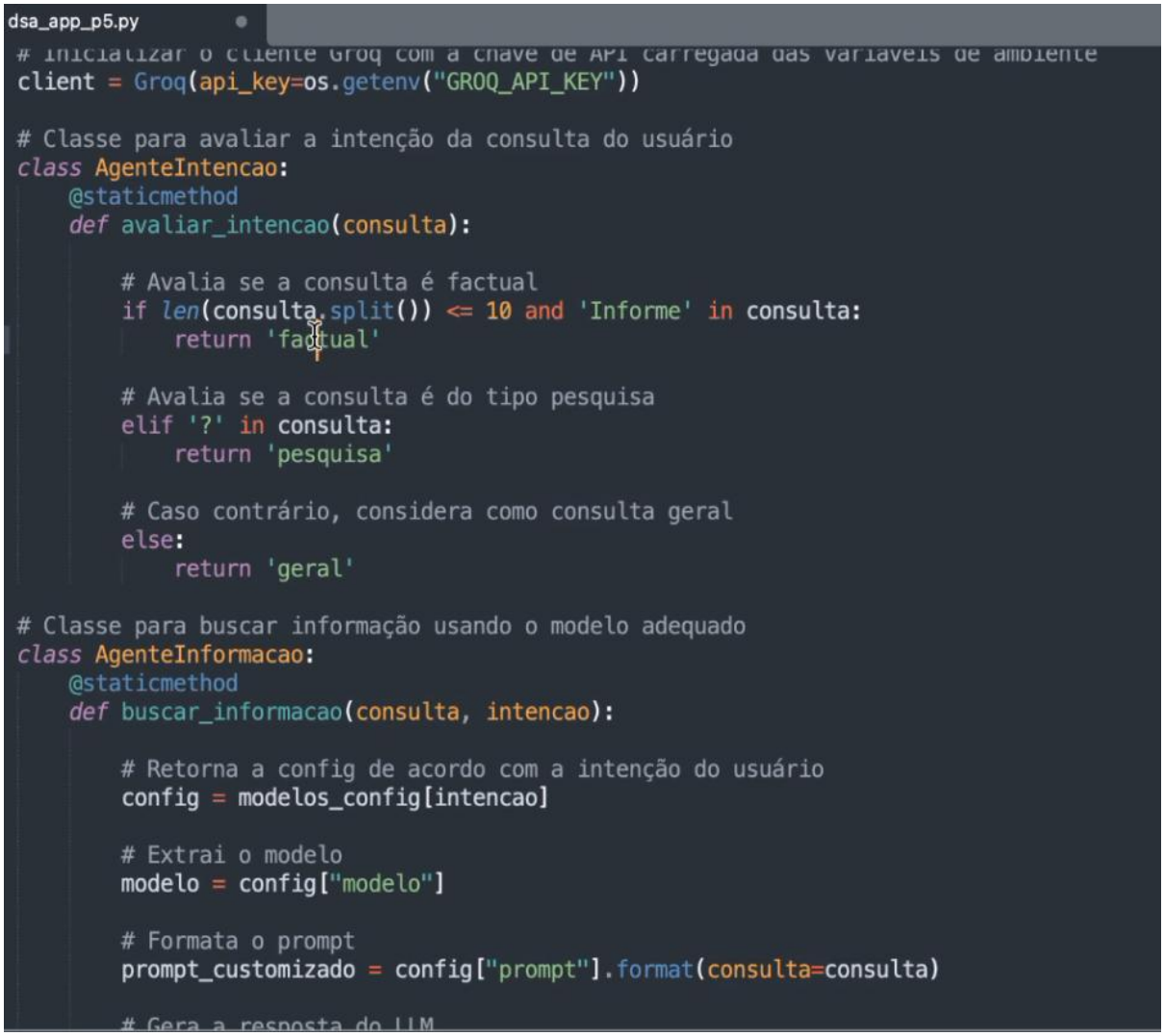
-> Em Python, o **@staticmethod** transforma um método em uma **função comum**, mas que fica guardada **dentro de uma classe.**

Com @staticmethod (não usa self nem cls):

A função **não mexe em nada da classe nem do objeto**. Ela só pega o que você passa nos parênteses, processa e devolve o resultado:

- Organização: agrupa funções relacionadas à classe, mesmo que elas não precisem da instância.
- Clareza: deixa explícito que o método não depende do estado interno da classe/instância.
- Eficiência: evita passar self ou cls desnecessariamente.

Se você tiver um método que só faz algum processamento com dados de entrada, sem precisar de atributos internos da classe, @staticmethod é o ideal.


dsa_app_p5.py

```
# Inicializar o cliente Groq com a chave de API carregada das variáveis de ambiente
client = Groq(api_key=os.getenv("GROQ_API_KEY"))

# Classe para avaliar a intenção da consulta do usuário
class AgenteIntencao:
    @staticmethod
    def avaliar_intencao(consulta):

        # Avalia se a consulta é factual
        if len(consulta.split()) <= 10 and 'Informe' in consulta:
            return 'factual'

        # Avalia se a consulta é do tipo pesquisa
        elif '?' in consulta:
            return 'pesquisa'

        # Caso contrário, considera como consulta geral
        else:
            return 'geral'

# Classe para buscar informação usando o modelo adequado
class AgenteInformacao:
    @staticmethod
    def buscar_informacao(consulta, intencao):

        # Retorna a config de acordo com a intenção do usuário
        config = modelos_config[intencao]

        # Extrai o modelo
        modelo = config["modelo"]

        # Formata o prompt
        prompt_customizado = config["prompt"].format(consulta=consulta)

        # Gera a resposta do LLM
```

-> É nessa classe e parte que o método

avaliar_intencao funcionam como o **roteador**. Eles inspecionam a pergunta e decidem qual é a rota ('factual', 'pesquisa' ou 'geral').

-> Na parte de retornar a config ... = O Despacho Dinâmico (Executando a Rota Escolhida)

No método
buscar_informacao da classe
AgenteInformacao, o valor retornado pelo roteador (intencao) é usado como chave para selecionar o modelo de LLM e o prompt adequados definidos no arquivo

groq_modelos_config.json
-> O **roteamento** é o conjunto dessa lógica:

1. Um componente que **avalia a pergunta** (regras com if/elif, outro LLM classificador ou similar).
2. Um mapa/dicionário que **associa cada tipo de pergunta ao agente ou modelo ideal**.
3. O direcionamento da execução para o caminho escolhido.

OBS: **roteamento (routing) não é um comando ou função nativa do Python** (como um import routing). Ele é um **padrão de arquitetura / técnica de design** implementado através da lógica do código.

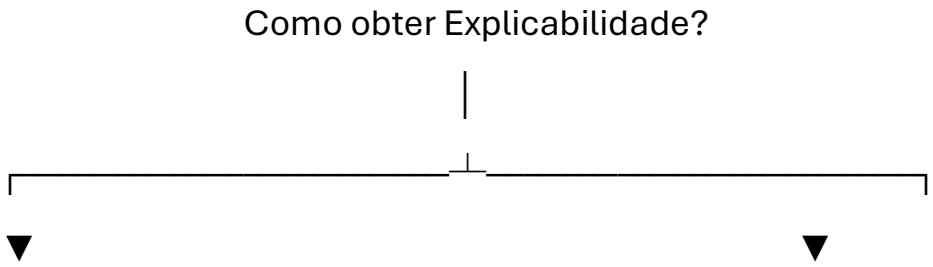
Mód 16 - Explicabilidade-Observabilidade-Guardrails-Monitor:
02 Explicabilidade de Modelos de Inteligência Artificial:

-> É a **capacidade de um sistema de Inteligência Artificial comunicar e justificar como chegou a determinada decisão ou previsão**, de forma compreensível para seres humanos.
É um conjunto de técnicas e métodos matemáticos.
As redes neurais profundas (Deep Learning), funcionam como "caixas-pretas".

Motivo	O que significa na prática?
1. Confiança e Adoção	Profissionais e usuários finais (ex: médicos, analistas de crédito, engenheiros) só confiam e implementam ferramentas de IA na rotina quando entendem o porquê de cada recomendação.
2. Detecção e Correção de Erros (*Debugging*)	Quando o modelo falha, a explicabilidade indica exatamente onde e por que o erro aconteceu, permitindo ajustes precisos em vez de tentativas cegas.
3. Identificação de Vieses (*Bias*) e Justiça (*Fairness*)	Ajuda a auditar se o modelo aprendeu preconceitos dos dados históricos (como discriminação de gênero, raça ou renda), permitindo corrigir distorções éticas.

4. Conformidade Regulatória	Atende a normas legais (como **LGPD** no Brasil e **GDPR** na Europa), que garantem ao cidadão o direito à explicação sobre decisões tomadas de forma 100% automatizada.
5. Segurança e Robustez Crítica	Em áreas de alto risco (medicina diagnóstica, aviação, carros autônomos), entender as decisões da IA é questão de vida ou morte para antecipar comportamentos em cenários imprevistos.
6. Melhoria Contínua e Descoberta de Conhecimento	Entender os padrões que a IA encontrou nos dados pode gerar novos *insights* de negócio e até impulsionar novas descobertas científicas.

03 Como se Obtém a Explicabilidade?



1- Modelos Intrinsecamente Explicáveis ("Caixa de Vidro")

São modelos **simples por natureza**. A própria fórmula ou lógica interna já é aberta e fácil de auditar.

- **Analogia:** Um fluxograma ou uma receita de bolo passo a passo.

1. Métodos Pós-Hoc ("Raio-X da Caixa-Preta")

Usados quando o modelo é muito complexo (como redes neurais profundas) e não conseguimos enxergar seu interior diretamente.
A técnica é aplicada **depois (*post-hoc*) que o modelo já foi treinado**.
Dois dos métodos mais famosos:

- **LIME (*Local Interpretable Model-agnostic Explanations*):**

    - **O que faz:** Foca em **decisões pontuais/individuais**.
    - **Como funciona:** Explica por que o modelo tomou *aquela* decisão específica para *aquele* caso (ex.: "Por que o cliente João teve o crédito negado hoje? Porque a variável 'dívidas recentes' pesou negativamente").
- **Vantagens:**
    - **Agnóstico ao Modelo:** Funciona com qualquer algoritmo (redes neurais, florestas, etc.) sem precisar saber como ele foi feito por dentro.
    - **Intuitivo:** A lógica de "aproximar o redor de um ponto com uma linha reta" é muito fácil de entender.
    - **Flexível:** Aceita dados tabulares (planilhas), textos e imagens.
    - **Mais Rápido:** Geralmente é mais rápido para gerar uma explicação pontual do que calcular valores exatos de Shapley.
- **Desvantagens:**
    - **Instabilidade:** Como gera perturbações aleatórias, rodar o LIME duas vezes para o mesmo dado pode gerar explicações ligeiramente diferentes.
    - **Definição da Vizinhança:** Definir o que é "perto" do ponto analisado é uma escolha arbitrária que pode mudar o resultado.
    - **Apenas Local:** Não te dá a visão global de como o modelo inteiro funciona.
    - **Fidelidade Limitada:** Se a decisão do modelo naquele ponto for muito complexa/curva, uma reta simples (modelo aproximado) não vai refletir a realidade com exatidão.

❖ **SHAP (*SHapley Additive exPlanations*):**

- **O que faz:** Baseia-se na **Teoria dos Jogos** (Valores de Shapley).
- **Como funciona:** Trata cada característica de entrada como um jogador em uma equipe e mede exatamente quanto cada uma contribuiu para afastar a previsão final da média de referência.

- **Vantagens:**

    - **Fundamentação Matemática Sólida:** Baseado em princípios formais (Valores de Shapley), garantindo justiça e a propriedade de aditividade (a soma dos impactos bate exatamente com o resultado final).
    - **Local e Global:** Explica tanto uma linha individual quanto o comportamento de todo o modelo quando agregamos as análises.
    - **Mais Robusto e Estável:** Não sofre da aleatoriedade do LIME; o resultado é consistente.
    - **Excelentes Visualizações:** Possui gráficos ricos e consagrados no mercado (*beeswarm plots, force plots, waterfall plots*).
- **Desvantagens:**
    - **Alto Custo Computacional:** Comparar todas as combinações possíveis exige muito processamento (o KernelSHAP pode ser muito lento em bases grandes, embora existam versões otimizadas como TreeSHAP para árvores e DeepSHAP para redes neurais).
    - **Curva de Aprendizado:** Interpretar seus gráficos e fundamentos exige mais conhecimento técnico.
    - **Suposição de Independência:** Algumas versões assumem que as variáveis não dependem umas das outras, o que nem sempre é verdade na prática.

--> Lime apenas local.

--> Shap local e global.

<u>05 - Explicabilidade e Observabilidade de Agentes de IA:</u>

-> A Observabilidade fornece os dados e a infraestrutura (logs, métricas, traces) que permitem a Explicabilidade. Você precisa observar detalhadamente o que o agente fez (Observabilidade) para poder começar a investigar e entender por que ele fez (Explicabilidade).

<u>06 - Ferramentas LangSmith e Pydantic LogFire O Que São e Quando Usar?</u>

-> Essas são duas ferramentas modernas que ajudam a implementar Observabilidade (e, por consequência, auxiliam na Explicabilidade) em aplicações, incluindo Agentes de IA.

- LangSmith: Uma plataforma desenvolvida pela LangChain, projetada especificamente para depuração, teste, avaliação e monitoramento de aplicações construídas com LLMs, incluindo Agentes complexos.

Foco Principal: Ciclo de vida de aplicações LLM e Agentes.

Por ser cíclico ajuda a ter rastreamento detalhado, depuração.

-> Ele fornece a infraestrutura de Observabilidade de ponta a ponta (os traços detalhados) especificamente para o ecossistema LLM/Agente. Esses traços são a base para entender o comportamento do agente e, assim, alcançar a Explicabilidade, permitindo analisar por que uma sequência específica de ações foi tomada.

- Pydantic Logfire: Uma plataforma/biblioteca de observabilidade moderna para aplicações Python, desenvolvida pela equipe por trás do Pydantic. Ela se integra fortemente com o ecossistema Python e adota padrões abertos como OpenTelemetry.

Logfire fornece a camada fundamental de Observabilidade (logs, traces, métricas) para qualquer aplicação Python, incluindo Agentes de IA desenvolvidos em Python.

07 - Guardrails Para Aumentar a Segurança em Soluções com Agentes de IA:

-> "Guardrails" (ou barreiras de proteção) são um conceito fundamental para aumentar a segurança, confiabilidade e o comportamento ético em soluções que utilizam Agentes de IA, especialmente aqueles baseados em Grandes Modelos de Linguagem (LLMs).

--> Por que Guardrails são Essenciais para Agentes de IA?

- ❑ A autonomia dos agentes, combinada com as características dos LLMs, apresenta riscos: pode alucinar, usar ferramentas perigosas, desvio de finalidade, vazamento de informações, loops e custos excessivos.

Riscos Éticos e de Reputação: Ações ou respostas inadequadas podem violar princípios éticos, leis ou prejudicar a reputação da organização.

08 - Como Funcionam os Guardrails (Tipos e Exemplos):

-> Podem ser implementados em diferentes pontos do fluxo de trabalho de um Agente de IA.

Tipo	Momento de Ação	Função Principal	Exemplos
Entrada (*Input*)	Antes do processamento pelo LLM	Proteger o agente contra entradas inadequadas ou maliciosas.	• Bloqueio de *prompt injection* • Filtro de tópicos fora de escopo • Mascaramento de dados pessoais (PII)
Saída (*Output*)	Após a geração da resposta pelo LLM	Garantir integridade, formato e segurança antes da entrega.	• Filtro de toxicidade ou alucinações • Validação de schemas estruturados (ex: JSON válido) • Respostas padronizadas de recusa
Procedimentais (*Behavioral*)	Durante o planejamento e execução de ações	Controlar o uso de recursos e ferramentas.	• Limite de chamadas de API, passos e custo • Bloqueio de comandos perigosos (ex: `rm -rf`) • Confirmação humana para ações de alto risco
Monitoramento / Supervisão	Em tempo de execução contínuo	Acompanhar o comportamento geral do sistema.	• Detecção de anomalias operacionais • Revisão humana (*Human-in-the-loop*) em pontos críticos

Formas de Implementação

- **Bibliotecas especializadas:** Ferramentas dedicadas como *NVIDIA NeMo Guardrails* e *Guardrails AI*.
- **LLMs dedicados:** Modelos menores e mais rápidos atuando exclusivamente como validadores/moderadores.
- **Módulos lógicos e heurísticas:** Expressões regulares (Regex), validações determinísticas de esquema e regras de negócio no código do agente.

09- Principais Métricas de Monitoramento de Agentes de IA:

Categoria	Foco Principal	Métricas-Chave (O que medem)	Por que monitorar (Impacto)
1. Desempenho e Eficiência	Velocidade, fluidez e capacidade de processamento	• **Latência Total:** Tempo ponta a ponta (solicitação até a entrega final) • **Latência por Passo:** Tempo em cada etapa (LLM, tool ou código) • **Vazão (*Throughput*):** Volume de requisições atendidas por minuto	Evita espera excessiva para o usuário e identifica onde estão os gargalos do fluxo.
2. Custos Operacionais	Sustentabilidade financeira e controle de orçamento	• **Consumo de Tokens:** Volume de *prompt* (entrada) e *completion* (saída) • **Custo por Tarefa/Interação:** Gasto consolidado (LLM + APIs + infra) • **Custo de Ferramentas:** Valores cobrados por APIs externas de terceiros	Evita faturas surpresa, permite cobrar por cliente/tarefa e viabiliza o cálculo de ROI.
3. Qualidade e Precisão	Assertividade, corretude e utilidade das respostas	• **Taxa de Sucesso da Tarefa:** % de demandas concluídas com êxito • **Precisão (*Accuracy*):** Exatidão em dados objetivos e cálculos • **Taxa de Alucinação:** Frequência de dados inventados ou incorretos • **Relevância:** Pertinência da resposta em relação à pergunta • **Aterramento (*Grounding*):** Fidelidade aos documentos e ferramentas (RAG) • **Completude:** Se todos os pontos solicitados foram abordados	Mantém a credibilidade da IA e assegura que ela segue a base de conhecimento real.
4. Segurança e Confiabilidade	Resiliência técnica, conformidade e defesa do sistema	• **Taxas de Erro (Geral, LLM e Ferramentas):** Falhas, timeouts e erros de API • **Ativação de Guardrails:** Frequência de disparos das travas de segurança • **Conteúdo Indesejado:** Respostas	Protege contra invasões maliciosas, preserva a reputação da marca e reduz falhas técnicas.

		tóxicas, enviesadas ou inadequadas • **Detecção de Ataques:** Tentativas de *Prompt Injection* e *jailbreak* • **Adesão a Restrições:** Respeito a proibições explícitas (instruções negativas)	
5. Experiência do Usuário (UX)	Aderência, facilidade de uso e satisfação real	• **Satisfação Direta:** Notas via CSAT, NPS e *Thumbs Up/Down* • **Conclusão Percebida:** Se o usuário sentiu que seu problema foi resolvido • **Turnos Médios por Tarefa:** Quantas mensagens foram trocadas para resolver • **Sentimento do Feedback:** Tom (positivo/negativo) nos comentários do usuário • **Retenção / Engajamento:** Frequência de retorno dos usuários à solução	Mede o valor percebido na ponta; muitos turnos podem indicar que o agente é prolixo ou confuso.
6. Uso de Ferramentas (*Tools*)	Saúde e relevância do ecossistema de integrações	• **Frequência de Chamada:** Quantas vezes cada ferramenta é acionada • **Taxa de Sucesso por Tool:** % de execuções bem-sucedidas por ferramenta • **Latência por Tool:** Tempo médio de resposta de cada API externa	Identifica ferramentas que falham com frequência, que são lentas ou que nunca são usadas.

10 - Projeto 6 Multi-Agentes de IA, RAG, Roteamento, Guardrails, Observabilidade e Explicabilidade com LangGraph, LangSmith e Pydantic LogFire:

-> O Projeto 6 traz a construção de uma app inteligente composta por múltiplos Agentes de IA que colaboram para responder a consultas complexas mantendo o histórico de conversas no chat de atendimento a usuários:

- Enriquecendo as respostas com fragmentos de documentos via **RAG (Retrieval-Augmented Generation)** e definindo fluxos dinâmicos de **roteamento para direcionar** cada requisição ao agente mais adequado, incluindo busca no RAG e busca na web.
- Para garantir segurança e conformidade, são implementados **guardrails** que monitoram as chamadas e interferem em comportamentos fora de política.

- Enquanto mecanismos de **observabilidade** capturam métricas de desempenho, rastreamento de chamadas e logs estruturados. A **explicabilidade** é tratada por camadas que registram, em cada etapa do processo, as decisões tomadas pelos agentes e as fontes de informação utilizadas, permitindo auditoria e análise retrospectiva.
- O **LangGraph** orquestra a topologia e o estado dos agentes, **LangSmith** auxilia no versionamento e no monitoramento dos prompts e modelos, e o **Pydantic LogFire** fornece um esquema de validação e serialização robusto para os logs, garantindo qualidade e consistência dos dados coletados durante toda a operação. Criaremos ainda um Dashboard de observabilidade para monitoramento da app de Agentes de IA em tempo real.

21: FAISS

Quadro Comparativo: FAISS vs. Outras Alternativas no RAG

Ferramenta	Categoria	Vantagens	Quando escolher no lugar do FAISS?
FAISS	Biblioteca matemática de busca vetorial	Desempenho bruto absurdo, execução local ou GPU, sem overhead de servidor.	Quando o foco for velocidade pura e controle de baixo nível em grande volume de vetores.
ChromaDB	Banco vetorial embutido (*embedded*)	Muito fácil de integrar em Python, gerencia metadados e persistência de arquivos nativamente.	Projetos locais, protótipos, PoCs e aplicações com LangChain/LangGraph.
Pinecone / Qdrant / Weaviate	Bancos vetoriais gerenciados em nuvem	Escala distribuída, suporte a CRUD completo (inserir, atualizar, apagar documentos em tempo real).	Ambientes de produção corporativos que precisam de banco dedicado e nuvem gerenciada.
PostgreSQL (pgvector)	Banco relacional com suporte a vetores	Mantém dados normais (tabelas, clientes) e vetores no mesmo banco SQL.	Quando seu sistema já usa Postgres e você não quer adicionar outro banco na infraestrutura.

Busca Léxica (BM25 / Elastic)	Busca clássica por palavras-chave	Não precisa de embeddings; excelente para termos exatos, números de protocolo e siglas.	Em RAG Híbrido (combinando busca vetorial com busca exata de palavras-chave).

Mód 17 – Flexibilidade e observabilidade...

05 Cache Para o Carregamento do LLM:

-> O decorador pega uma função, "embrulha" ela com ações extras (antes ou depois) e devolve a função pronta para uso, sem que você precise mexer no código interno dela.

Onde os decoradores são muito usados?

1. **Segurança / Login:** Verificar se o usuário tem permissão antes de abrir uma página (@login_required).
2. **Medição de tempo:** Calcular quantos segundos uma função demora para rodar.
3. **Logs:** Registrar a data e hora em que uma ação foi executada.
4. **Cache:** Salvar o resultado de um cálculo pesado para não ter que recalcular na próxima vez.

LOGFIRE: Ele é genérico, não é obrigado a usar, porém o **Logfire** funciona como um "raio-X" da sua aplicação de IA, permitindo auditar, depurar e provar que componentes como o Cache do Retriever (**pesquisar e recuperar informações relevantes) estão** realmente funcionando e economizando tempo e dinheiro.

-> O LogFire calcula a duração total, coleta todos os eventos registrados e envia um pacote estruturado para o seu painel de observabilidade.
-->Isso facilita identificar quanto tempo cada etapa levou, quais logs ocorreram sob qual operação e como diferentes spans se encadeiam em fluxos distribuídos, dando visibilidade precisa sobre a performance e o comportamento do seu pipeline.

08 Função Para o Nó de Roteamento:

-> Um decorator do LangSmith é usado para instrumentar automaticamente chamadas a modelos de linguagem (LLMs) no seu código. Ao colocar:
@traceable(run_type="llm", name="Node_RouteQuery")

Na prática, isso facilita o monitoramento e a depuração dos seus agentes:

- Consegue visualizar no LangSmith como as queries estão sendo roteadas;
- Quanto tempo cada etapa leva;
- Quais inputs geraram quais outputs e muito mais, sem precisar instrumentar manualmente cada chamada.

Basta importar o decorator do pacote e aplicá-lo às funções críticas do seu grafo ou pipeline de agentes.

28:

O LangSmith é essencial para monitoramento, observabilidade, explicabilidade voltada para LLms.
Mas para monitorar app como um todo, pode usar a ferramenta logfire.

Mód 18 – Gerenciamento de memória e contexto e acesso a banco de dados:

02 Por que a memória é importante?

-> É um componente fundamental na arquitetura de Agentes de (IA), capacitando-os a transcender respostas meramente reativas e a exibir comportamentos mais sofisticados, adaptáveis e inteligentes.

Sem memória, um Agente de IA trataria cada interação como um evento isolado, incapaz de aprender com o passado, manter o contexto, personalizar interações ou tomar decisões informadas.

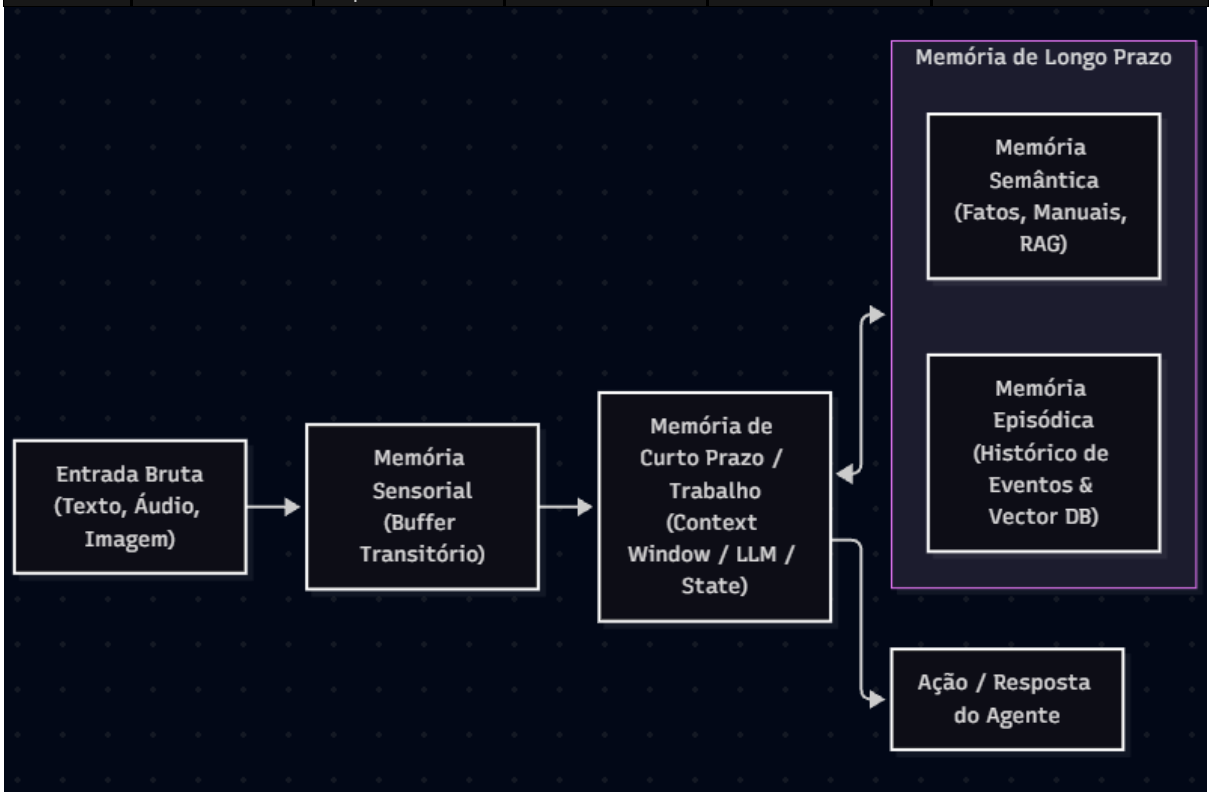
Pilar	Tipo de Memória	Função Central	Impacto / Benefício Prático
1. Contexto e Coerência	**Curto Prazo**	Acompanha o fluxo imediato da conversa e eventos recentes.	Garante respostas relevantes e fluidas, evitando diálogos desconexos e frustrantes.

2. Aprendizado e Adaptação	**Longo Prazo**	Registra experiências passadas (sucessos e erros) e conhecimentos adquiridos.	Permite refinar estratégias, otimizar desempenho e se adaptar a novas situações.
3. Personalização	**Histórico e Preferências**	Armazena o perfil, preferências e interações individuais do usuário.	Antecipa necessidades, gera recomendações precisas e cria uma relação natural e eficaz.
4. Planejamento e Decisão	**Base de Conhecimento**	Cruza dados históricos com projeções de consequências futuras.	Permite avaliar múltiplos caminhos e escolher a ação ideal para cumprir metas complexas.
5. Construção de Confiança	**Consistência Relacional**	Evidencia que o agente se lembra de acordos e preferências prévias.	Transmite competência, aumentando a segurança do usuário para delegar tarefas.

03 Diferenças Entre Memória de Curto e Longo Prazo:

Tipo de Memória	Subtipo / Conceito	Função no Agente de IA	Volatilidade & Persistência	Implementação Típica em IA	Exemplo Prático
Sensorial	*Buffer Transitório*	Reter dados brutos de entrada antes de qualquer inferência ou processamento.	**Extremamente volátil** (dura apenas frações de segundo / tempo de ingestão).	Buffers de I/O, streams de áudio/vídeo, fila de requisições HTTP.	Ingestão crua de um prompt de texto, frame de imagem da câmera ou áudio antes do tokenizador/transcrição.
Curto Prazo	*Memória de Trabalho (Working Memory)*	Reter o foco imediato, histórico recente da conversa e estado atual de execução em tarefas de múltiplos passos.	**Volátil e limitada** (perdida ao reiniciar a sessão ou atingir o limite de tokens).	**Context Window** do LLM, buffers de mensagens, estado da máquina (*LangGraph State / Scratchpad*).	O chatbot lembrar o que você disse duas perguntas atrás ou manter o objetivo intermediário de um plano.
Longo Prazo	**Semântica** *(Bases de Conhecimento)*	Armazenar fatos gerais, conceitos, políticas corporativas, ontologias e regras que não mudam a cada conversa.	**Persistente** (duradoura, consultada conforme a necessidade).	Bases documentais, Knowledge Graphs, bases RAG, bancos relacionais/NoSQL.	Agente consultar o manual interno de reembolso da empresa ou regras de garantia de produtos.

Longo Prazo	**Episódica** *(Histórico de Interações)*	Gravar experiências passadas, sequências de ações anteriores, resultados obtidos e preferências de usuários específicos.	**Persistente** (alimentada continuamente com o aprendizado/uso do sistema).	**Bancos de Dados Vetoriais** (*Vector DBs* com busca por similaridade semântica) e logs de histórico.	Agente lembrar que, há três semanas, o usuário teve problema de faturamento e preferiu contato por e-mail.



04 Estratégias de Preservação de Contexto em Interações Prolongadas:

Objetivo do Projeto	Estratégia Recomendada	Por quê?
Baixa latência e baixo custo	**Janela Deslizante (1)**	Sem overhead de banco de dados ou chamadas intermediárias de LLM.
Sessões longas no mesmo dia	**Sumarização (2)**	Compacta tokens antigos e preserva decisões centrais.
Lembrar preferências do usuário ao longo de meses	**Memória Longo Prazo (3) + RAG (4)**	Persiste fora da memória volátil e consulta apenas sob demanda.
Precisão cirúrgica em dados numéricos/cadastrais	**Rastreamento de Entidades (7)**	Evita que valores, datas e nomes sofram perda de resolução por resumo.

Estratégia	Como Funciona	Principais Vantagens
Janelas de Contexto *(Sliding Windows)*	Mantém apenas os últimos N tokens ou turnos recentes, descartando as mensagens mais antigas.	• Rápida e simples de implementar. • Baixo custo e previsibilidade de recursos.
Sumarização de Contexto	Um modelo gera resumos progressivos dos pontos-chave e substitui o histórico bruto por esses resumos.	• Altamente escalável para conversas longas. • Economiza espaço na janela de contexto.
Memórias Explícitas *(Curto e Longo Prazo)*	Separa a memória de trabalho (atual/imediata) de um banco persistente de fatos, preferências e histórico.	• Recorda interações de conversas passadas. • Permite personalização contínua.
RAG *(Recuperação de Informação)*	Armazena histórico e bases externas em banco de dados e busca apenas trechos relevantes para o prompt atual.	• Acesso a grandes volumes sem estourar a janela. • Reduz alucinações com base factual.
Mecanismos de Atenção	Camada nativa do Transformer que calcula pesos matemáticos para destacar segmentos relevantes da entrada.	• Conecta informações distantes no texto de forma dinâmica e automática.
Embeddings & Busca Semântica	Converte mensagens em vetores numéricos e recupera turnos passados por similaridade de significado (Vector DB).	• Recupera contexto por intenção e assunto, sem depender de palavras-chave exatas.
Rastreamento de Entidades e Tópicos	Identifica entidades (nomes, produtos, datas) com NER e mantém uma tabela de estados e atributos atualizada.	• Garante consistência máxima sobre dados específicos citados no diálogo.

11 - Compreendendo o Que São Sistemas CRM (Customer Relationship Management):

-> Um sistema CRM (Customer Relationship Management), ou Gestão de Relacionamento com o Cliente, é uma tecnologia (geralmente um software) que ajuda empresas a gerenciar e analisar todas as interações e dados de seus clientes ao longo de todo o ciclo de vida do cliente.

Seus principais objetivos são:

- **Melhorar o relacionamento com os clientes**, centralizando dados como histórico de compras, contatos e preferências para oferecer um atendimento mais personalizado e eficiente.
- **Aumentar as vendas e a retenção de clientes**, otimizando os processos de vendas, marketing e atendimento, identificando oportunidades, automatizando tarefas, aumentando a satisfação, fidelizando clientes e atraindo novos.

17 Estado do agente:

`class AgentState(TypedDict):`

- Cria a classe **AgentState**, que herda de **TypedDict**.
- **TypedDict** define um **dicionário com estrutura e tipos pré-definidos**, permitindo verificação estática de tipos.
- O objetivo é representar o **estado do agente** com um formato bem definido.

`messages: Annotated[List[BaseMessage], operator.add]`

- `messages` é um campo (chave) do `AgentState`.
- `List[BaseMessage]` indica que esse campo deve ser uma **lista de objetos `BaseMessage`** (ou subclasses), garantindo que apenas mensagens válidas sejam armazenadas.

`Annotated[...]`

- Permite adicionar **metadados** a um tipo sem alterar seu funcionamento.
- O **primeiro argumento** é o tipo real (`List[BaseMessage]`).
- Os **demais argumentos** são metadados usados por bibliotecas, como o **LangGraph**.

`operator.add`

- É o metadado associado ao campo `messages`.
- No **LangGraph**, informa como atualizar esse campo: **novas mensagens não substituem as antigas**, mas são **concatenadas** à lista existente, como em `lista1 + lista2`.

- Esse comportamento funciona como a **memória do agente**, preservando o histórico de mensagens ao longo da conversa.

19:

-> O professor não recomenda utilizar funções como: update, insert, drop e delete, usando modelos de IA.

22- Cuidados ao Usar Agentes de IA Para Acessar Bancos de Dados Corporativos:

-> Ao usar **Agentes de IA para acessar bancos de dados corporativos**, é fundamental priorizar **segurança, governança de dados e controle de acesso**.

Autenticação e autorização granular:	o agente deve acessar apenas o necessário (**princípio do menor privilégio**), usando autenticação forte (OAuth2, tokens temporários ou chaves rotativas).
Validação e sanitização de entradas:	validar prompts para evitar **injeção de SQL** e comandos maliciosos.
Auditoria e rastreamento:	registrar consultas, dados acessados e tempo de resposta para **compliance**, rastreabilidade e investigação de incidentes.
Segurança acima do desempenho:	a aplicação pode ficar mais complexa e um pouco mais lenta, mas a proteção dos dados é prioridade.
Proteção contra Prompt Injection e Jailbreak:	usar filtros e mecanismos para impedir que o agente execute comandos não autorizados.
Proteção de dados sensíveis:	evitar exposição de informações confidenciais, aplicando **mascaramento, agregação e filtros** antes de retornar dados.

Limitação de operações de escrita:	preferir **acesso somente leitura**; quando houver escrita, exigir validações extras e, de preferência, aprovação humana ou de outro sistema.
Contexto e versionamento do esquema:	fornecer ao agente uma visão **controlada e atualizada** da estrutura do banco (ex.: LangChain ou Semantic Layer), evitando uso de esquemas desatualizados.
Testes em ambiente seguro:	validar o comportamento do agente em ambientes isolados antes de acessar bancos de produção.
Observabilidade e alertas:	monitorar continuamente as ações do agente e gerar alertas para acessos suspeitos ou possíveis vazamentos.
Conformidade legal e ética:	garantir que o uso da IA esteja de acordo com normas como **LGPD** e **GDPR**, com decisões automatizadas **auditáveis e explicáveis**.

Mód 19 – Parte II: Gerenciamento de memória e contexto e acesso a banco de dados:

06 - Construção do No da junção e roteamento:

Característica	Hub (Nó de Junção)	Roteador (Router)
Papel Principal	Ponto de encontro / convergência estrutural.	Lógica de decisão e direcionamento.
No Grafo	Representado por um Nó (`workflow.add_node("router", ...)`).	Representado por uma Aresta Condicional (`add_conditional_edges`).
Manipulação de Estado	Neutro (geralmente não altera dados).	Avalia o estado para retornar o nome do próximo nó.

Responsabilidade	Centralizar as conexões e unificar os caminhos.	Classificar intenções, checar ferramentas ou alternar turnos.

-> Resumo do fluxo:

1. O fluxo chega no nó "router" (dsa_route_junction_node).
2. O nó não mexe no estado (return {}).
3. O LangGraph chama a aresta condicional dsa_router_logic(state).
4. A função inspeciona state['messages'] e decide uma string (por exemplo, "openai_agent").
5. O LangGraph encaminha a execução imediatamente para o nó do agente OpenAI.

2. O Hub vs. O Roteador: como cada um lida com o `state`

Aqui está a diferença crucial entre o nó Hub e a função do Roteador:

A) O Hub apenas recebe o estado e não altera nada:

python

```python
def dsa_route_junction_node(state: AgentState) -> dict:
    print("--- Nó de Junção de Roteamento (Sem Mudança de Estado) ---")
    return {}  # <-- Retorna vazio!
```

- **Por que retorna `{}`?** No LangGraph, o que uma função de nó retorna serve para **atualizar** o estado. Retornar `{}` significa: *"Eu sou apenas um nó de passagem estrutural, não adicionei nem modifiquei nenhuma mensagem na prancheta".*

B) O Roteador é onde o `state` é inspecionado:

Veja onde o estado aparece na função `dsa_router_logic`:

python

```python
def dsa_router_logic(state: AgentState) -> str:
    # 1. AQUI ELE ACESSA O ESTADO:
    messages = state['messages']  # Pega a lista de mensagens da prancheta

    # 2. AQUI ELE ANALISA O QUE ESTÁ DENTRO DO ESTADO:
    last_message = messages[-1] if messages else None

    # Se a última mensagem pediu para rodar ferramenta SQL:
    if isinstance(last_message, AIMessage) and last_message.tool_calls:
        return "tools"  # <- Retorna o NOME do próximo nó

    # Se o usuário pediu explicitamente um agente:
    if isinstance(last_message, HumanMessage):
        if "@openai" in last_message.content.lower():
            return "openai_agent"
        elif "@groq" in last_message.content.lower():
            return "groq_agent"
```

--> O estado é uma estrutura de dados.

Ele está usando 2 modelos OpenIA e Groq, e pergunta se o modelo precisa visitar o banco de dados e a resposta seria não, pois OpenIA ja foi, já executou a query e gravou o estado, então está apenas roteando para Groq. Então, o segundo agente não precisa ir para o banco de dados de novo.

09 - Função Para a Lógica de Roteamento:

A **transição de estado** é o mecanismo que define e direciona o fluxo de execução do agente, guiando a tomada de decisão a cada etapa:

- **O que é:** A mudança entre diferentes fases ou comportamentos do agente durante sua operação.
- **Como funciona (Modelagem FSM):** É estruturada por **Máquinas de Estados Finitos**, onde cada estado representa uma condição específica e as transições são disparadas por gatilhos (ações do usuário ou respostas do sistema).
    - *Exemplo:* Aguardando Solicitação ➔ Processando Pedido ➔ Encerrando Conversa.
- **Ferramental (ex.: LangGraph):** Cria fluxos orientados a estados, permitindo responder dinamicamente às entradas e **preservar o contexto** ao longo de múltiplas interações.
- **Objetivo na lógica de roteamento:** Garantir que o agente tome caminhos coerentes e precisos diante de **decisões condicionais** e **tarefas complexas de múltiplas etapas**.

15 - Gerenciamento da Memória e Inicialização do Estado:

* O st.session_state é usado no Streamlit para armazenar e manter dados do usuário durante a sessão.

* Isso é necessário porque o Streamlit executa o script inteiro novamente a cada interação (cliques, preenchimento de campos etc.), fazendo com que variáveis comuns sejam reinicializadas.

* O st.session_state funciona como um dicionário persistente, preservando os valores entre as execuções do script.

* Cada aba do navegador cria uma sessão independente, com seu próprio session_state, evitando que as informações de um usuário interfiram nas de outro.

* Exemplo: um contador pode ter seu valor salvo no st.session_state, mantendo a contagem mesmo após várias interações.

21 - UX e os Benefícios do Gerenciamento Eficiente de Memória

-> O gerenciamento eficiente de memória melhora a experiência do usuário (UX), principalmente em aplicações interativas, como apps web, desktop e Agentes de IA.

Principais benefícios:

1. Resposta mais rápida: evita leituras e gravações desnecessárias no disco, tornando a aplicação mais rápida, fluida e responsiva.

2. Evita travamentos: reduz problemas como vazamento e consumo excessivo de memória, prevenindo lentidão, travamentos e abandono da aplicação.

3. Preserva dados do usuário: mantém sessões, preferências e interações recentes, proporcionando uma experiência mais personalizada e contínua.

4. Menor consumo de recursos: utiliza menos CPU, RAM e bateria, beneficiando dispositivos móveis e computadores mais simples.

5. Melhor escalabilidade: permite que a aplicação suporte mais usuários simultaneamente sem perda significativa de desempenho.

6. Desempenho constante: libera memória de objetos e arquivos que não são mais usados, mantendo a aplicação rápida mesmo após longos períodos de uso.

Conclusão:

-> Uma boa UX depende não apenas do design, mas também de um gerenciamento eficiente de memória, que torna a aplicação mais rápida, estável, confiável e agradável de usar.

Mód 20 Orquestração de agentes de IA para Deep Reserarch:

02- Deep Research na IA Generativa:

-> Deep Research (Pesquisa Profunda) é a capacidade de uma IA Generativa realizar uma investigação detalhada e complexa antes de gerar uma resposta. Não é um termo técnico oficial como RAG ou Fine-Tuning, mas descreve um processo mais avançado de pesquisa.

Principais características:

* Consulta múltiplas fontes: pesquisa em artigos científicos, bancos de dados, notícias, documentos internos, páginas da web etc.

* Recuperação avançada de informações: usa técnicas sofisticadas (geralmente baseadas em RAG) para encontrar conteúdos altamente relevantes.

* Pesquisa em várias etapas: divide perguntas complexas em subperguntas, faz buscas iterativas, analisa resultados, identifica lacunas e busca novas informações.

* Síntese e raciocínio: compara, relaciona e combina informações de diferentes fontes para formar uma resposta completa e coerente.

* Contextualização: utiliza as informações encontradas para gerar respostas mais profundas, precisas e bem fundamentadas.

* Confiabilidade: prioriza fontes confiáveis para reduzir erros e alucinações da IA.

Diferença para o RAG:

* RAG simples: recupera alguns trechos relevantes para responder à pergunta.

* Deep Research: realiza uma pesquisa completa, iterativa e semelhante ao trabalho de um pesquisador humano, analisando diversas fontes antes de gerar a resposta.

03-Quando usar Deep Research?

-> O Deep Research deve ser usado quando uma resposta simples ou apenas o conhecimento pré-treinado da IA não são suficientes. Ele é indicado para tarefas que exigem:

* Perguntas complexas e multifacetadas: quando é necessário analisar diferentes perspectivas, fatores e informações de várias áreas.

Exemplo: impacto das mudanças climáticas na agricultura brasileira considerando aspectos econômicos, sociais e tecnológicos.

* Alta precisão e confiabilidade: em situações críticas onde erros podem causar consequências importantes.

Exemplo: decisões médicas, análises financeiras ou pesquisas jurídicas.

* Informações específicas ou de nicho: quando o assunto é muito especializado, recente ou pouco presente nos dados gerais da IA.

Exemplo: novas tecnologias, doenças raras ou processos internos de empresas.

* Informações recentes ou dinâmicas: quando é necessário usar dados atualizados, eventos recentes ou tendências atuais.

Exemplo: mercado, notícias ou mudanças recentes em uma área.

* Produção de conteúdos aprofundados: para criar relatórios, artigos, análises ou documentos baseados em pesquisas e fontes confiáveis.

* Fact-checking (verificação de fatos): para confirmar informações comparando diferentes fontes externas confiáveis.

* Redução de limitações e vieses da IA: para complementar o conhecimento do modelo com informações externas mais diversas e específicas.

—> Resumo: O Deep Research é usado quando a tarefa exige mais profundidade, precisão, atualidade, especificidade ou complexidade do que uma resposta comum da IA ou uma busca simples/RAG conseguem oferecer. Ele amplia a capacidade da IA por meio de pesquisa e análise de informações externas confiáveis.

04:

-> Handoffs permitem delegar subtarefas a agentes especializados, criando pipelines que se sucedem até a conclusão de cada etapa sem necessidade de código adicional de controle.

-> Guardrails são camadas de segurança que verificam condições pré-definidas, interrompendo o fluxo caso ocorra alguma violação e reforçando a confiabilidade da aplicação.

-> O tracing integrado registra cada iteração do loop de execução, fornecendo logs estruturados e visualizações para monitoramento e otimização contínua dos agentes.

05- Pesquisa Web com Firecrawl:

-> Firecrawl é uma plataforma open-source que transforma qualquer site em dados LLM-ready, oferecendo recursos de scrape, crawl, map, extract e search. O endpoint /search foi projetado para realizar buscas na web e opcionalmente raspar o conteúdo dos resultados, simplificando pipelines que antes exigiam etapas distintas debusca e scraping (raspando).

Como Funciona:

- Você chama app.search(query, limit, scrape_options) no SDK de sua escolha.

- Firecrawl executa a busca na web, recuperando o conteúdo correspondente.

- Se solicitado, o serviço carrega cada link e extrai o conteúdo principal, convertendo-o em markdown ou outro formato definido.

- A operação é gerenciada de forma assíncrona, possibilitando call backs de atividade para feedback em tempo real.

14 - O uso do Async:

-> O async na declaração async def serve para definir uma função assíncrona. Isso significa que essa função pode usar await para pausar sua execução até que operações assíncronas sejam concluídas, sem bloquear a execução do restante do programa.

-> A função deep_research é assíncrona (async) porque faz chamadas a serviços externos, como buscas na web, scraping e acesso a banco de dados. Isso permite:

* Executar várias requisições em paralelo.

* Evitar bloqueios enquanto aguarda respostas.

* Controlar melhor o tempo de execução (útil com time_limit).

-> Ela retorna um Dict[str, Any], mas pode aguardar outras funções assíncronas antes de retornar, mantendo a aplicação responsiva e eficiente.

-> A função async def run_research_process(topic: str) coordena todo o fluxo da pesquisa utilizando await.

Mód 21 - Como melhorar as respostas de LLMs e Agentes de IA através de integrações com diferentes ferramentas e fontes de dados, através de MCP – Model Context Protocol:

02 MCP:

-> É um **protocolo de comunicação**, ou seja, um padrão que permite que aplicações de IA se conectem a ferramentas e fontes de dados de forma padronizada. Ele **não é uma técnica de IA.**

- **Primitivas Chave (Capacidades Expostas pelos Servidores)**
  -> Os Servidores MCP expõem suas funcionalidades através de três primitivas principais:
  - Ferramentas (Tools).
  - Recursos (Resources): Representam dados contextuais que são anexados e gerenciados pela aplicação cliente (Host).
  - Prompts: São instruções pré-definidas.

- **Componentes do MCP**

1. Host MCP

- É a aplicação onde o usuário interage com a IA (como ChatGPT, um IDE ou um sistema próprio).
- Atua como o coordenador central da comunicação.
- Cria e gerencia os Clientes MCP.
- Define quais Servidores MCP os clientes podem acessar.
- Aplica políticas de segurança, permissões e privacidade.

2. Cliente MCP

- **Opera sob o controle do Host**.
- Cada Cliente MCP estabelece uma conexão **um-para-um** com um Servidor MCP específico.
- Traduz as solicitações do Host para o protocolo MCP.
- Envia as mensagens ao Servidor MCP e devolve as respostas ao Host.

3. Servidor MCP

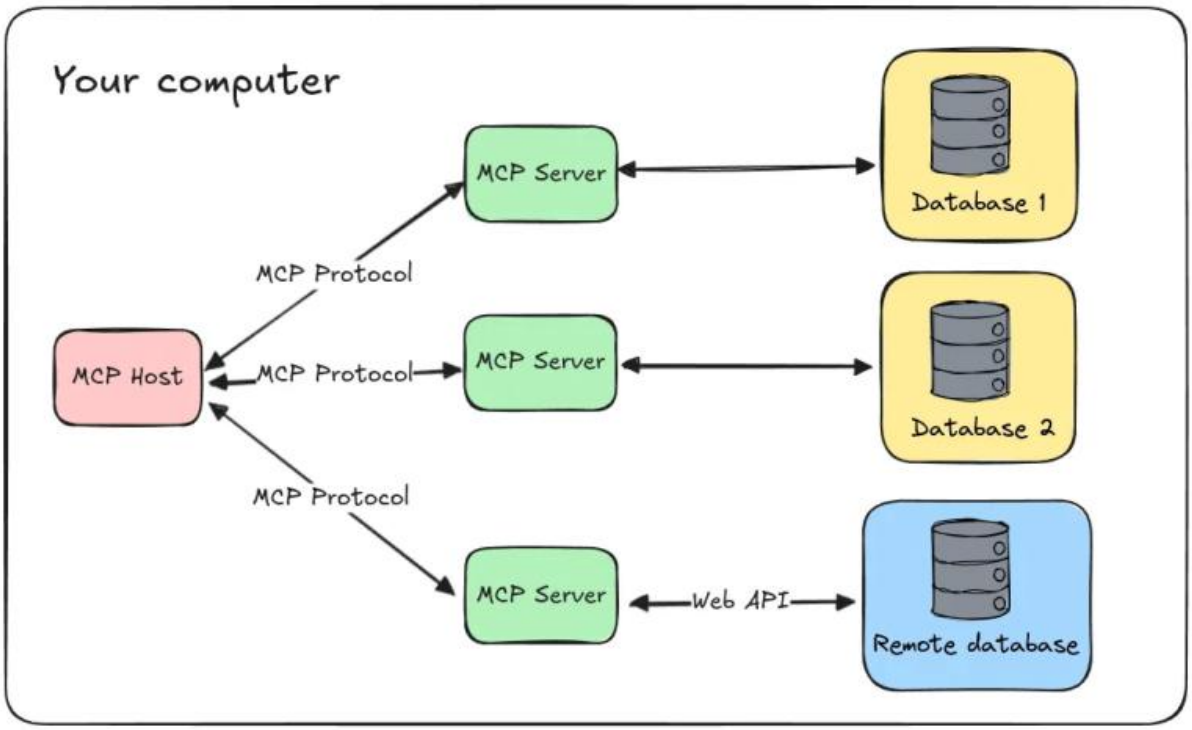
- Conecta-se a uma ferramenta ou fonte de dados específica.
- Expõe suas capacidades ao agente por meio do protocolo MCP.
- Essas capacidades podem incluir:

    - **Tools (Ferramentas):** executar ações, como pesquisar arquivos, enviar mensagens ou consultar um banco de dados.
    - **Resources (Recursos):** disponibilizar dados ou informações para o modelo.
    - **Prompts:** instruções pré-definidas que podem orientar o comportamento do modelo em determinadas tarefas.
- Exemplos de servidores: Google Drive, GitHub, Slack, bancos de dados, APIs e sistemas internos.

Fluxo de funcionamento

Usuário → Host → Cliente MCP → Servidor MCP → Ferramenta/Dados → Servidor → Cliente → Host → Usuário

- **MCP = protocolo de comunicação.**
- **Host = coordena a comunicação e controla segurança e permissões.**
- **Cliente = faz a ponte entre o Host e um Servidor MCP específico.**
- **Servidor = conecta a IA às ferramentas e aos dados, expondo suas capacidades.**



Componente	Função Principal	Interage Com	Exemplos
Host MCP	Coordena instâncias de Cliente, gerencia ciclo de vida, políticas de segurança e permissões, integra com o LLM.	Clientes MCP, Usuário, LLM	Chatbots (Claude Desktop), IDEs (Cursor, Zed), Aplicações de IA customizadas
Cliente MCP	Executa no Host, mantém conexão 1:1 com Servidor, negocia capacidades, orquestra mensagens.	Host MCP, Servidor(es) MCP	Conectores dentro do Host para cada serviço externo
Servidor MCP	Expõe capacidades (Ferramentas, Recursos, Prompts) de uma fonte de dados ou ferramenta externa específica.	Cliente(s) MCP, Sistema Externo	Servidores para Slack, GitHub, Google Drive, bancos de dados, APIs internas

<u>03 PydanticAI:</u>

-> LLM gera resposta, Pydantic valida, e o PydanticAI junta os dois para você criar agentes de forma mais segura.

Use PydanticAI em projetos de IA que precisam de:

- **Saídas estruturadas e validadas:** garante que as respostas do modelo sigam um formato esperado, evitando inconsistências em produção.
- **Tipagem estática:** reduz erros na integração entre agentes, funções e ferramentas.
- **Fluxos complexos com múltiplas etapas ou agentes:** permite que cada etapa trabalhe com dados tipados e organizados.
- **Integração com diferentes LLMs:** possibilita trocar modelos sem precisar reescrever a lógica de validação.

- **Monitoramento e testes facilitados:** usa injeção de dependências e integração com **Logfire** para acompanhar e testar agentes.
- **RAG, agentes autônomos e pipelines de dados:** ajuda a validar os esquemas dos dados em sistemas que usam recuperação de informações e automações.

04 – LangGraph x PydanticAI:

LangGraph x PydanticAI

Ambas são bibliotecas Python usadas para construir agentes de IA, mas possuem focos diferentes:

- LangGraph: organiza e orquestra agentes por meio de grafos, permitindo fluxos complexos, múltiplas etapas, ciclos, condições e gerenciamento de estado compartilhado.
- PydanticAI: utiliza o Pydantic para garantir tipagem, estruturação e validação dos dados de entrada e saída dos LLMs e ferramentas, aumentando a confiabilidade.

Semelhanças

- Ambas são bibliotecas Python.
- Facilitam a construção de agentes de IA com LLMs.
- Permitem sistemas modulares, combinando modelos, ferramentas e lógicas personalizadas.
- Integram-se a diferentes provedores de LLMs.
- São projetos de código aberto.

Quando escolher?

- LangGraph: quando o projeto exige fluxos complexos, múltiplas etapas, ciclos, condições e gerenciamento de estado. Exemplo: agente de pesquisa que refina buscas com base nos resultados anteriores.

- PydanticAI: quando é necessário garantir que os dados de entrada e saída dos LLMs e ferramentas sejam estruturados, tipados e validados. Exemplo: extrair informações de textos e garantir que sigam um esquema definido.

Integração entre os dois

Não são ferramentas concorrentes ou mutuamente exclusivas. Podem ser utilizadas juntas:

- LangGraph: coordena o fluxo de trabalho e as etapas do agente.
- PydanticAI: dentro de um nó do LangGraph, valida e estrutura os dados enviados e recebidos pelo LLM.

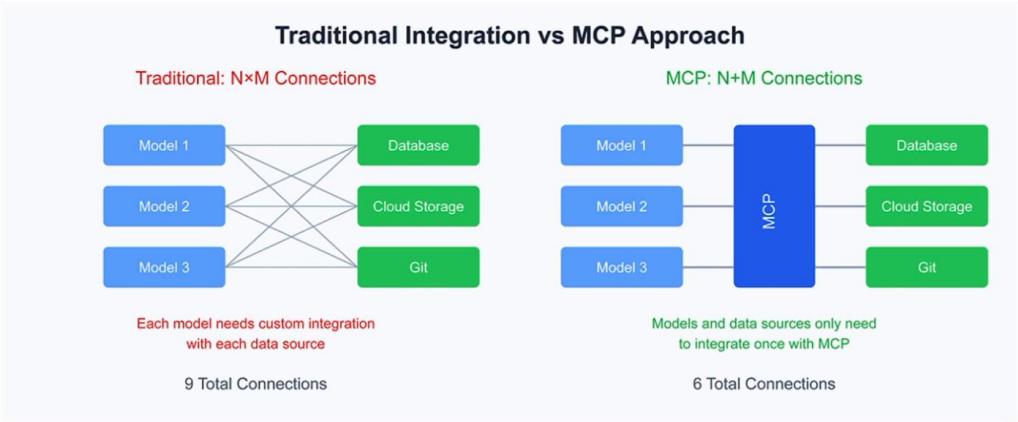
Resumindo para memorizar:

- LangGraph = orquestração, fluxo de trabalho e gerenciamento de estado.
- PydanticAI = tipagem, estruturação e validação de dados.

Os dois podem trabalhar juntos para criar agentes mais organizados, complexos e confiáveis.

07 Por que usar MCP?

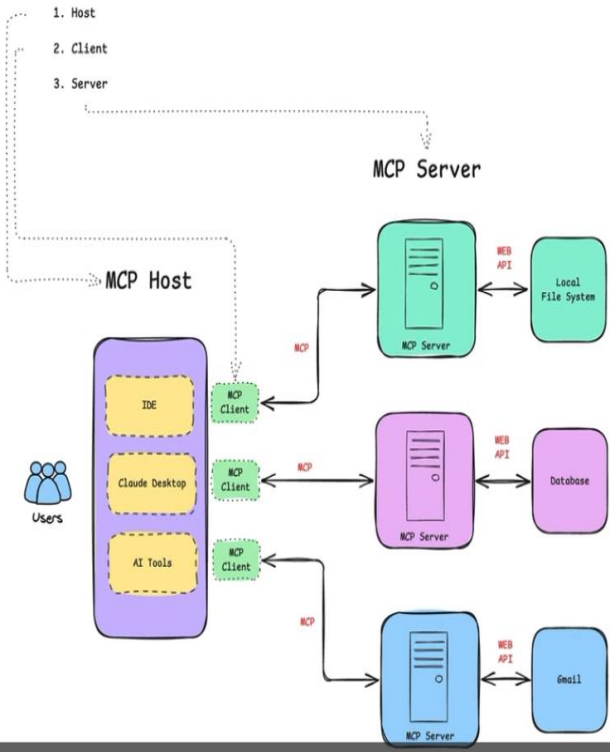
Por Que Usar MCP? Vamos Responder Através de Diagramas



Com MCP reduzimos consideravelmente o número de conexões, principalmente ao usar múltiplos LLMs (algo cada vez mais comum).

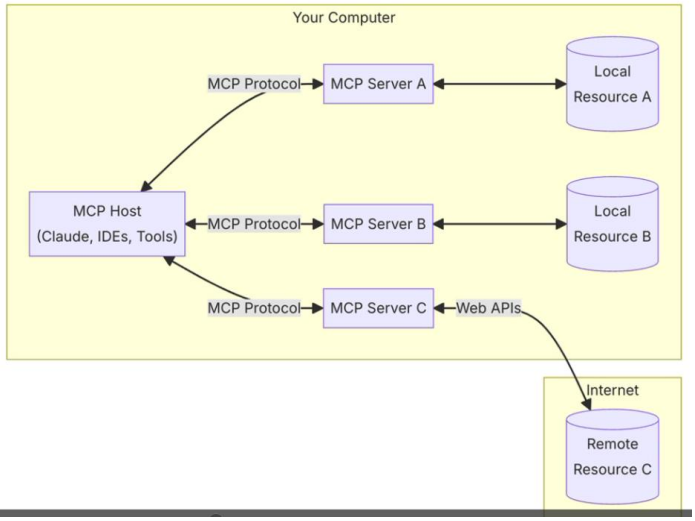
08:

Cada Servidor MCP é de fato uma **aplicação**, que recebe uma conexão a partir de um cliente (LLM, ferramenta, etc...) que reside em um host.



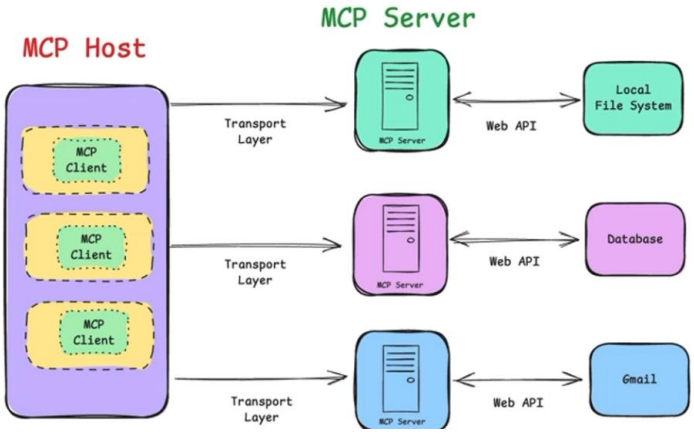
O cliente (que fica no host) não terá acesso direto ao recurso (busca na web, banco de dados, acesso a aplicações, etc...).

Todas as requisições externas ao LLM passam pelo Servidor MCP.



Cada Servidor MCP é de fato uma aplicação e a comunicação entre cliente (host) e servidor requer uma camada de transporte.

STDIO, Streamable HTTP e SSE são os mais comuns, como veremos no Estudo de Caso.



13 - Camadas de Transporte no MCP:

-> Essas camadas são responsáveis por garantir a comunicação eficiente entre Agentes de IA (clientes) e ferramentas externas (servidores), utilizando o formato de mensagens JSON-RPC 2.0. Essas camadas convertem mensagens do MCP para o formato JSON-RPC para transmissão e vice-versa, facilitando a interoperabilidade entre diferentes sistemas .

- Existem dois métodos principais de transporte no MCP:
  - ★ Stdio (entrada/saída padrão): Ideal para integrações locais, como acesso a sistemas de arquivos ou bancos de dados locais. É leve e adequado para comunicação síncrona.
  - ★ HTTP: Utilizado para conexões remotas, permitindo que Agentes de IA interajam com APIs e serviços externos pela web.

-> No STDIO, não tem HTTP, a comunicação acontece direto entre processos no mesmo computador. Já no Streamable HTTP, usa HTTP pela rede, com suporte a streaming, tipo enviando pedacinhos da resposta conforme fica pronta. O SSE também usa HTTP, mas é mais focado em mandar eventos do servidor para o cliente de forma contínua. Então, sim, dois são HTTP e um não. A diferença está na camada de transporte.

-> SSE é uma tecnologia de comunicação unidirecional baseada em HTTP, padronizada pelo HTML5, que permite ao servidor enviar dados em tempo real para o cliente de forma contínua, mantendo uma única conexão aberta.

- Pode ser usado como um transporte quando o agente precisa apenas "ouvir" eventos ou respostas de ferramentas longas ou assíncronas.

--> Características Principais;

- Unidirecional: apenas o servidor envia dados ao cliente. O cliente não envia dados por essa conexão, para isso, deve abrir requisições HTTP separadas.
- Conexão persistente: mantém uma conexão HTTP aberta por tempo indefinido, enviando dados conforme necessário.

- Reconexão automática: se a conexão for perdida, o cliente tenta reconectar automaticamente.
- Formato simples: baseado em texto, fácil de debugar e implementar.

--> Limitações:

- Apenas servidor → cliente: sem comunicação bidirecional.
- Sem suporte nativo a envio de binários (tudo é texto).
- Menor suporte em algumas linguagens/frameworks no backend, em comparação a WebSocket ou HTTP stream.

CURSO 03 – RAG, GraphRag e AgenticRag:

Mód 1 - RAG, Agentic RAG e GraphRAG:

1. RAG (Retrieval-Augmented Generation)

-> RAG é uma técnica usada em sistemas de IA para melhorar as respostas do modelo usando uma base de conhecimento externa.

Limitação do RAG tradicional:

* Ele é muito bom para encontrar informações parecidas.

* Porém, pode ter dificuldade quando a resposta depende de várias relações entre informações diferentes.

2. Agentic RAG (Agente RAG)

-> Agentic RAG é uma combinação de agente de IA + RAG.

A diferença é que ele possui uma camada de decisão.

- Um RAG tradicional faz:

Pergunta → Busca → Resposta

- Um Agentic RAG faz:

Pergunta → Analisa a tarefa → Decide o que fazer → Usa ferramentas (como RAG) → Refina → Responde

❑ Todo Agentic RAG é um agente, mas nem todo agente é Agentic RAG.

3. GraphRAG

-> GraphRAG também é uma técnica baseada em RAG, mas adiciona um grafo de conhecimento.

A ideia é que, além de guardar informações em vetores, o sistema cria relações entre entidades.

Exemplo: Nós (entidades):

* João * Empresa X * Maria * São Paulo

Arestas (relações): * João → trabalha na → Empresa X

* Empresa X → foi fundada por → Maria

* Maria → mora em → São Paulo

O grafo representa como as informações estão conectadas.

★ Rag x GraphRAG:

Busca principalmente por:

✅ Similaridade semântica

GraphRAG busca por:

✅ Similaridade semântica + ✅ Relações entre entidades

OBS: LangGraph é diferente de GraphRag, LG é um framework que cria fluxo de execução dos agentes, já o GR é uma técnica de recuperação de conhecimento.

Grafo de execução e o outro grafo de conhecimento (GraphRag):

LangGraph: grafo = “quais passos o agente vai executar?”

* GraphRAG: grafo = “como as informações estão relacionadas?”

-> Os dois usam nós e arestas, mas com objetivos totalmente diferentes.

Mód 02 - 08 - Ambientes virtuais:

-> O conda env é um recurso do Conda (gerenciador de pacotes e ambientes) que permite

criar ambientes isolados com versões específicas de Python, bibliotecas e dependências.

-> O conda env é um comando que gerencia ambientes virtuais no Conda, garantindo que diferentes projetos possam usar configurações e versões de pacotes independentes uns dos outros.

Benefícios:

- Isolamento total: evita conflitos de dependências entre projetos.

- Reprodutibilidade: facilita replicar ambientes em outras máquinas.

- Compatibilidade: permite testar códigos com diferentes versões dos pacotes.

- Gerenciamento centralizado: integra pacotes Python, R, C/C++, etc., em um só ambiente.

-> Isso ajuda no isolamento para uma versão em Python não sobrescrever a outra versão no meu pc.

Mód 03 - aula 02 Modelos de Linguagem, Deep Learning e Arquitetura de Transformadores

-> Redes neurais profundas são redes com várias camadas de processamento. Cada camada vai pegando um padrão mais abstrato do que a anterior.

Exemplo simples:

* Deep Learning = o método de aprendizado.

* Transformer = a estrutura da rede que aprende.

* Modelo de Linguagem = a IA pronta que conversa e gera textos.

1. Deep Learning (Aprendizado Profundo)

- É uma técnica dentro da Inteligência Artificial e do Machine Learning.
- Usa redes neurais profundas, que são redes com várias camadas de processamento.
- Essas camadas aprendem padrões a partir de grandes quantidades de dados =
  - Cada camada consegue identificar características mais complexas:
  - Camadas iniciais → padrões simples.
  - Camadas mais profundas → padrões mais abstratos e complexos.

* Exemplo: em imagens, pode aprender linhas → formas → objetos; em textos, pode aprender palavras → relações → significado.

2. Arquitetura de Transformadores (Transformers)

- É um tipo específico de arquitetura de rede neural dentro do Deep Learning.
- Foi criada para lidar muito bem com dados em sequência, principalmente textos.
- Seu principal diferencial é o mecanismo de atenção (Attention):
- Ele consegue analisar várias palavras ao mesmo tempo.
- Identifica quais palavras são mais importantes para entender o contexto.
- Essa arquitetura é a base de muitos modelos modernos de IA generativa.

3. Modelos de Linguagem (Language Models)

- São modelos de IA treinados para entender e gerar linguagem humana.
- Eles aprendem padrões de textos durante o treinamento e conseguem:
- completar frases;

- responder perguntas;
- resumir textos;
- criar conteúdos.
- Muitos modelos de linguagem atuais usam a arquitetura de Transformers.

Como tudo se conecta:

- Deep Learning

⬇️ (técnica geral usando redes neurais profundas)

- Arquitetura de Transformadores

⬇️ (um tipo específico de rede neural dentro do deep learning)

- Modelos de Linguagem

⬇️ (modelos construídos usando essas técnicas para trabalhar com texto)

<u>03 SLMs e LLMs:</u>

- LLM (Large Language Model)

* Modelo de linguagem grande.

* Treinado com muitos dados e possui muitos parâmetros.

* Tem maior capacidade de entender contexto e realizar tarefas complexas (gerar textos, resumir, traduzir, planejar).

- SML (Small Language Model)

* Modelo de linguagem pequeno.

* Possui menos parâmetros e é mais leve.

* Consome menos recursos e pode rodar em dispositivos mais simples.

* Faz tarefas de linguagem, mas geralmente com menor capacidade em situações complexas.

Resumo em uma frase:

➡️ LLM é um modelo de linguagem grande, mais poderoso e complexo; SML é um modelo menor, mais leve e eficiente, porém com menos capacidade.

CAG (Cache-Augmented Generation / Geração Aumentada por Cache)

15 Cache-Augmented Generation (CAG):

_. O CAG é uma abordagem para otimizar LLMs (modelos de linguagem grandes) pré-carregando informações importantes diretamente no contexto do modelo, evitando a necessidade de buscar dados durante a resposta.

Diferente do RAG, que faz uma busca em fontes externas em tempo real durante a geração da resposta, o CAG já deixa as informações necessárias disponíveis para o modelo usar, aproveitando as janelas de contexto maiores dos LLMs modernos.

- Como funciona:

1. As informações relevantes são carregadas previamente no contexto do modelo.

2. O usuário faz uma pergunta.

3. O LLM responde usando esse conhecimento já disponível, sem precisar recuperar dados externamente.

- Vantagens do CAG:

* Menor latência: como não precisa fazer uma busca antes de responder, a resposta é mais rápida.

* Maior confiabilidade: usa informações previamente selecionadas e verificadas, reduzindo erros de recuperação.

* Arquitetura mais simples: elimina a necessidade de sistemas complexos de busca e recuperação.

- Limitações do CAG:

* Tamanho do conhecimento: todo o conhecimento necessário precisa caber na janela de contexto do modelo, o que pode ser difícil em bases muito grandes.

* Limite de contexto: textos muito extensos podem prejudicar o desempenho do LLM, apesar dos avanços nas janelas de contexto.

16 KAG (Knowledge-Augmented Generation / Geração Aumentada por Conhecimento):

-> O KAG é uma abordagem que combina LLMs (modelos de linguagem grandes) com grafos de conhecimento para melhorar respostas que exigem raciocínio lógico e conhecimento específico de um domínio.

Diferente do RAG, que busca informações em documentos e textos, o KAG utiliza informações estruturadas em grafos de conhecimento,

mostrando relações entre conceitos e permitindo que o modelo compreenda melhor as conexões entre eles.

- Como funciona:

* O conhecimento é organizado em um grafo, onde existem:

* Entidades/conceitos (ex.: vinho, carne, tanino);

* Relações entre eles (ex.: “Cabernet possui taninos altos” e “taninos combinam com carnes gordurosas”).

* O LLM usa essas conexões para realizar inferências e gerar respostas mais precisas.

- Vantagens do KAG:

* Raciocínio lógico aprimorado: os grafos permitem que o modelo faça conexões e inferências mais complexas.

* Respostas mais precisas: como o conhecimento está estruturado, reduz ambiguidades e melhora a relevância das respostas.

* Uso em áreas especializadas: é útil em domínios que exigem alta precisão, como medicina, direito e finanças.

- Desafios do KAG:

* Maior complexidade de implementação: criar, integrar e manter grafos de conhecimento exige mais engenharia.

* Dependência de dados estruturados: a qualidade das respostas depende da existência e da qualidade do grafo de conhecimento.

17 GraphRag x KAG:

- GraphRAG (Graph Retrieval-Augmented Generation)

-> O GraphRAG é uma evolução do RAG tradicional que adiciona grafos de conhecimento ao processo de recuperação de informações.

* Constrói o próprio grafo a partir de textos.

* Extrai entidades e relacionamentos de um conjunto de documentos.

* É útil para analisar dados complexos e encontrar temas e conceitos dentro de dados privados ou específicos.

Fluxo:

📄 Texto bruto → 🕸️ Criação do grafo → 🔎 Recuperação de informações → 🤖 Resposta da LLM

- KAG (Knowledge-Augmented Generation)

* Utiliza grafos de conhecimento já existentes.

* Esses grafos normalmente são especializados em algum domínio específico.

* Fornece um contexto estruturado para a LLM, permitindo respostas mais precisas e inferências mais complexas.

Fluxo:

Grafo de conhecimento pronto → Recuperação do conhecimento → Resposta da LLM.

- Diferença entre GraphRAG e KAG

Apesar de ambos utilizarem grafos de conhecimento para melhorar respostas, a principal diferença está na origem desse conhecimento.

19 SELF RECLETIVE RAG:

- Self-RAG (Self-Reflective Retrieval-Augmented Generation)

-> O Self-RAG (Geração Aumentada por Recuperação Autorreflexiva) é uma evolução do RAG tradicional que combina LLMs, recuperação de informações e autorreflexão.

-> A principal diferença é que o RAG tradicional sempre recupera uma quantidade fixa de informações antes de responder, enquanto o Self-RAG decide de forma adaptativa quando precisa buscar informações, quantas vezes deve buscar e se a busca é realmente necessária. Além disso, ele avalia a própria resposta antes de finalizá-la.

- Principais características:

1. Recuperação Adaptativa

* O modelo decide se precisa consultar informações externas.

* Pode fazer uma ou várias recuperações durante a geração da resposta.

* Também pode não realizar busca quando considera que já possui conhecimento suficiente.

2. Autorreflexão

* O modelo utiliza tokens de reflexão especiais para analisar suas próprias respostas.

* Ele verifica se a resposta está correta, relevante e baseada nas informações recuperadas.

* Isso ajuda a reduzir erros e informações falsas.

3. Controle durante a Inferência

* Os tokens de reflexão permitem maior controle sobre o comportamento do modelo durante a geração.

* O modelo consegue ajustar sua resposta conforme a necessidade da tarefa.

- Vantagens do Self-RAG:

✅ Maior precisão factual

* A avaliação da própria resposta reduz informações incorretas ou desatualizadas.

✅ Mais flexibilidade e versatilidade

* O modelo escolhe quando buscar informações e quando revisar sua resposta, adaptando-se melhor a diferentes tarefas.

✅ Melhoria contínua

* A autorreflexão permite que o modelo avalie suas gerações e melhore a qualidade das respostas.

20 – ADAPTIVE RAG:

- Adaptive RAG (RAG Adaptativo)

-> O Adaptive RAG é uma evolução do RAG tradicional que permite ao sistema escolher dinamicamente a melhor estratégia de recuperação de informações conforme a complexidade da pergunta.

Enquanto o RAG tradicional usa sempre a mesma forma de busca, o Adaptive RAG analisa cada consulta e decide se precisa ou não recuperar informações externas, e quantas vezes deve fazer isso.

- Como funciona:

1. Classificação da complexidade da consulta

* Um classificador avalia a pergunta e identifica se ela é simples, moderada ou complexa.

2. Escolha dinâmica da estratégia

Dependendo da complexidade, o sistema escolhe:

- Sem recuperação:

Para perguntas simples, o LLM responde usando apenas seu conhecimento interno.

– Recuperação única:

Para perguntas moderadas, realiza uma busca externa uma vez para complementar a resposta.

– Recuperação iterativa:

Para perguntas complexas, faz várias buscas e combina informações de diferentes fontes até conseguir uma resposta mais completa.

- Vantagens:

* Maior eficiência: evita gastar processamento fazendo buscas desnecessárias em perguntas simples.

* Maior precisão: usa mais etapas de recuperação quando a pergunta exige mais conhecimento.

* Flexibilidade: consegue lidar com perguntas de diferentes níveis de dificuldade.

21 – Corrective RAG (CRAG):

-> O CRAG é uma evolução do RAG tradicional que adiciona avaliação e correção da recuperação de informações antes da geração da resposta. Ele verifica se os documentos encontrados são realmente relevantes e confiáveis, evitando respostas baseadas em informações incorretas.

- Como funciona:

* Um avaliador analisa os documentos recuperados e gera um nível de confiança.

* Alta confiança: usa os documentos encontrados para gerar a resposta.

* Baixa confiança: faz uma nova busca (como na web) para encontrar informações melhores.

* Confiança ambígua: combina refinamento dos documentos com novas buscas.

-> Além disso, realiza refinamento do conhecimento, separando os documentos em fragmentos relevantes e removendo informações desnecessárias ou repetidas.

- Vantagens:

✅ Maior precisão e redução de respostas erradas.

✅ Recuperação mais flexível com novas buscas quando necessário.

✅ Pode ser integrado a outros sistemas RAG para melhorar a qualidade das respostas.

- Em resumo:

CRAG avalia a qualidade dos documentos recuperados, corrige a busca quando necessário e só então gera a resposta.

22 JANELA DE CONTEXTO:

-> Modelos de Linguagem de Grande Porte com Contexto Longo (Long-ContextLLMs) são modelos de linguagem projetados para processar e gerar texto considerando um contexto extenso, frequentemente excedendo dezenas de milhares de tokens. Isso permite que eles mantenham coerência e relevância em tarefas que envolvem grandes volumes de informações ou documentos longos.

Mód 04 LLMOps:

02 Compreendendo o Conceito de LLMOps:

-> LLMOps, ou Large Language Model Operations, refere-se ao conjunto de práticas, ferramentas e processos envolvidos na gestão, implantação e otimização de modelos de linguagem de grande porte (LLMs) em ambientes de produção.

Ou seja, LLMOps é o conjunto de práticas para colocar, monitorar, atualizar, proteger, otimizar e manter LLMs funcionando com qualidade em produção.

03 Workflow para deploy de LLM e RAG:

I) Começa com a preparação da infraestrutura, escolhendo o ambiente de execução (como **AWS, Google Cloud, Azure, Hugging Face Spaces ou Databricks**), com **GPU** e um **banco vetorial** (como **FAISS, LanceDB, Milvus ou Weaviate**).

II) Depois, os documentos são convertidos em **embeddings**, indexados para busca semântica e podem passar por **Re-Ranking** para melhorar a recuperação.

III) Em seguida, o **LLM é implantado (deploy)** em **tempo real via APIs** (usando ferramentas como **FastAPI, Flask ou Triton Inference Server**) ou em **batch**, com respostas pré-processadas.

IV) Após o deploy, o sistema é **monitorado e otimizado** com ferramentas como **Prometheus, Grafana e OpenTelemetry**, além de técnicas como **quantização**, **distilação** e melhorias contínuas por feedback.

V) Por fim, é necessário garantir **segurança e conformidade**, com autenticação, autorização, criptografia e atendimento a normas como **LGPD, GDPR e HIPAA**.

04 Re- Ranking:

-> Re-ranking é tipo uma segunda passada para filtrar ainda melhor os resultados que vieram da busca. Primeiro, você faz a busca semântica normal no seu banco vetorial, pega, sei lá, os top 20 documentos, e aí passa por um modelo mais forte, que relê pergunta e documento juntos, e reordena pela relevância real.

05 MLOps x LLMops:

-> MLOps (operações de aprendizado de máquina) e LLMOps (operações de grandes modelos de linguagem).

->Ou seja, MLOps cuida de qualquer modelo de machine learning em produção. LLMops é a versão focada nos grandes modelos de linguagem, com coisas específicas de texto tipo prompts, RAG, embeddings, bancos vetoriais e controle de alucinações. É tipo, a mesma prática, só que com o tempero certo pra LLM.

18 - O Que é Elastic Search?

-> É uma ferramenta de **busca e análise de dados**. Ela é muito usada para criar sistemas que precisam encontrar informações rapidamente em grandes volumes de dados.

Ex: "vinho tinto italiano encorpado até R$ 200"

O Elasticsearch consegue encontrar rapidamente os vinhos mais relevantes porque ele não faz apenas uma busca simples por palavra exata; ele entende termos, relevância e pode considerar vários critérios.

-> O Elasticsearch não só busca, mas te dá visibilidade do que está sendo buscado, para melhorar respostas e o próprio conteúdo.

20 - O Que é e Quando Usar o Apache Airflow?

-> É uma ferramenta usada para **criar, organizar, agendar e acompanhar pipelines de dados**.

-> Ele permite criar DAGs **= Directed Acyclic Graph** (**Grafo Direcionado Acíclico**).

-> O Apache Airflow deve ser utilizado quando há necessidade de gerenciar pipelines de dados complexos, garantindo que as tarefas sejam executadas na ordem correta e com monitoramento contínuo.

30:

-> Hugging Face é uma plataforma de **modelos de inteligência artificial**.

-> Apache Airflow é uma ferramenta para **orquestrar pipelines de dados**.

-> Grafana Labs é uma ferramenta de **monitoramento e visualização de dados**. Ela cria dashboards.

-> Streamlit é uma ferramenta para criar **interfaces web rapidamente usando Python**.

-> Docker é uma ferramenta para **empacotar aplicações em containers**.

Mód 05 Parte II:

09 Métricas para medir a eficácia do módulo de RAG:

- **Hit Rate** → verifica se o sistema encontrou algo relevante (acertou ou não).
- **MRR** → verifica quão bem o sistema posicionou o primeiro resultado relevante.

-> Quanto maior o MRR, melhor o sistema organiza os resultados, pois significa que as informações relevantes aparecem mais próximas do topo da lista.

21 Automação de Pipelines RAG:

-> A automação de pipelines RAG permite que sistemas de IA **melhorem continuamente** usando feedback dos usuários e métricas de desempenho.

O processo envolve:

- **Coleta de feedback:** registra interações, avaliações e comportamentos dos usuários para identificar melhorias.
- **Armazenamento e análise:** organiza os dados em bancos tradicionais ou vetoriais para encontrar problemas e oportunidades.
- **Melhoria do sistema:** atualiza a base de conhecimento, ajusta a LLM (fine-tuning) e melhora a recuperação de contexto com técnicas como **re-ranking e embeddings**.
- **Monitoramento:** acompanha métricas como precisão, respostas úteis e **MRR** para avaliar evolução.
- **Versionamento e testes:** permite comparar versões do pipeline e validar melhorias.

-> Ou seja, pipeline RAG cria um ciclo contínuo de coleta → análise → melhoria → teste, reduzindo alucinações e tornando as respostas mais precisas e contextualizadas.

Mód 06 – SQL, LLM e RAG para sistema de recomendação persona:

02 - O que é Sistema de Recomendação

-> Um **Sistema de Recomendação** é um conjunto de técnicas e algoritmos que analisa dados dos usuários e dos itens para sugerir conteúdos relevantes com base em **preferências, comportamento e histórico**. É muito usado em **streaming, e-commerce, redes sociais** e outras plataformas.

Principais abordagens:

- **Filtragem Colaborativa:** recomenda itens com base no comportamento de usuários com gostos semelhantes.
- **Filtragem Baseada em Conteúdo:** recomenda itens parecidos com os que o usuário já gostou, analisando suas características.
- **Métodos Híbridos:** combinam as duas abordagens para aumentar a precisão.

Atualmente, esses sistemas utilizam **IA, aprendizado de máquina, redes neurais, aprendizado profundo (Deep Learning) e aprendizado por reforço** para gerar recomendações mais **personalizadas e contextuais**.

04 - Uso de SQL Para Recuperação Eficiente de Dados Estruturados:

-> Linguagem SQL é usada para recuperar dados de forma eficiente em bancos de dados estruturados, permitindo consultas otimizadas para grandes volumes de informações. A eficiência da recuperação depende de diversos fatores, como a estrutura da consulta, o design do banco e o uso correto de índices.

Algumas práticas são fundamentais:

- Seleção de Colunas Específicas;
- Uso de Índices;
- Filtragem com WHERE;
- JOINs Eficientes;
- Agrupamento e Ordenação Otimizada – O uso de GROUP BY e ORDER BY deve ser feito com moderação, pois pode exigir ordenação e processamento adicional.

- Uso de Views e Materialized Views – Permitem otimizar consultas complexas préprocessando os dados.
- Particionamento de Tabelas – Em bancos de grande escala, dividir os dados em partições pode melhorar a recuperação.

05 - Métricas e Avaliação de Desempenho em Recomendações Personalizadas:

-> A avaliação de um **Sistema de Recomendação** mede a qualidade das recomendações e ajuda a melhorar o modelo. As métricas variam conforme o objetivo do sistema.

1. Métricas de Precisão

- **RMSE e MAE:** medem o erro entre as previsões do sistema e as avaliações reais dos usuários.
- **Precision:** mede quantos itens recomendados são realmente relevantes (Top-K).
- **Recall:** mede quantos itens relevantes o sistema conseguiu recomendar (Top-K).

2. Métricas de Ranqueamento

- **NDCG:** avalia se os itens mais relevantes aparecem nas primeiras posições da lista.
- **MRR:** mede a posição do primeiro item relevante, favorecendo listas em que ele aparece logo no início.

3. Métricas de Cobertura e Diversidade

- **Cobertura:** mede quanto do catálogo foi recomendado.
- **Diversidade:** verifica a variedade dos itens recomendados.
- **Serendipidade:** mede a capacidade de recomendar itens inesperados, mas relevantes.

4. Métricas de Engajamento

- **CTR (Click Through Rate):** porcentagem de recomendações clicadas.
- **Tempo de retenção:** quanto tempo o usuário interage com os itens recomendados.
- **Conversão:** verifica se o usuário realizou a ação desejada (compra, play, leitura etc.).

--> A métrica ideal depende do objetivo do sistema. Por exemplo, sistemas de vendas priorizam **CTR e conversão**, enquanto sistemas educacionais podem priorizar **diversidade e retenção**. O uso de **várias métricas** fornece uma avaliação mais completa do desempenho.

17 – Endpoints:

-> Em uma API REST, um endpoint geralmente com caminho específico e um método HTTP, como:

- GET /users → Retorna uma lista de usuários.
- POST /users → Cria um novo usuário.
- GET /users/{id} → Retorna um usuário específico.
- PUT /users/{id} → Atualiza os dados de um usuário.
- DELETE /users/{id} → Remove um usuário.

★ RAG serve para dar melhor contexto para o LLM.

Mód 07 - Aplicação-Integrada-com-SLM:

-> Neste capítulo vamos construir um sistema de RAG para automatizar um processo específico. Trabalharemos com GPU no ambiente em nuvem do Google Colab.

13 - Por que usar banco de dados vetorial?

-> Bancos de dados vetoriais são ideais para cenários onde a busca por

similaridade em dados complexos e não estruturados é uma necessidade,

oferecendo escalabilidade e desempenho adequados para grandes volumes de dados.

15 – Quem Disse Que Precisamos Colocar Todos os Dados no RAG?

-> Não precisa colocar todos os dados no RAG, pois ele enviou somente as perguntas, então quando o usuário digitar a perguntar, então buscará perguntas similares. Quando trouxer as perguntas similares, para cada pergunta terá o ID, e no banco terá as respostas relacionadas com as perguntas.

Por que se levar tudo para o banco vetorial, ficará mais lento.

17 - Distância Cosseno (Cosine Distance):

-> A distância cosseno (Cosine Distance) é uma métrica amplamente utilizada para

medir a dissimilaridade entre dois vetores em um espaço de características. Ela se

baseia no cosseno do ângulo entre esses dois vetores, sendo particularmente útil

para dados de alta dimensão.

1: vetores têm a mesma direção (ângulo 0°), são muito semelhantes.

* 0: vetores não têm relação (ângulo 90°), sem semelhança.

* -1: vetores têm direções opostas (ângulo 180°).

19 – Definindo SLM Tokenizador Prompt e Decode:

-> O tokenizador transforma textos em tokens (palavras, partes de palavras ou caracteres), convertendo-os em números que o LLM consegue processar. Ele reduz e organiza o vocabulário, melhorando o aprendizado e o desempenho do modelo.

Cada token recebe um Token ID, formando sequências numéricas chamadas token_inputs, que são usadas pelo modelo para criar

embeddings, entender o contexto, prever o próximo token e gerar respostas.

Mód 10 – RAG:

03 Benefícios da RAG Para Aplicações Corporativas:

- Maior precisão e confiança nas respostas.
- Informações sempre atualizadas e contextuais.
- Economia de tempo e custo: Novos dados podem ser adicionados sem reentreinar o modelo, tornando a solução mais flexível e acessível.
- **Melhor experiência do usuário:** O RAG gera respostas mais relevantes, contextualizadas e personalizadas.
- **Controle e auditabilidade:** A empresa controla quais fontes de informação serão usadas, mantendo apenas conteúdos atualizados e autorizados.

4 – Infraestrutura para implementar RAG:

-> Para funcionar na prática, o **RAG não depende apenas de um LLM**. Ele precisa de uma infraestrutura com diferentes componentes: um **banco de dados vetorial** para armazenar informações, **modelos de embeddings** para transformar dados em representações numéricas e permitir buscas por significado, um **orquestrador** para conectar todos os componentes e um **LLM** para gerar as respostas.

Essa arquitetura pode ser dividida em **microserviços**, onde cada parte executa uma função específica, como busca de documentos, processamento da linguagem e geração de texto. Serviços em nuvem, como a **AWS e Azure**, oferecem ferramentas que ajudam a montar esse fluxo, conectando busca e modelos de IA por meio de APIs.

Assim, o RAG adiciona uma camada de busca e conhecimento externo ao LLM, permitindo que ele consulte informações atualizadas e relevantes em tempo real, gerando respostas mais precisas do que um LLM usado sozinho.

05 - Modelos:

a. Modelos de linguagem generativos (Large Language Models): São o motor gerador das respostas.
b. Modelos/algoritmos de recuperação (por exemplo, embeddings semânticos ou buscadores especializados): são aqueles responsáveis por converter textos em vetores numéricos (embeddings) e efetuar a busca semântica.

Mód 11 SLMs:

3 - A importância dos modelos de embeddings:

-> Os modelos de Embeddings transformam textos em vetores numéricos que representam seu significado semântico.

No RAG, eles conectam os documentos armazenados ao modelo gerador (SLM ou LLM), permitindo comparar vetores por similaridade e recuperar rapidamente as informações mais relevantes para a pergunta do usuário.

-> Assim, garantem uma recuperação mais precisa, eficiente e coerente, melhorando a qualidade e a relevância das respostas.

- Teria outra maneira de usar RAG somente com BM25 que é a busca por palavras-chave.

Mód 12 – Grafo de conhecimento:

20 - Armazenamento Vetorial com FAISS:

-> FAISS (Facebook AI Similarity Search) é uma bibiblioteca desenvolvida pelo Facebook especificamente projetada para realizar buscas rápidas por similaridade em grandes volumes de dados vetoriais. Seu principal uso é no gerenciamento de embeddings, que são representações numéricas de informações, como textos, imagens ou áudios, obtidas por modelos de aprendizado de máquina.

Mód 13 – Parte II:

15 – Motor de consultas:

-> O dsa_queryengine.py implementa o motor de consultas de um sistema GraphRAG, que combina busca vetorial com um grafo de conhecimento para responder perguntas.

- QueryEngine é a classe principal. Quando o usuário faz uma pergunta, ela primeiro realiza uma busca vetorial (em um índice como FAISS ou ChromaDB) para encontrar os documentos mais relevantes e formar um contexto inicial.
- AnswerCheck verifica se esse contexto já contém informações suficientes para responder completamente à pergunta.
- Se o contexto for suficiente, a resposta é gerada imediatamente.
- Se não for suficiente, o QueryEngine expande o contexto navegando pelo grafo de conhecimento, buscando nós relacionados.
- Essa expansão usa uma fila de prioridade (heapq), para visitar primeiro os nós com maior similaridade semântica com a pergunta.
- A cada nova informação encontrada, o AnswerCheck verifica novamente se já é possível responder.
- Se, mesmo após a expansão, não houver uma resposta completa, o modelo de linguagem (LLM) gera a melhor resposta possível usando todo o contexto acumulado.

-> O sistema começa com uma busca por embeddings (vetorial), verifica se já consegue responder e, se necessário, explora o grafo de conhecimento para enriquecer o contexto antes de gerar a resposta final.

Mód 15 – Agentic RAG e LLM routing com LangGraph:

02 - Roteamento Inteligente:

-> Roteamento Inteligente é o uso de regras, dados ou IA para decidir automaticamente o melhor destino para uma tarefa, analisando a solicitação e os recursos disponíveis.

-> Diferente do roteamento simples (regras fixas), ele escolhe dinamicamente o melhor modelo, agente ou recurso, melhorando custo, velocidade, qualidade e eficiência do sistema.

03 – LLM Routing:

-> LLM Routing é uma técnica de roteamento inteligente que direciona cada solicitação para o LLM, agente ou ferramenta mais adequado para aquela tarefa.

Ele funciona como um despachante inteligente: analisa o prompt, decide o melhor recurso e encaminha a solicitação.

- Benefícios:
  - Maior qualidade: usa o modelo especializado para cada tarefa.
  - Menor custo: tarefas simples vão para modelos mais baratos.
  - Maior velocidade: escolhe modelos mais rápidos quando possível.
  - Mais capacidades: direciona para agentes ou ferramentas com recursos específicos.
  - Mais modularidade: combina vários LLMs e ferramentas em um sistema mais eficiente.
- Funcionamento:

1. Analisa a solicitação.
2. Escolhe o LLM, agente ou ferramenta ideal.
3. Encaminha o prompt.
4. Retorna a resposta ao usuário.

- ❖ LLM Routing x Routeamento inteligente:

-> O LLM Routing é um tipo específico de roteamento inteligente aplicado a modelos de linguagem.

Enquanto o roteamento inteligente decide dinamicamente para onde enviar uma tarefa (LLM, agente, ferramenta ou recurso), o LLM Routing foca em escolher o melhor LLM ou agente de IA para cada solicitação.

--> Todo LLM Routing é roteamento inteligente, mas nem todo roteamento inteligente é LLM Routing.

6- Exemplo de Prompt Few-Shot Para LLM Routing:

Ex: Sua tarefa é classificar a solicitação do usuário e determinar o melhor destino para processá-la. Escolha uma das seguintes opções: [LLM_GERAL, LLM_CODIGO, LLM_CRIATIVO, FERRAME

NTA_MATEMATICA]

- Solicitação: "Qual a população atual do Egito?"
  Destino: LLM_GERAL
- Solicitação: "Escreva uma função em Python para calcular o fatorial de um número."
  Destino: LLM_CODIGO
- Solicitação: "Crie um poema sobre a chuva na cidade."
  Destino: LLM_CRIATIVO
- Solicitação: "Quanto é 12 * (5 + 8)?"
  Destino: FERRAMENTA_MATEMATICA

Mód 16 – Agentic RAG com LangGraph:

02 – IA Generativa multimodal:

-> IA Generativa Multimodal é um tipo de Inteligência Artificial capaz de **processar, compreender e gerar conteúdos usando diferentes tipos de dados (modalidades) ao mesmo tempo**, como **texto, imagens, áudio, vídeos e código**.

-> A IA Generativa Multimodal é uma IA capaz de **"ver", "ouvir" e "ler"**, combinando diferentes tipos de informações para **criar conteúdos novos e variados**.

Diferente de uma IA limitada a apenas uma modalidade (ex.: somente texto ou somente imagem), ela consegue:

- **Receber entradas de diferentes formatos** (ex.: uma imagem junto com uma pergunta em texto).
- **Entender a relação entre essas modalidades** (ex.: interpretar uma imagem e relacioná-la com uma descrição escrita).
- **Gerar novos conteúdos**, podendo ser:

- De uma única modalidade (ex.: criar um texto a partir de uma imagem).
- De múltiplas modalidades (ex.: criar um vídeo com narração a partir de um roteiro em texto).